IMPERIAL COLLEGE LONDON

DEPARTMENT OF ELECTRICAL & ELECTRONIC ENGINEERING



ADVANCED SIGNAL PROCESSING & MACHINE LEARNING

BENJAMIN YONG

CID: 00810048

BY1413@IC.AC.UK

Very good report, mathematically and theoretically sound throughout. Good usage of figures and tables to present and summarize results. Explanations were always clear and concise.

Course Leader:

PROF. DANILO P. MANDIC

Good table of contents, clear and tidy

CONTENTS

1 Spectrum Estimation 1
1.1 Discrete Fourier Transform Basics 1
1.2 Properties of Power Spectral Density (PSD) 2
1.2.1 Approximation in the Definition of PSD 2
1.2.2 Using DTFT to determine PSD from ACF 3
1.3 Resolution and Leakage of Periodogram-based Methods 5
1.4 Periodogram-based Methods Applied to Real-World Data 9
2 Parametric and Line Spectra 11
2.1 Correlation Example 11
2.1.1 Unbiased Correlation Estimation and Preservation of Non-negative Spectra 11
2.1.2 Frequency Estimation by MUSIC 12
2.2 Spectrum of Autoregressive Processes 15
2.3 Principal Component Analysis 16
2.4 Respiratory Sinus Arrhythmia from RR-Intervals 18
3 Adaptive Signal Processing 20
3.1 The Least Mean Square (LMS) Algorithm 20
3.1.1 Identification of AR Processes 20
3.1.2 The Leaky LMS Algorithm 22
3.2 Adaptive Step Sizes 24
3.3 Adaptive Noise Cancellation 27
4 Widely Linear Filtering & Adaptive Spectral Estimation 32
4.1 Complex LMS and Widely Linear Modelling 32
4.1.1 Frequency Estimation in Three Phase Power Systems 34
4.2 Adaptive AR Model Based Time-Frequency Estimation 37
4.3 A Real Time Spectrum Analyser Using Least Mean Square 38

1 SPECTRUM ESTIMATION

1.1 DISCRETE FOURIER TRANSFORM BASICS

a) Since a sine wave is real and periodic, its ideal Fourier magnitude spectrum will be two Dirac delta functions at $\pm f_0$, where f_0 is the frequency of the sine wave. For a sinusoid of frequency $20Hz$, the ideal magnitude spectrum is plotted and shown in **Figure 1 (left)**, which represents two Dirac delta functions at frequency of $\pm 20Hz$. In Discrete time Fourier transform (DTFT), the discrete time signal has finite energy and a sinusoid with finite energy can be obtained by multiplying the continuous sine wave with a rectangular window of finite length. Since the Fourier transform of a rectangular window is a *sinc* function, the theoretical DTFT magnitude spectrum of a sine wave will be the convolution of its ideal Fourier spectrum with the *sinc* function. The DTFT magnitude spectrum of the $20Hz$ sinusoid is illustrated in **Figure 1 (left)**, and it can be seen that the windowing operation has resulted in spectral smearing and leakage as the spectrum no longer resembles two Dirac delta functions.

clear graphs, properly labeled and titled no pictures like some of his colleagues.

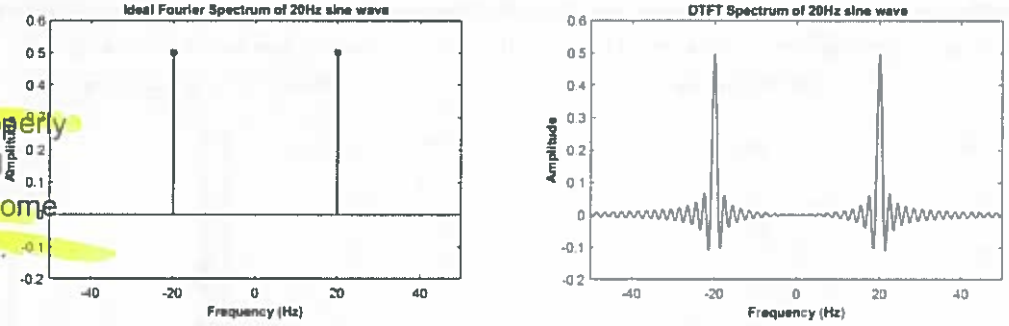


Figure 1: Ideal Fourier Transform and Theoretical DTFT of 20Hz Sine

b) A $20Hz$ sine wave of length $N = 100$ (equivalent to windowing with a rectangular window of length 100) is sampled at $1000Hz$, and the 100 point DFT spectrum is plotted and shown in **Figure 2 (left)**.

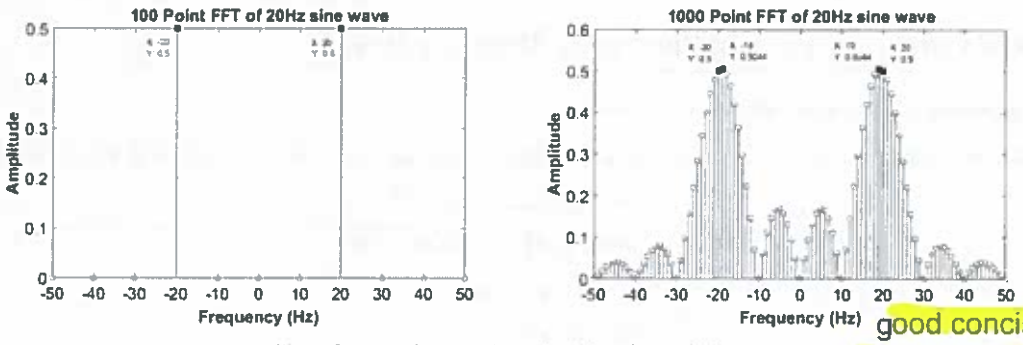


Figure 2: 100 and 1000 Point DFT Spectra for 20Hz Sine

good concise explanations, with support of formulae.

At first glance, the spectrum obtained using the 100-point DFT resembles that of the ideal Fourier spectrum shown previously. However, it is obvious that since windowing and sampling had been carried out on the infinite length continuous sine wave, the signal had already been altered and thus the two magnitude spectra cannot be identical. For a rectangular window of length N sampled at a sampling frequency f_s, the nulls its Fourier spectrum will occur at frequencies

$$f_{null} = \frac{n \times f_s}{N} \tag{1}$$

where n is an integer. Upon convolution with a sinusoid of frequency f_0, the nulls will be at frequencies of $f_0 + \frac{n \times f_s}{N}$. Next, for a K point DFT operation, the samples returned will be at frequencies

$$f_{DFT\ samples} = \frac{n \times f_s}{K} \tag{2}$$

In the simulation carried out ($K = N = 100$), the null frequencies occurs at intervals of $10Hz$ from $\pm 20Hz$, while the DFT samples are also at intervals of $10Hz$. Hence the similarity observed between the 100-point DFT spectrum and the ideal Fourier spectrum is solely due to the fact that the DFT operations returned samples at frequencies which coincides with the nulls of the *sinc* function, except at frequencies of $\pm 20Hz$.

To further investigate this, K is increased to 1000 and the DFT spectrum is shown in **Figure 2 (right)**. The spectral leakage artefacts due to windowing the signal with a rectangular window can be now properly observed and the spectrum resembles the DTFT spectrum shown previously which illustrates the convolution between a two Diracs with a *sinc* function. Furthermore, it can be observed from **Figure 2** that the amplitude at $19Hz$ is slightly greater than that at $20Hz$ (0.504 to 0.5). This is due to interference of the side lobes of the *sinc* function centred at $-20Hz$. The distortion in amplitude values is thus due to the spectral leakage introduced by the windowing process. It is especially visible in this particular simulation since the frequency separation between the original Diracs is only $40Hz$, while the frequency of the *sinc* function is $20Hz$.

c) A $24Hz$ sine wave of length $N = 100$ and $f_s = 1000Hz$ is generated and a 100-point DFT is performed on it. The resulting spectrum is shown in **Figure 3 (left)**. Two peaks at $20Hz$ and $30Hz$ are observed instead of a clear peak at $24Hz$. This is known as the incoherent sampling problem, and it is due to the fact that the frequency bins produced due to the sampling done by the DFT process does not coincide with the frequency of the signal. Based on Eq. (2), a K-point DFT for $K = 100$ produces samples at multiples of $10Hz$, and is unable to provide the frequency bin needed to capture the DFT component at $24Hz$. In order to circumvent this problem, the value of K can be increased. This translates to zero padding the input signal, such that there are now K DFT samples available, hence increasing the number of frequency bins calculated using the DFT process. The frequency spectrum of a 1000-point DFT on the same sine wave is shown in **Figure 3 (right)**. Since the frequency bins are now only $1Hz$ apart, the correct peaks can now be identified at $\pm 24Hz$.

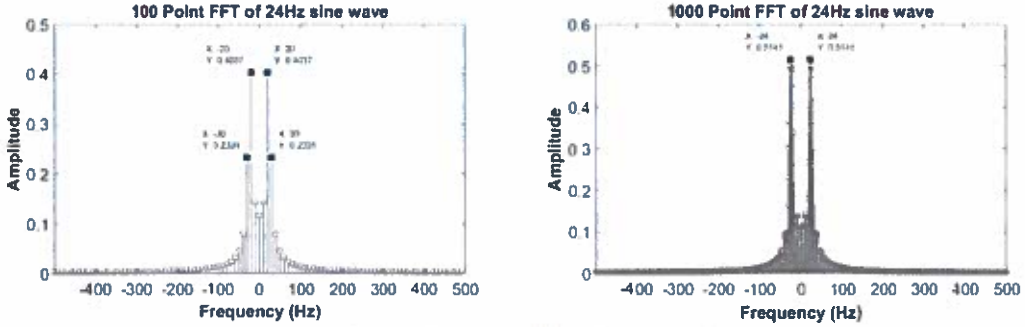


Figure 3: 100 Point and 1000 Point DFT of 24Hz Sine

1.2 Properties of Power Spectral Density (PSD)

1.2.1 Approximation in the Definition of PSD

The power spectral density of a signal can be defined in two ways, and these are shown in Eq. (3) and Eq. (4).

$$P(\omega) = \sum_{k=-\infty}^{\infty} r(k) e^{-j\omega k} \tag{3}$$

$$P(\omega) = \lim_{N\to\infty} E\left\{\frac{1}{N}\left|\sum_{n=0}^{N-1} x(n) e^{-j\omega n}\right|^2\right\} \tag{4}$$

Where $r(k)$ is the autocovariance function (ACF) and is defined as

$$r(k) = E\{x(n)x^*(n-m)\} \tag{5}$$

To show the equivalence between Eq. (3) and Eq. (4), Eq. (4) can be rewritten as

$$\begin{aligned} P(\omega) &= \lim_{N\to\infty} E\left\{\frac{1}{N}\left|\sum_{n=0}^{N-1} x(n) e^{-j\omega n}\right|^2\right\} \\ &= \lim_{N\to\infty} E\left\{\frac{1}{N}\sum_{k=0}^{N-1} x(k) e^{-j\omega k} \sum_{l=0}^{N-1} x^*(l) e^{j\omega l}\right\} \\ &= \lim_{N\to\infty} \frac{1}{N}\sum_{k=0}^{N-1} \sum_{l=0}^{N-1} E\{x(k)x^*(l)\} e^{-j\omega(k-l)} \\ &= \lim_{N\to\infty} \frac{1}{N}\sum_{k=0}^{N-1} \sum_{l=0}^{N-1} r(k-l) e^{-j\omega(k-l)} \end{aligned} \tag{6}$$

Substituting $(k-l) = \tau$, and replacing the two summations by a single summation yields

$$\begin{aligned} P(\omega) &= \lim_{N\to\infty} \frac{1}{N}\sum_{\tau=-N+1}^{N-1} (N - |\tau|)\, r(\tau) e^{-j\omega(\tau)} \\ &= \sum_{\tau=-\infty}^{\infty} r(\tau) e^{-j\omega(\tau)} - \lim_{N\to\infty} \frac{1}{N}\sum_{\tau=-N+1}^{N-1} |\tau|\, r(\tau) e^{-j\omega(\tau)} \end{aligned} \tag{7}$$

Under the assumption that the covariance sequence $r(\tau)$ decreases rapidly such that

$$\lim_{N\to\infty} \frac{1}{N}\sum_{\tau=-(N-1)}^{N-1} |\tau||r(\tau)| = 0 \tag{8}$$

the second term in Eq. (7) goes to zero and hence is exactly equal to Eq. (3). It is thus proven that both forms of the PSD are equivalent if the ACF of the signal decreases rapidly enough.

The equivalence between Eq. (3) and Eq. (4) is verified using a Dirac function, which has an ACF which is also a Dirac, meaning that it will satisfy the condition stated in Eq. (8). It can be seen from **Figure 4** that the PSD calculated using both equations are indeed equivalent.

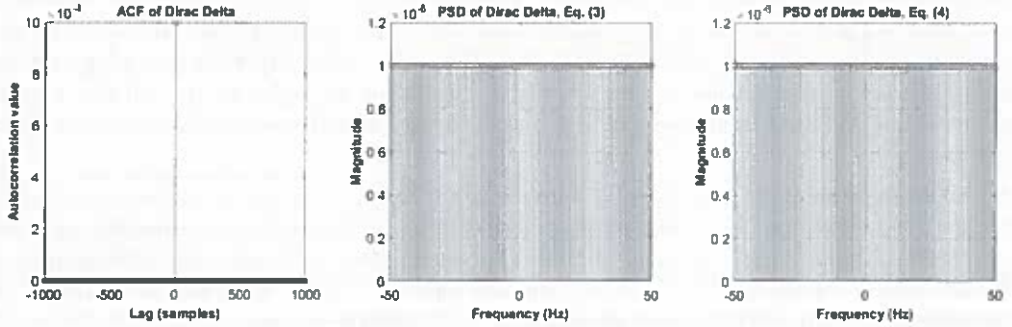
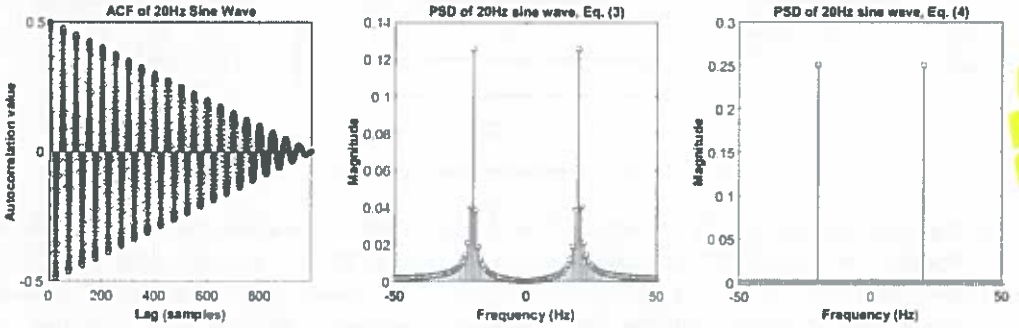


Figure 4: ACF and PSD Estimates for Dirac Delta Function

Next, a case where the equivalence does not hold is investigated. In this instance, a finite $20Hz$ sine wave is used instead. The ACF of the sine wave decays at a constant rate and is not rapid enough to satisfy Eq. (8). There is thus a discrepancy between the estimated PSDs of the sinusoid using the two methods. This is due to the fact that the second term in Eq. (7) is not zero, and hence the estimated PSD using Eq. (4) appears cleaner than that estimated using Eq. (3).



good placement of figures, in line with the text. all in eps format.

Figure 5: ACF and PSD Estimates for 20Hz Sine Wave

1.2.2 Using DTFT to determine PSD from ACF

a) Since the auto-covariance function is real and symmetric, the Fourier transform of the ACF will also be real and symmetric. This property should be preserved when the Fast Fourier Transform (FFT) function is carried out in Matlab. However, since the FFT function in Matlab is written in a way which assumes that the input data vector is wrapped about the middle sample. The particular choice of **x** where the addition of zeroes in the middle of the vector is required in order to produce results which preserves the symmetry and real-valued properties of the Fourier transform. If the zeroes are appended at the end of **x** instead, it will no longer be symmetric about the middle sample and hence, Matlab's FFT will produce complex valued solutions.

all figures in eps. good

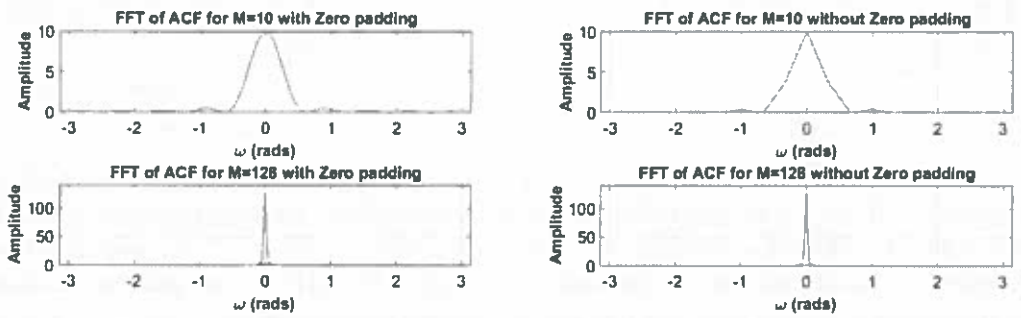


Figure 6: FFT of x with and without Zero-padding

Another important aspect of adding zeroes to the vector **x** is that it increases the frequency resolution of the FFT operation. Frequency resolution is described as the minimum frequency spacing between two discernible components in the FFT spectrum. The addition of zeroes can be seen as an increase in the window length of the input data, and this has

clear explanations, easy to follow

the effect of reducing the period of the *sinc* function introduced by windowing. This reduction in period will reduce the main lobe width of the *sinc* function, allowing finer frequency resolution. This is very different from zero-padding done by the DTFT (or FFT) process. The zero-padding carried out in DTFT does not increase the frequency resolution of the resulting spectrum. It merely increases the number of points at which the continuous DTFT is sampled, such that the number of frequency bins is increased and information of the spectrum can be obtained at more frequency values. The different effects of zero padding in **x** and zero padding in the FFT process are illustrated in **Figure 6**. When M is increased from 10 to 128, there is a large reduction in the main lobe width of the *sinc* function present. This increases the frequency resolution since the frequency spacing needed between successive peaks before distortion (due to spectra smearing and spectral leakages) occurs is much smaller. On the other hand, comparing the graphs for $M = 10$ with and without zero-padding shows that zero-padding carried out by the FFT process merely increases the number of samples, hence resulting in a smoother curve.

b) Due to the symmetry of the input signal **x**, there should not be any imaginary components present in the FFT, as explained previously. However, due to the finite precision of the Matlab software, very minute imaginary components were introduced. The FFT spectrum obtained using only the real components of the FFT result along with the spectrum obtained using the absolute value of the complex FFT result are shown in **Figure 7**. There is no significant differences observed, as the imaginary components of the FFT result are too small (10^{-15}) to affect the spectrum of the signal.

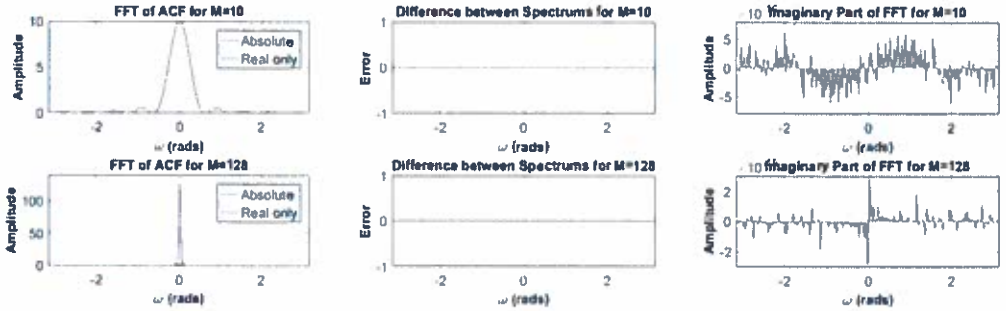


Figure 7: Comparing the Real and Imaginary Components of PSD, Symmetric Input

c) When the input signal to the FFT operation is no longer symmetric, the imaginary components increases in magnitude and becomes non-negligible (magnitude of around 10). **Figure 8** below shows the plots of the FFT spectrum when only the real components are used as well as the spectra when the absolute values are used. The latter spectrum resembles the desired FFT spectrum, while the former exhibits a completely different shape altogether, resulting in significant error values between the two, since the imaginary components present are large. This leads to the conclusion that the imaginary part of the FFT result should not be removed unless it has been ascertained that the amplitude of these components are negligible.

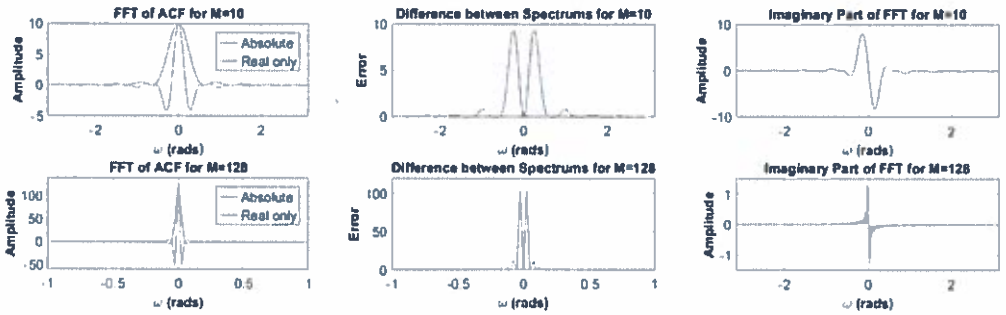


Figure 8: Comparing the Real and Imaginary Components of PSD, Non-symmetric Input

d) The `fftshift` function can be used to center both the ACF and FFT output about zero-lag and zero frequency respectively. However, when doing so, the frequency vector **w** and the lag vector **n** has to be modified as well, so that the right magnitude values can be given at each frequency of lag. If the number of ACF or FFT output samples is denoted by L, Matlab functions in a way that the zero value corresponds to the $(\frac{L}{2}+1)^{th}$ and $(\frac{L+1}{2})^{th}$ sample when L is even and odd respectively. Hence, there is a need to change the formulae used to obtain the correct **w** and **n** vectors, and these are shown in **Table 1** below.

L	w	n
Even	$-\pi:\frac{2\pi}{L}:(\pi-\frac{2\pi}{L})$	$-\frac{L}{2}:1:(\frac{L}{2}-1)$
Odd	$(-\pi+\frac{\pi}{L}):\frac{2\pi}{L-1}:(\pi-\frac{\pi}{L})$	$-\frac{L-1}{2}:1:\frac{L-1}{2}$

Table 1: Formulae for w and n vectors

Appropriate use of tables to summarize ideas

1.3 Resolution and Leakage of Periodogram-based Methods

a) Using the inbuilt Matlab function `bartlett`, the magnitude response of the Bartlett window is obtained for various values of window length N, and this is shown in **Figure 9** below. The first row of Figure 9 shows the amplitude spectrum in the log scale, while the second row shows it in linear scale. For all the plots, the 3dB point is represented by the dotted red line, while the maximum side lobe peak value is illustrated by the magenta line. It is obvious that the main lobe width of the Bartlett window decreases as N increases, while the maximum side lobe peak is invariant to N.

As before, good figures, but had they been bigger a few extra points would have been earned.

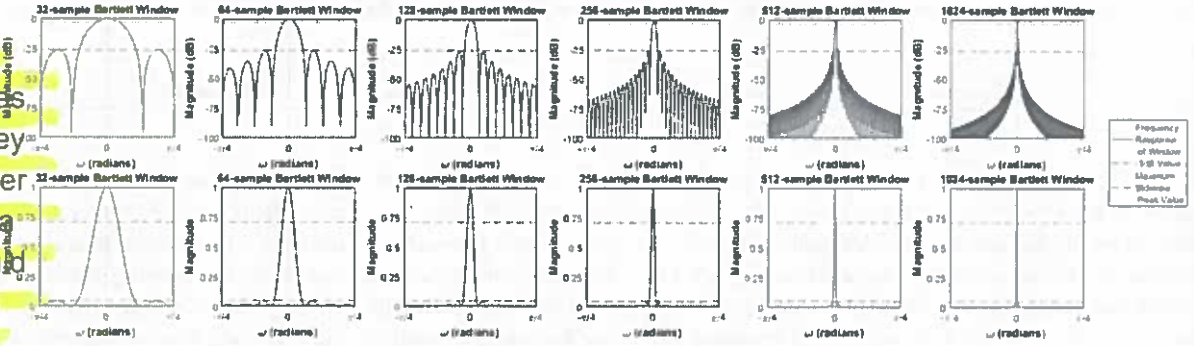


Figure 9: Magnitude Spectrum of Bartlett Window for varying N

To find the relationship between the mainlobe width and the length of the Bartlett window, the 3dB frequency is calculated for values of N from 1 to 2000. The results obtained are plotted against $\frac{1}{N}$ as shown in **Figure 10 (left)**. It can be seen that there is a linear relationship between mainlobe width $\omega_{mainlobe}$ and $\frac{1}{N}$. Plotting the line of best fit and obtaining its equation yields the relationship

$$\omega_{mainlobe} = \frac{8.11578}{N} + 0.00179 \approx \frac{1.29\times 2\pi}{N} \tag{9}$$

Which is very close to the theoretical relationship given by

$$\omega_{mainlobe,theoretical} = \frac{1.28\times 2\pi}{N} \tag{10}$$

The max peak amplitude of the side lobes of the Bartlett Window is also plotted as a function of N, and this is depicted in Figure 10 (right). It has a constant value of -26.8dB, and this is also very close to the theoretical value of -27dB.

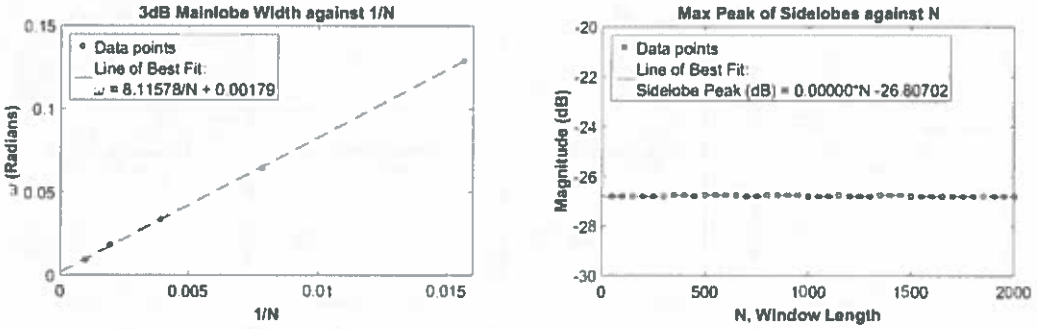


Figure 10: Empirical Relationship between Window Length and Main lobe Width, Side lobe Peak

b) The signal $x(n)$ is given by

$$x(n) = a_1\sin(f_0 2\pi n + \phi_1) + a_2\sin\left(\left(f_0+\frac{\alpha}{N}\right)2\pi n + \phi_2\right) + w(n), \quad w(n)\sim\mathcal{N}(0,\sigma^2) \tag{11}$$

In this section, $a_1 = a_2 = 1$, $f_0 = 0.2$, $N = 256$ and $\phi_1 = \phi_2 = 0$. Also, in order to study the bias properties of the periodogram, σ^2 is set to zero. α is then varied and the periodogram using a rectangular window is calculated and these plots are shown in **Figure 11**. The frequency resolution of a periodogram in terms of α is given by

$$f_{res} = \alpha \times \frac{2\pi}{N} \tag{12}$$

The resolution threshold is defined as the minimum value of α for which the two frequency components in Eq. (11) can no longer be distinguished from each other, and it can be visually observed that for values of $\alpha < 0.67$ only one peak can be distinguished. Peak detection is then carried out in Matlab using the `findpeak` function and the minimum value of α is calculated to be 0.64. This means that the frequency resolution of the periodogram obtained using a rectangular window of length 256 is $0.0157Hz$.

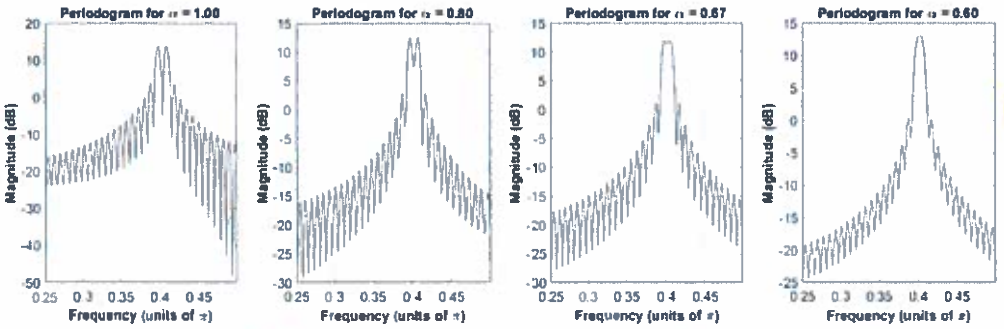


Figure 11: Periodograms using Rectangular Window for varying α

good explanations, in line with theory.

c) The steps taken in part (b) above are repeated using a hamming window instead of a rectangular window. The periodograms obtained for varying values of α are shown in **Figure 12**. The minimum value of α which allows the two peaks to be distinguished was calculated to be 0.72. This means that the periodogram with a hamming window has a frequency resolution of $0.0178Hz$, which is slightly worse than that of the rectangular window. The hamming window has a wider main lobe than the rectangular window and the greater spectral smearing leads to larger interferences between the main lobes at each of the two frequencies, resulting in a worse frequency resolution. The Hamming window however has much lower side lobes which will greatly reduce the effects of spectral leakages.

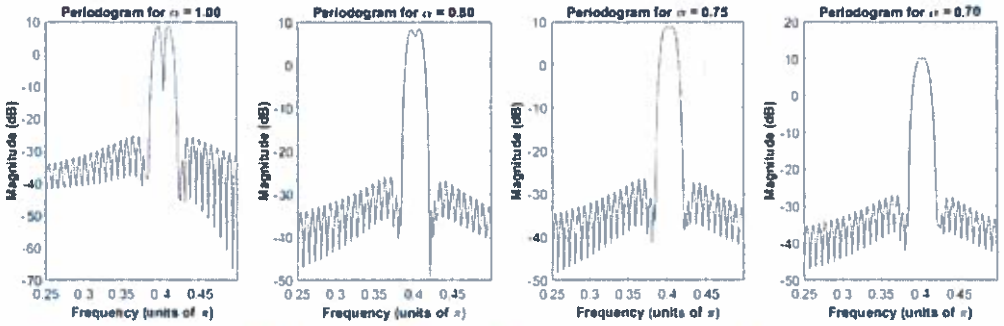


Figure 12: Periodograms using Hamming Window for varying α

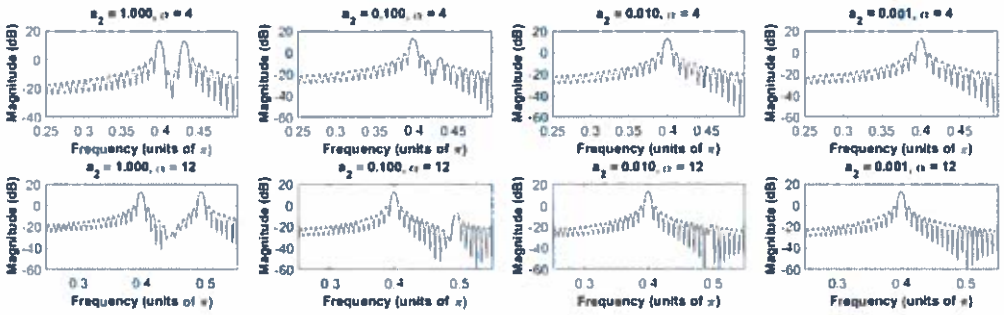


Figure 13: Periodograms using Rectangular Window for varying a_2 and $\alpha = 4, 12$

d) Spectral Leakage is due to the presence of side lobes which are an artefact of the windowing process. Leakage can be easily visualised when estimating two sinusoidal components which are well separated in frequency but have differing amplitudes. Hence, in this section, α is kept constant while a_2 is varied to observe the effects of spectral leakage. The periodograms (rectangular window) obtained when a_2 is varied are obtained and shown in **Figure 13** below, for $\alpha =$ 4 and 12. It can be seen that when a_2 is increased, the ability of the periodogram to distinguish between the two peaks deceases rapidly, such that the second peak is no longer discernible when $a_2 = 0.01$. This is due to the fact that although the rectangular window has a very narrow main lobe, it has very high side lobe values. This means that if the amplitude of

a sinusoidal component is significantly lower than that of another sinusoidal component present in the signal, the presence of the former may be completely masked by the side lobes of the latter.

Next, when α is increased from 4 to 12, it can be seen that the second peak can still be identified when $a_2 = 0.01$. This is because the side lobes of the rectangular window decreases exponentially. Since the second sinusoidal component has a frequency that is significantly different from the first, it can still be identified even when its amplitude is low due to the smaller leakages.

e) Since the input data used to calculate the periodogram is multiplied by a rectangular window, the correlation is multiplied by a triangular window, which is the Bartlett window. The expected value of a periodogram can thus be defined as the convolution between the PSD of the signal and the Fourier transform of the Bartlett window. Hence, in order to explain the reason behind a better frequency identification ability when $\alpha = 12$ as compared to $\alpha = 4$, the Fourier transform of the Bartlett window will be analysed. The Fourier transform of the Bartlett window is a $sinc^2$ function and is plotted in **Figure 14**. If the frequencies of two sinusoidal components in the signal is greater than $\frac{2f_s}{N}$, the side lobes of the larger sinusoid may conceal the main lobe of the smaller sinusoid. At frequencies of $\frac{4f_s}{N}$ and $\frac{12f_s}{N}$, the side lobes have values of $-63dB$ and $-86dB$ respectively. Therefore, the side lobe at $\frac{4f_s}{N}$ will be able to shroud a sinusoid of a higher amplitude as compared to the side lobe at $\frac{12f_s}{N}$, giving an explanation as to why a sinusoid with smaller amplitude can be identified when $\alpha = 12$ as compared to when $\alpha = 4$.

Excellent explanation, with appropriate support of graph.

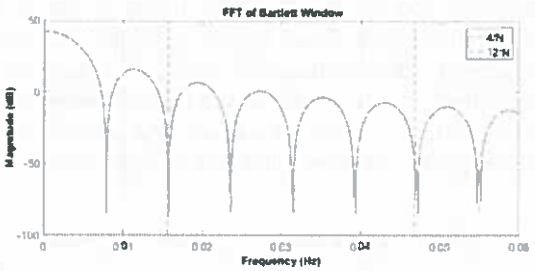


Figure 14: Fourier Transform of Bartlett Window

f) The Chebyshev window is a window which has a constant side lobe level which can be chosen by the user. The experiment carried out previously is repeated using a Chebyshev window and the periodograms obtained are shown in **Figure 15**. It can be observed that as compared to the rectangular window, the Chebyshev window is able to identify both sinusoids for $\alpha = 12$ even when the amplitude of the second sinusoid is very low ($a_2 = 0.001$). However, on the other hand, when $\alpha = 4$, the Chebyshev window performs a lot worse than the rectangular window.

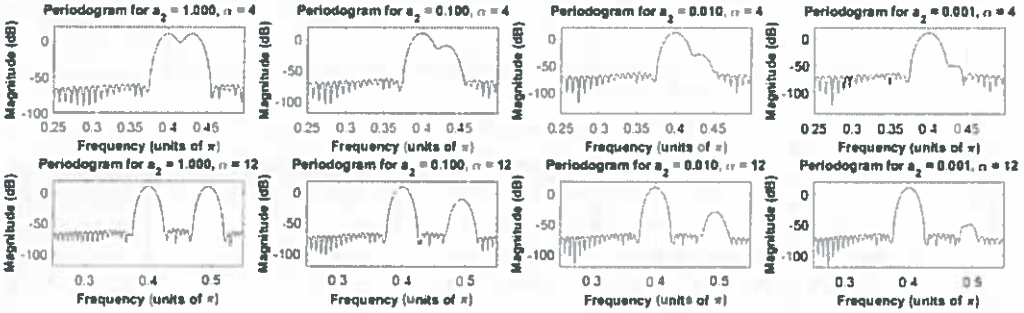


Figure 15: Periodograms using Chebyshev Window for varying a_2 and $\alpha = 4, 12$

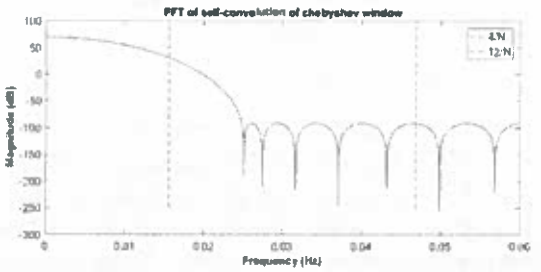


Figure 16: Fourier Transform of the Self-Convolution of the Chebyshev Window

In order to better explain the two observations made, the Fourier transform of the self-convolution of the Chebyshev window can be investigated (**Figure 16**). From the FFT plot, it can be seen that when $\alpha = 4$, the main lobe of the second sinusoid will overlap with that of the first, and vice versa. Hence, there is spectral smearing, resulting in the inability to identify the two frequency peaks. On the other hand, when $\alpha = 12$ the main lobe of the second sinusoid is now overlapping with the side lobe of the first sinusoid. The interfering side lobe has a side lobe peak of $-100dB$, which is much lower than that of the rectangular window and thus the distortions due to spectral leakage are smaller for the Chebyshev window. This results in the better identification performance of the Chebyshev windowed periodogram when $\alpha = 12$. The rectangular window exhibits very little spectral smearing due to its narrow main lobe but has large spectral leakage. The Chebyshev window on the other hand has very large spectral smearing due to a wide main lobe, but minimal spectral leakage due to the highly suppressed side lobe levels. This highlights the very important trade-off between spectral smearing and leakage in window design.

Exhaustive reply A modified periodogram method, known as the Blackman-Tukey method, can be employed. Since the input data used are of finite lengths, the autocorrelation estimates which are used to obtain the periodogram are unreliable at large lags, since only a few samples are used to calculate the estimates. This increases the variance of the periodogram. To counter this, a process known as "periodogram smoothing" can be employed where the variance can be decreased by only using robust ACF estimates. This is done in the Blackman-Tukey method by applying a window of length M, where $M < N$, to the estimated values of ACF such that the contribution of unreliable estimates to the periodogram is reduced. The Blackman-Tukey method is implemented in Matlab, and using the same input signal as before, Blackman-Tukey periodograms are obtained for varying a_2 and α, with $M = 100$. These are shown in **Figure 17**. Compared to the Chebyshev windowed periodogram, the Black-man Tukey version performs worse at every value of α and a_2. This is however, to be expected since the Blackman-Tukey method is not designed to increase the resolution of the periodogram, but to decrease its variance. This reduction in variance comes at a cost of resolution since less samples are used in the calculation of the periodogram. In this experiment where the estimated periodograms are deterministic due to the absence of noise in the input signal, the Blackman-Tukey method is not useful as there is no variance to reduce and the frequency distinguishing ability has suffered.

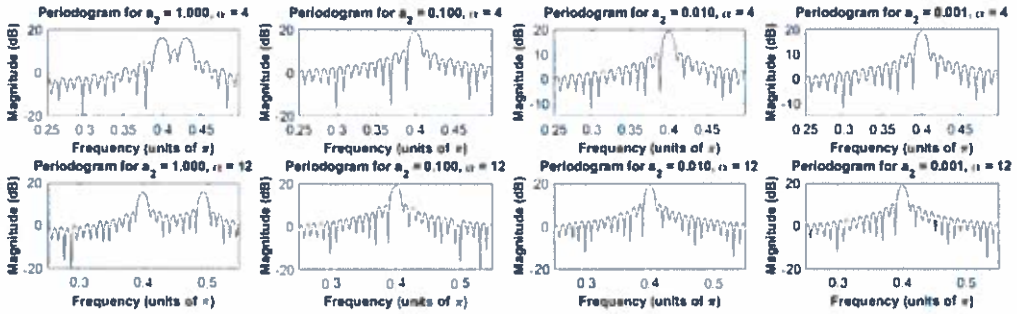


Figure 17: Periodograms using Blackman-Tukey Method for varying a_2 and $\alpha = 4, 12$

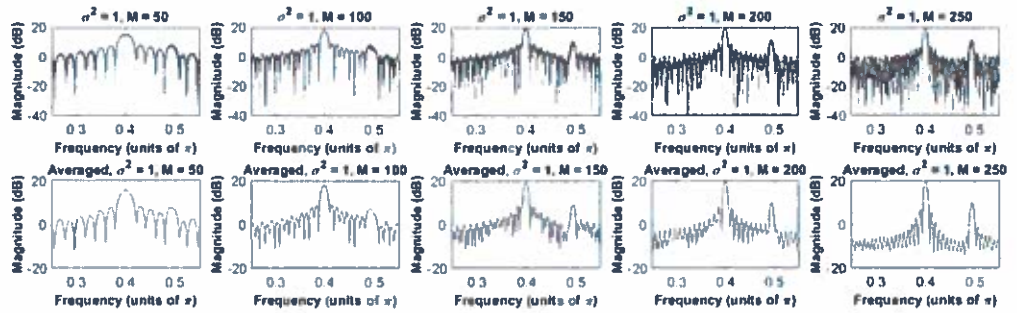


Figure 18: 50 Realisations and Averaged Blackman-Tukey Periodograms with varying M

In order to examine the usefulness of the Blackman-Tukey method, noise is introduced into the input signal $x(n)$ by setting $\sigma^2 = 1$, while $\alpha = 12$, $a_1 = \sqrt{10}$ and $a_2 = 1$ (instead of reducing a_2, due to the presence of noise and so that the signal has a decent SNR, the value of a_1 is increased instead). The Blackman-Tukey method is then used to calculate the periodograms with varying M. The top row of **Figure 18** shows the periodograms for 50 realisations of $x(n)$ using the specified parameters, while the bottom row illustrates the periodograms obtained by averaging the 50 different realisations. When $M = 50$, the overlay of 50 periodograms closely resembles that of the averaged periodogram, and as M is increased to 250, greater and greater deviation from the averaged value is observed. This shows that the Blackman-Tukey method is indeed capable at reducing the variance of the periodograms by discarding unreliable estimates of the ACF. The trade-off

between variance reduction and resolution can also be gleaned from the averaged periodogram plots of **Figure 18**. When M is increased, although the variance increases, the peaks of the two sinusoids also become both narrower and sharper, due to the increase in frequency resolution due to a greater number of ACF samples used, which translates to a longer window.

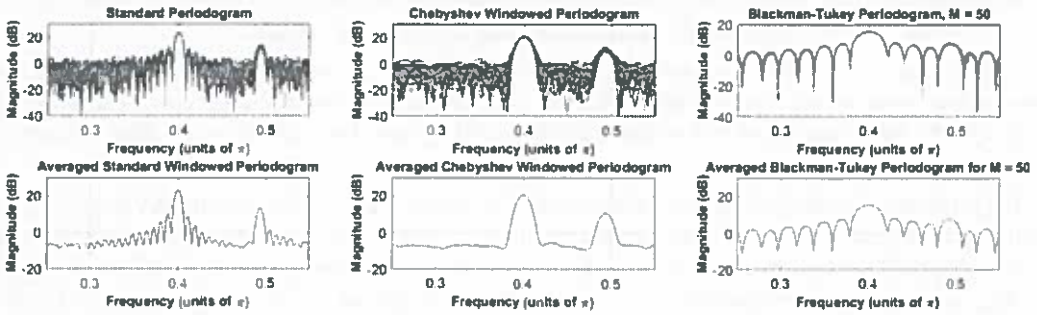


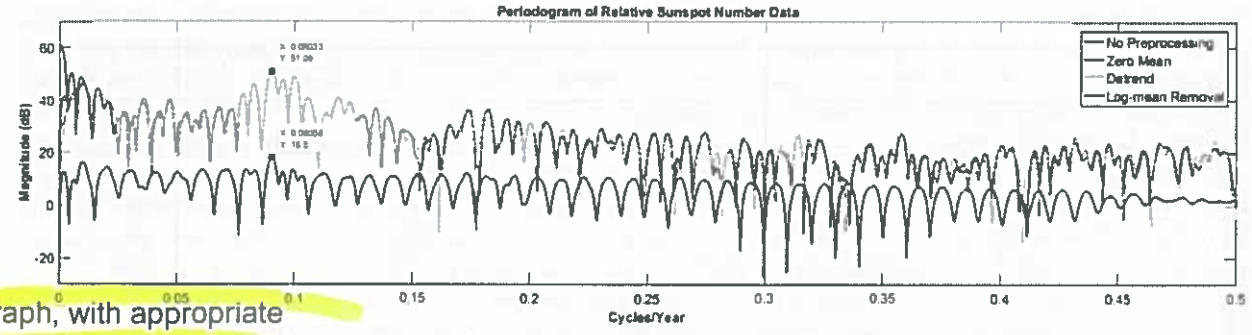
Figure 19: 50 Realisations and Averaged Periodograms for the Standard, Chebyshev windowed and Blackman-Tukey Method

To conclude this section, periodograms are calculated for 50 realisations of $x(n)$ using the standard method, the Chebyshev windowing method and the Blackman-Tukey method. The results are plotted and shown in **Figure 19**. It can be seen that for both the standard and Chebyshev windowed periodogram, the variance is very high while in the Blackman-Tukey method, the variance is almost negligible. The averaged Blackman-Tukey method however exhibits the worst frequency resolution, while the standard periodogram has the best resolution. There is almost an absence of side lobes in the averaged Chebyshev windowed periodogram however, as compared to its two counterparts. The six plots in **Figure 19** very nicely sum up the two very important trade-offs in periodogram design: the trade-off between low spectral smearing and low spectral leakage and the trade-off between low variance and high frequency resolution.

1.4 Periodogram-based Methods Applied to Real-World Data

a) The periodogram method using a rectangular window is applied to the sunspot series data with and without pre-processing. Three pre-processing schemes are explored, and they are namely mean removal, de-trend, and log-mean removal. The periodograms obtained are shown in **Figure 20**. For the detrend and mean removal pre-processing schemes, the periodograms obtained are similar across the entire spectrum except in the 0.01 cycles/year region. Since detrend essentially removes the mean as well as any linear relationship in the data set, the presence of a slight difference in magnitude values at a low frequency means that the linear relationship present in the original sunspot number series has a gradual gradient.

Next, comparing the periodogram when no pre-processing is carried out with the ones obtained after pre-processing shows a large difference in the DC component of the periodogram. The removal of the mean (and detrending) removes the spectral components of the signal close to zero frequency. A signal with a non-zero mean can be seen as the summation of a rectangular function and the same function with the mean removed. The Fourier transform of a signal with non-zero mean can thus be separated into the superposition of the Fourier transform of a rectangular function and that of the zero-mean function. Since the Fourier transform of a rectangular function is a Dirac at zero frequency, removing the mean from the signal removes the DC component, as seen in the periodogram plots.



Good graph, with appropriate labels and description.

Figure 20: Standard Periodogram for Sunspot Number with and without Pre-processing

Lastly, in the log-mean removal pre-processing scheme, the logarithm of the signal is first taken. After which, the mean is subtracted from the log data. In the sunspot number series, in order for the logarithm to be taken, a very small non zero value (`eps` in Matlab) is added to the data since there are some zero valued entries which results in an infinite value. From **Figure 20**, this gives a completely different periodogram as compared to the other pre-processing schemes. The

similarity between this pre-processing scheme and the other schemes is that it too removes the DC component present. Due to the nature of the logarithm, the log-mean removal method is able to amplify signal components which have amplitude between zero and one, while attenuating components which have an amplitude greater than one. This has the effect of evening out the periodogram, and it now resembles the periodogram of a windowing function with constant side lobe attenuation, allowing the most significant frequency component to be easily identified.

All three pre-processing schemes are able to remove the spurious low frequency artefacts, improving the frequency identification process via the use of the periodogram. From all three periodograms, the frequency of the dominant frequency component is 0.09 cycles/year, equating to about a cycle every 11 years. This coincides with the frequency of the solar cycle, as expected.

b) The periodogram method is used to identify the steady state visual evoked potential (SSVEP) from an EEG signal recorded from an electrode located at the posterior/occipital (POz) region of the head. Using the standard method and the modified periodogram method with a hamming window, the resulting periodograms are obtained and shown in **Figure 21**. From the plot, the peaks corresponding to the SSVEP and its harmonics are observed at frequencies $f = 13, 26$ and $39Hz$. The $13Hz$ peak has the highest frequency since it is the fundamental frequency of the SSVEP. Furthermore, the strong power-line interference of $50Hz$ can also be observed. The same peaks can be observed as well using the modified periodogram method, although the peak corresponding to the $50Hz$ power-line interference is now significantly lower.

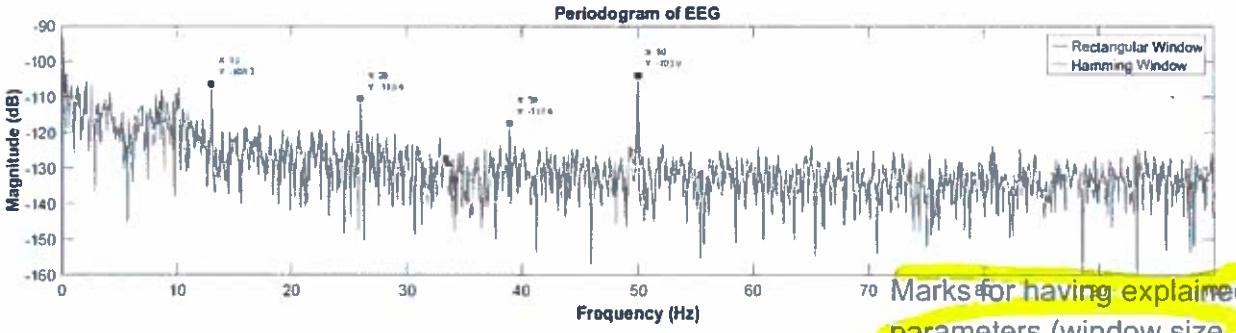


Figure 21: Standard Periodogram of EEG Signal

Marks for having explained the parameters (window size, overlap, etc) used in the periodogram estimation

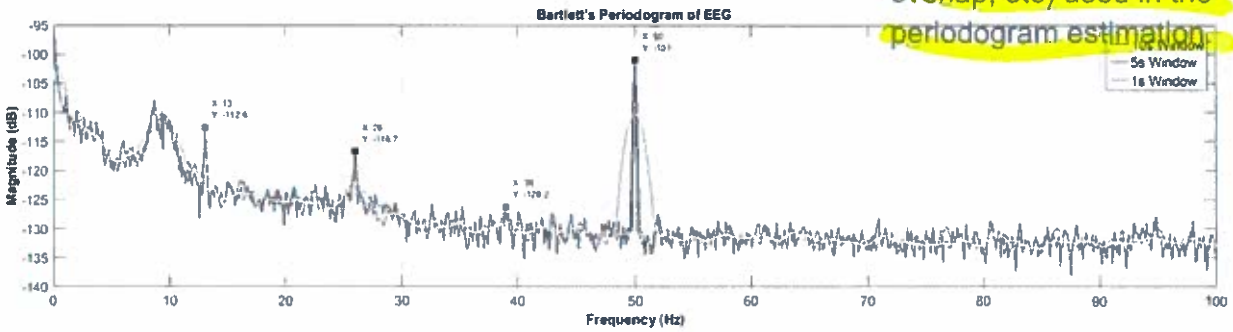


Figure 22: Bartlett's Periodogram of EEG Signal with varying Window Lengths

Next, the Bartlett's periodogram is used to analyse the EEG signal. The Bartlett's periodogram method is used to reduce the variance of the periodogram estimate. It does so by first segmenting the signal into non-overlapping subsets of a specified length, and subsequently calculating the periodogram of each of these segments. Finally, the Bartlett periodogram is obtained by taking the average of the periodograms of every segment. The Bartlett's periodogram is plotted in **Figure 22** for various window sizes, and it can be seen that for each window length chosen, the Bartlett's periodogram is a lot less noisy and exhibits less variance than the standard periodogram (**Figure 21**), which allows for an easier detection of the peaks associated with the SSVEP and its harmonics. As the length of each segment decreases it can be observed that the variance of the periodogram also decreases, resulting in a smoother periodogram. However, since each periodogram used to obtain the average is calculated using a smaller number of samples, the frequency resolution decreases. This is especially evident when the window length is set to $1s$. Although the peaks of the second harmonic of the SSEVP can still be discerned, its fundamental frequency component and third harmonic are lost. The third harmonic is masked by the spectral leakage effect of the strong $50Hz$ main interference, while the fundamental frequency response peak of the SSEVP is consumed by the large alpha-rhythm response present in the $8 - 10Hz$ range. The trade-off between good frequency resolution and low variance is again evident, and based on the plots obtained, the $5s$ window length strikes the best compromise between the two, producing a smooth periodogram which retains the peaks of interest.

2 PARAMETRIC AND LINE SPECTRA

2.1 CORRELATION EXAMPLE

appropriate theoretical introduction

2.1.1 Unbiased Correlation Estimation and Preservation of Non-negative Spectra

a) Using the correlation-based definition of the PSD leads to the correlogram spectral estimator, which is given by

$$P(\omega) = \sum_{k=-(N-1)}^{N-1} \hat{r}(k) e^{j\omega k} \tag{13}$$

The estimated autocorrelation function $\hat{r}(k)$ can be computed using the biased or unbiased estimators, given by

$$\textbf{Biased:} \quad \hat{r}(k) = \frac{1}{N}\sum_{n=k+1}^{N} x(n)x^*(n-k) \tag{14}$$

$$\textbf{Unbiased:} \quad \hat{r}(k) = \frac{1}{N-k}\sum_{n=k+1}^{N} x(n)x^*(n-k) \quad (0 \le k \le N-1) \tag{15}$$

A Matlab script is written which computes the correlogram of a signal using both the biased and unbiased estimates of the ACF. Three different signals are used to investigate the effects of the two different types of ACF estimates on the correlogram, and they are a white Gaussian noise (WGN) signal with $\sigma^2 = 1$, a sinusoidal signal of frequency $200Hz$, and a WGN signal with $\sigma^2 = 1$ filtered with a moving average (MA) rectangular window of length 5. The ACF of the signals are plotted and shown in the upper row of **Figure 23**, while their corresponding correlograms are illustrated in the bottom row. As evident from the figure, the unbiased and biased estimates of the ACF function are equal when the lag is small. However, when the lag increases, the two estimates start to diverge. This is due to the fact that the unbiased estimate uses a factor of $\frac{1}{N-k}$ while the biased estimate uses a factor of $\frac{1}{N}$. As the lag increases and the estimate ACF grows increasingly inaccurate due to the usage of a smaller number of data points in its calculation, the biased estimator is able to suppress these inaccurate ACF estimates while the unbiased estimator is unable to do so. The presence of the different multiplicative factors for both estimators can also be seen as a windowing operation. The unbiased estimator windows its ACF estimates with a rectangular window, while the biased estimator windows its ACF estimates with a Bartlett window, and this is especially evident in the ACF plot for the noisy sinusoid.

It has been established that the biased estimator is able to suppress the erroneous ACF estimates at large lags, while the unbiased estimator is not. This has a large impact on the accuracy of the correlogram obtained using these estimates. The correlograms obtained using the biased estimators closely resembles the expected periodograms of the signals (almost flat spectrum for WGN, two distinct Diracs for the noisy sine and a windowed spectrum for the filtered noise). The unbiased estimator on the other hand produced correlograms which are inaccurate and this is mainly due to the use of the erroneous ACF estimates at large lags. Furthermore, the correlogram produced using the unbiased estimator results in negative power values, which is impossible. This is due to the structure of the estimated ACF using the unbiased estimator. While the biased estimator will produce estimated ACF for random sequences which are positive definite, the unbiased estimator can result in estimated ACF which are not positive definite. The autocorrelation matrix has to be positive definite in order to ensure that the calculated power values are always positive.

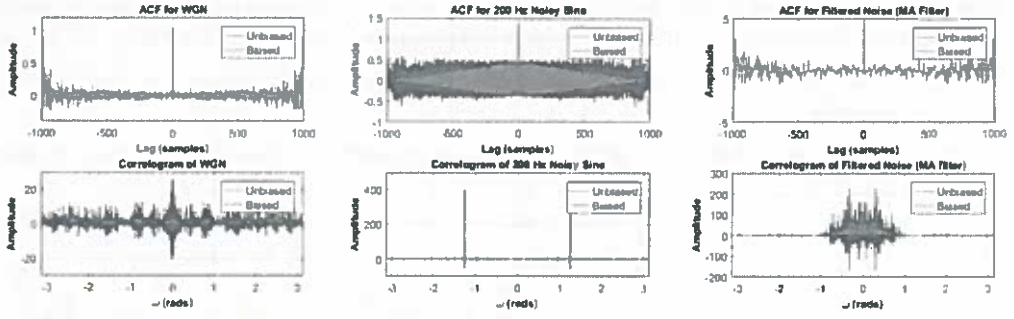


Figure 23: Unbiased and Biased ACF Estimates of WGN, Sine and Filtered WGN and their respective Correlograms

Graphs are good but could have been slightly bigger.

b) Using the biased ACF estimator, the PSD estimates of 100 realisations of two different random processes with different parameters are obtained. The top row of **Figure 24** illustrates the PSD estimates of the different realisations, while the bottom row illustrates their standard deviation. Firstly, as noise power is increased, the standard deviation of the estimated PSD is also increased, as expected. Secondly, it can be observed that the standard deviation of the PSD estimates are greatest at frequencies where the power spectrum has the highest amplitude values (corresponding to the frequencies of the sinusoids used). This is because the PSD estimator has standard deviation which is proportional to the value of the PSD and this is one of the major drawbacks of the estimator. The confidence interval, defined by $\hat{P}(\omega) \pm \mu\hat{\sigma}_{P(\omega)}$ where

$\hat{P}(\omega)$ is the mean and $\hat{\sigma}_{P(\omega)}$ is the standard deviation of the estimated PSD, widens at regions where the PSD increased, but it is exactly at these frequencies of interest where an accurate PSD estimate is required.

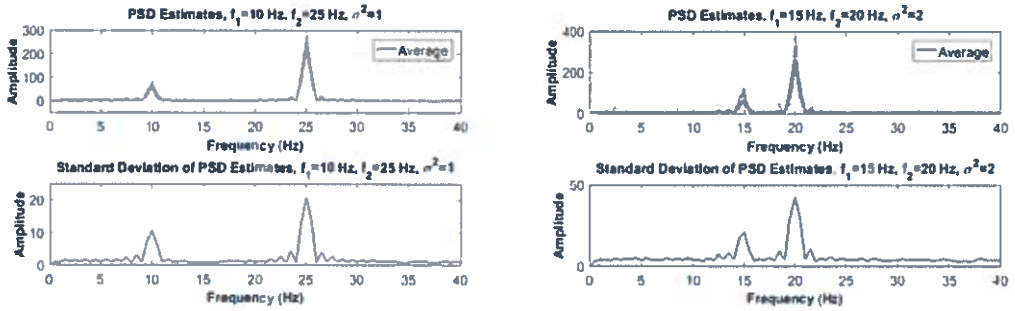
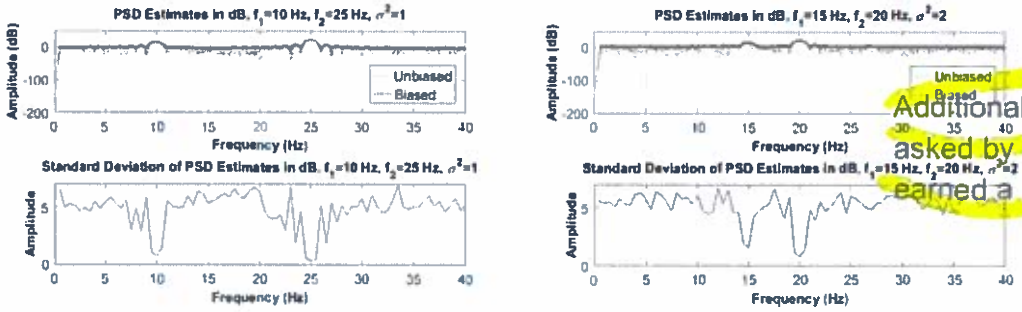


Figure 24: PSD Estimates for 100 Realisations of 2 Random Processes and their corresponding Standard Deviation

c) The PSD estimates obtained previously are plotted on a logarithmic scale in **Figure 25**, along with the standard deviation of the logarithmic data. From the PSD estimates plotted, it can be seen that now the amount of variance about the peaks of the estimates is a lot smaller than the amount of variance at the pure noise frequencies. This is to be expected since small fluctuations around a large value in the logarithmic scale will be insignificant while small fluctuations around a small value (< 1) will be amplified. This is corroborated by the dipping of the standard deviation values at the frequencies of interest (f_1 and f_2 in both plots). This means that in using the logarithmic representation of the estimated PSD the confidence interval narrows at frequencies which are of particular interest, which is desirable since the relative power of different frequency components in a signal can be confidently ascertained.



Additional graphs (not asked by the report) earned a few more marks.

Figure 25: PSD Estimates for 100 Realisations of 2 Random Processes and their corresponding Standard Deviation in dB

2.1.2 Frequency Estimation by MUSIC

d) A signal which consist of two complex sinusoids of frequencies $0.3Hz$ and $0.32Hz$ and additive complex WGN with zero mean and $\sigma^2 = 0.2$ are generated. Spectral estimates using the periodogram with a rectangular window and 100 frequency bins are obtained for signals of different lengths N and plotted in **Figure 26** below. Firstly, the periodograms are now plotted from DC to f_s instead of $\frac{f_s}{2}$ as in the previous sections because the periodograms are no longer symmetric since the input data is now complex.

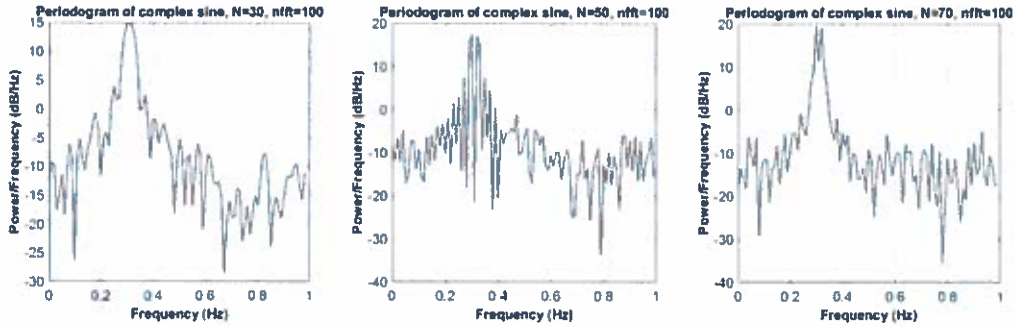


Figure 26: Periodogram of Complex Exponentials with different Signal Length

As mentioned previously, since the frequency separation between the two sinusoids is $0.2Hz$, a signal length of at least $N = \frac{f_s}{0.2}$ is needed. Since $f_s = 1$, the minimum required value of N is 50. This is corroborated by the periodogram plots, as the two frequencies are indistinguishable when $N = 30$, but can be discerned when $N = 50$ or 70. Furthermore,

the two peaks at frequencies $0.3Hz$ and $0.32Hz$ are more distinct when $N = 50$ as compared to $N = 70$. This is because when $N = 50$, the frequency bins of the periodogram coincides with the troughs of the windowed sinusoids, and hence a better response is obtained.

e) The MUtiple Signal Classification (MUSIC) method is a subspace method which utilises Eigen analysis for frequency estimation. In order to understand the lines of code provided, the MUSIC method will firstly be explored. Consider a random process $x(n)$ of length M, which consists of p complex sinusoids in the presence of complex AWGN:

$$x(n) = \sum_{i=1}^{p} A_i e^{jn\omega_i} + w(n), \quad w \sim \mathcal{N}(0, \sigma^2) \tag{16}$$

This can be rewritten in matrix form as

Excellent theoretical presentation, consistent with symbols and structure. Easy to follow

$$\mathbf{x} = \mathbf{EA} + \mathbf{w} \tag{17}$$

where $\mathbf{E} = [\mathbf{e}_1, \dots, \mathbf{e}_p]$ and $\mathbf{A} = diag(A_1, \dots, A_p)$. Taking the autocorrelation of $\mathbf{x}$, the autocorrelation matrix can be written as

$$\mathbf{R}_{xx} = E(\mathbf{x}\mathbf{x}^H) = \mathbf{E}\mathbf{P}\mathbf{E}^H + \sigma^2\mathbf{I} \tag{18}$$

where $\mathbf{P} = diag\left(|A_1|^2, \dots, |A_p|^2\right)$ and $\mathbf{R}_{xx}$ is a $M \times M$ matrix with full rank. $\mathbf{E}\mathbf{P}\mathbf{E}^H$ is referred as the signal autocorrelation, and has is of rank p, while $\sigma^2\mathbf{I}$ is the noise correlation matrix with rank M. Performing eigen-decomposition on the matrix $\mathbf{R}_{xx}$, the p largest eigen values will correspond to the p sinusoids present in the signal while the remaining $M - P$ eigenvalues will correspond to the noise present. The eigenvectors associated with the p largest eigenvalues will span the signal subspace, while the remaining eigenvectors will span the noise subspace. Since $\mathbf{R}_{xx}$ is a Hermitian matrix, all of its eigenvectors are orthorgonal to each other. This means that each vector in the signal subspace is orthogonal to each vector in the noise subspace, and this can be represented as

$$\mathbf{e}_i^H \mathbf{v}_k = 0 \begin{cases} i = 1, \dots, p \\ k = p+1, \dots, M \end{cases} \tag{19}$$

where $\mathbf{v}_k$ denotes the noise eigen vectors and $\mathbf{e}_i = \left[1, e^{j\omega_i}, \dots, e^{j\omega_i(M-1)}\right]^T$. It is now possible to formulate a way of obtaining the frequencies of the p sinusoids present in the signal. Since it is known that a signal vector will be orthogonal to the noise subspace, the PSD estimation can be performed as

$$\hat{P}_{sub}(\omega) = \frac{1}{|\mathbf{e}^H \mathbf{v}_k|^2} \tag{20}$$

where $\mathbf{e} = \left[1, e^{j\omega}, \dots, e^{j\omega(M-1)}\right]^T$ and $\omega = \frac{2\pi n}{f_s}$ for $n = (0,1, \dots, L-1)$, L is the number of FFT samples and f_s is the sampling rate of the signal. This means that for each frequency bin in the PSD estimation, Eq. (20) is calculated and if the frequency of the frequency bin corresponds to that of a sinusoid in the input signal, the denominator of Eq. (20) will tend to zero and a peak will be observed in $\hat{P}_{sub}(\omega)$ for $\omega = \omega_i$. This means that the Eq. (20) can also be seen as a pole-zero transfer function, and there will be p poles which lie on the unit circle and these poles will correspond to the frequency of the sinusoids in $x(n)$. However, the problem with this formulation of the subspace PSD estimation is that the remaining $M - p - 1$ poles can lie anywhere within the unit circle, resulting in the presence of spurious peaks at certain frequencies in the estimated PSD if they lie very close to the unit circle. These peaks may then be mistaken as the frequencies of sinusoids in the input. The MUSIC PSD estimation technique circumvents this problem by taking the summation of the dot product of all the noise vectors with $\mathbf{e}$, which can be written as

$$\hat{P}_{MUSIC}(\omega) = \frac{1}{\sum_{k=p+1}^{M} |\mathbf{e}^H \mathbf{v}_k|^2} \tag{21}$$

This process removes the presence of spurious peaks appearing in the estimated PSD spectrum since a peak will now only occur if the vector corresponding to a specific frequency bin is orthogonal to all the noise vectors of $x(n)$. By iterating Eq. (21) through the all the frequency values of the frequency bins, the spectrum estimated by the MUSIC algorithm can be obtained.

With a better understanding of the MUSIC algorithm, the operation of the Matlab code becomes relatively easier. The first line of the code returns an estimated autocorrelation matrix of size M, which will correspond to the $\mathbf{R}_{xx}$ matrix used in the MUSIC algorithm. M is set to 14 in this case, so the autocorrelation matrix obtained will have a dimension of $\mathbf{R}_{xx} \in \mathbb{R}^{14\times14}$. The input argument 'mod' specifies that the autocorrelation matrix estimated uses forward and backward prediction error estimate, and is essential because it is the only option which will return an autocorrelation matrix which is of the form $E(\mathbf{x}\mathbf{x}^H)$ as it is the only method which utilises the complex conjugate of the input signal $\mathbf{x}$. The second line of the code performs the MUSIC algorithm and the pseudo spectrum is returned in the variable `S`, while the variable `F` contains the vector of frequency values evaluated. The first argument to the function is the input, the second is the number of sources p, the third is the number of FFT samples L (the default is 256 samples) and the last argument '`corr`' specifies that a correlation matrix $\mathbf{R}_{xx}$ is used as the input to the function. The third line plots the pseudo-spectrum, while specifying the frequency range to be shown.

Using the code provided, the MUSIC algorithm is used to obtain the pseudo-spectrum of the complex exponential signal generated in part d). The various parameters of the MUSIC algorithm are varied in order to investigate their effects on the pseudo-spectrum obtained. Firstly, **Figure 27** shows the spectra produced when varying lengths of input signals are used. It can be seen that the MUSIC algorithm is able to identify the presence of two sine waves even when N=30, which was not possible with the periodogram method.

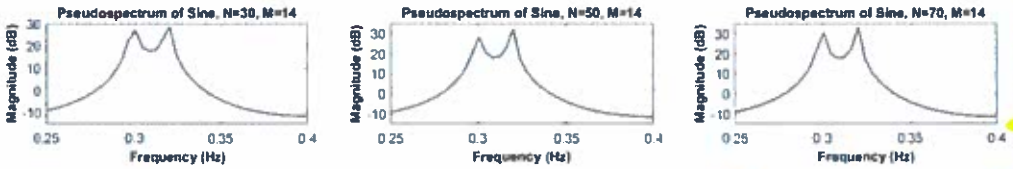


Figure 27: Pseudo-spectrum of MUSIC algorithm with varying N

Various graphs for different examples, to show robustness of the method.

Next, M is varied and the pseudo-spectra are shown in **Figure 28**. It can be seen that the value of M does affect the performance of the MUSCI algorithm. When M is low (for example $M = 5$), the sinusoid at $0.3Hz$ could not be identified. As M increases, it is observed that the peaks get sharper and more distinct, and this aids in the identification of the complex exponentials. This is due to a better description of the noise subspace, allowing for a more accurate estimation of the spectrum due to the summation operation in the denominator of Eq. (21). A larger value of M however, will lead to a higher computational complexity.

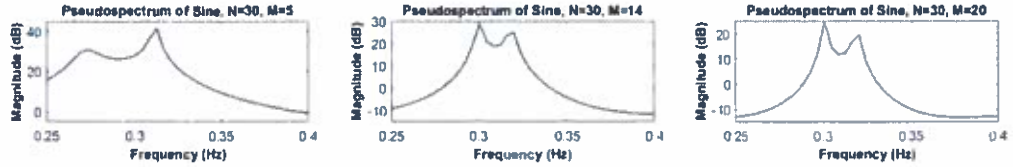


Figure 28: Pseudo-spectrum of MUSIC algorithm with varying M

An additional complex exponential at a frequency of $35Hz$ is then added to $x(n)$, to investigate the effects of using an erroneous value of p for the MUSIC algorithm. It can be seen in **Figure 29** that when p is smaller than the number of signals present, the MUSIC algorithm fails in identifying any of the three complex exponential, due to the interference from the third signal eigenvector which was wrongly assumed to be a noise eigenvector. However, when p is increased beyond the number of complex exponentials present, there appears to be no observable deterioration in the pseudo-spectrum obtained. However, as p continues to increase, it is expected that the peaks will become less and less distinct as the number of noise eigenvectors decreases.

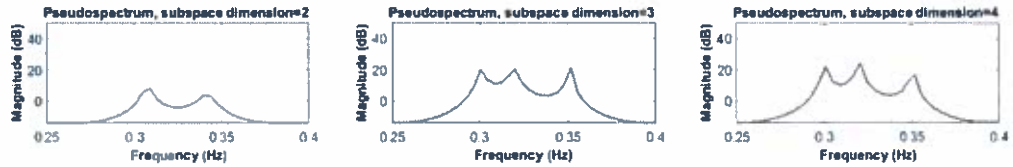


Figure 29: Pseudo-spectrum of MUSIC algorithm with varying p

The amplitudes of the complex exponentials are varied, and the pseudo-spectra are shown in **Figure 30**. It can be observed that there seems to be no clear relationship between the height of the peaks and the amplitudes of the sinusoids present in the signal. Compared to the periodogram where the magnitude of each peak in the spectrum is able to provide information on the amplitude of the sinusoid with that frequency, the pseudo-spectra is unable to offer any information about the magnitude, or even the relative magnitude, of the complex exponentials in the input signal.

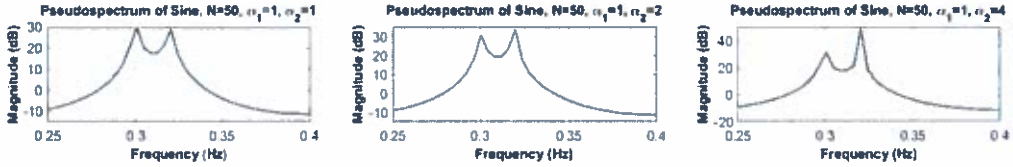


Figure 30: Pseudo-spectrum of MUSIC algorithm with varying α_2

Finally, the noise power is varied and the pseudo-spectra is again obtained (**Figure 31**). Firstly, it can be seen that a the noise increases, the pseudo-spectrum is no longer able to identify the two signals, while the periodogram still exhibits two peaks at the correct frequency. This means that the periodogram performs better than the MUSIC algorithm in cases of low SNR, and this is because if the noise power is too high such that the eigenvalue which corresponds to a noise eigenvector is larger than the signal eigenvalue, the signal eigenvector will included in the noise subspace, leading

to erroneous results. Secondly, when the noise power is varied, it can be seen that the minimum power of the periodogram increases across all frequencies. This is known as the noise floor, and it provides an indication to the power of the noise present. The periodogram is also able to provide information about the type of noise present. Since the noise floor increases by a constant value across all frequency values, it can be concluded that the noise present in $x(n)$ is AWGN. For the pseudo-spectrum however, the increase in noise power did not affect any part of the spectrum except for its peaks. This means that the pseudo-spectrum is unable to provide any information about the noise present.

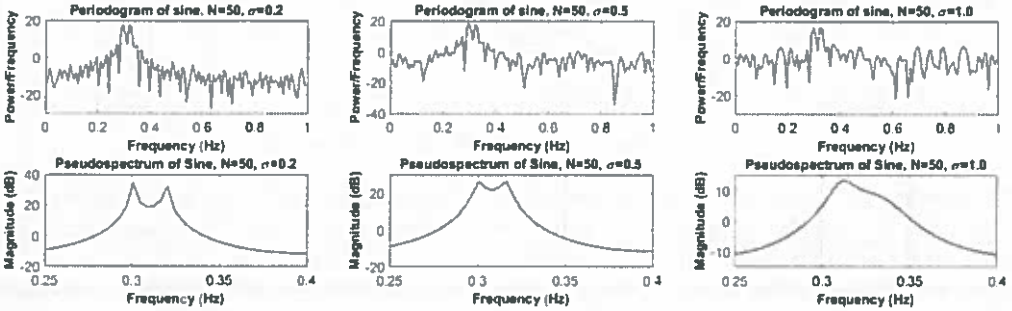


Figure 31: Pseudo-spectrum of MUSIC algorithm with varying σ^2

The advantages and disadvantages of the spectrum estimate when using MUSIC are summarised in **Table 2** below. In general, the MUSIC algorithm is effective in attaining good resolution and accurate estimations of the frequencies of complex exponentials present in the signal with limited data samples, but is unable to provide any further information about either the signal or noise present.

Advantages	Disadvantages
- Has good frequency resolution even with short data lengths - Is able to accurately detect frequency components with short data lengths	- Requires accurate information on the number of exponentials present in a signal for identification - Does not provide any information about the noise present - Does not provide any information about the relative amplitudes of the sinusoids present in the signal

Table 2: Summary of the Advantage and Disadvantages of Spectrum Estimation using MUSIC

Good usage of table to summarize findings and to conduct proper analysis.

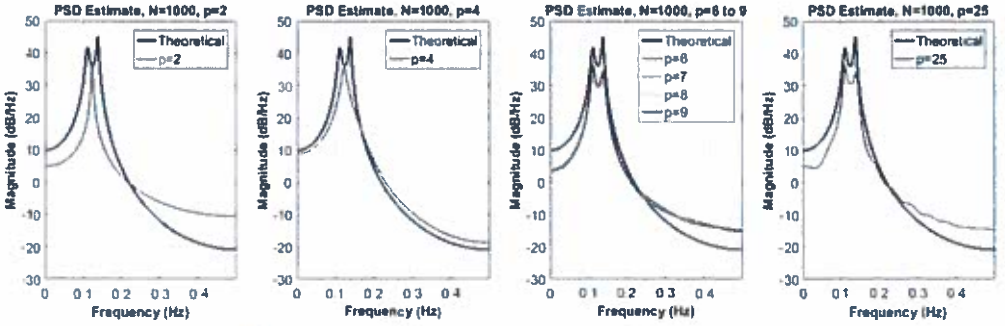
2.2 Spectrum of Autoregressive Processes

a) The AR parameters of an AR process, denoted by $\mathbf{a}$, can be obtained by solving the following equation

$$\mathbf{a} = \mathbf{R}_{xx}{}^{-1}\sigma^2\mathbf{b} \tag{22}$$

where $\mathbf{R}_{xx}$ is the estimated ACF matrix σ^2 is the noise power and $\mathbf{b} = [1\ 0 \dots 0]^T$. Hence, the AR parameters will be affected by the ACF estimates used. As mentioned in **Section 2.1**, the unbiased ACF estimator produces inaccurate ACF estimates at large lags due to the lack of sufficient data points. This will lead to an inaccurate estimated ACF and hence AR parameters at large lags. More importantly, it has been observed that the use of the unbiased ACF estimator does not necessarily return an autocorrelation matrix that is positive definite. This means that the $\mathbf{R}_{xx}$ matrix may not be invertible and hence Eq. (22) will not have a solution, leading to the inability to calculate the AR parameters.

b) 1000 samples of the autoregressive process, $x(n)$, is generated in Matlab, and the power spectrum density of the signal is estimated using different model numbers to investigate the effects of increasing the order of the underlying AR model on the PSD estimate. The results obtained are shown in **Figure 32**, and the black line present in each plot corresponds to the theoretical PSD of the AR process. When the order, $p = 2$, there is severe underfitting of the model as the model is unable to accurately model the two distinct peaks present in the theoretical PSD of the signal. When $p = 4$, the model is also unable to model the two frequency peaks although the input is an AR(4) process. This is due to the relatively small number of samples used (500), resulting in insufficient information present for the AR parameters to be accurately predicted. When the order is increased to 8, there are a greater number of poles which can be placed to estimate the AR process, and this leads to a better estimate. The PSD of the AR model is able to exhibit the two characteristic peaks of the original AR(4) process. As the order is further increased ($p = 6,7,8$), the number of poles increases but there is no increase in accuracy in the resulting PSD estimate, and magnitude of the predicted peaks do not approach those in the theoretical spectrum. When the order number is increased drastically to $p = 25$, although the distinct peaks are still present in the estimated PSD, there now appears to be slight fluctuations in the rest of the PSD spectrum as compared to the theoretical response. This may be an effect of over-fitting, where the model now attempts to model the WGN present as well, resulting in an inaccurate PSD estimate.



Excellent graphs, with appropriate legends.

Figure 32: PSD Estimates of an AR4 Process with varying p

c) The experiment is repeated for a data length of $N = 10000$ samples. The resulting PSD estimates are shown in **Figure 33**. In general, when the number of data sample available is increased, the PSD estimates obtained are more accurate, as the magnitudes of the peaks approaches the theoretical values for orders $p = 6$ to 8. The increase in the number of data samples available means that there is now more information for the AR parameters to be calculated accurately, and this results in the estimated model with $p = 4$ exhibiting the two correct peaks in its PSD. However, the increase in the number of samples does not affect the under-fitted model of order 2. The model will still be unable to model the PSD of the AR4 process due to a lack of poles (or degrees of freedom), and hence, only a single peak can be observed. When the model order is again increased greatly to 25, overfitting can still be observed as there appears to be some ripples in the estimated PSD in the region of $0.3 - 0.5Hz$. The effects of overfitting however is a lot less severe as compared to the previous section, when only 1000 data points are available. This may be due to the fact that the fluctuations in signal values due to the presence of noise is a lot smaller due to averaging over a much larger data set.

As always, strong explanations to accompany the graphs.

To further investigate the effects of the increase in data size, the estimated PSD spectra of the AR4 process using a model order of $p = 4$ for various data sizes are plotted in **Figure 34**. It can be seen that as the number of input samples increases, the resemblance between the theoretical PSD and the estimated PSD increases, such that when $N = 100000$, the estimated PSD is almost identical to the theoretical one. This supports the previous postulation that when N is small, there is insufficient information and inaccurate data captured in the autocorrelation matrix $\mathbf{R}_{xx}$ of the input for the predicted parameters to closely resemble those used to generate the AR(4) process.

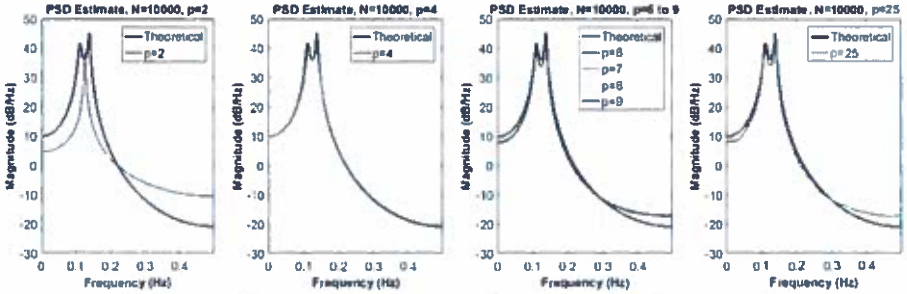


Figure 33: PSD Estimates of an AR4 Process with varying p for 10000 data samples

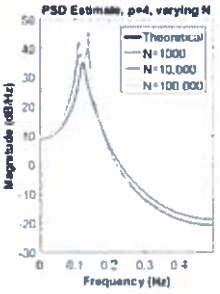


Figure 34: PSD Estimates of an AR4 Process with varying N

2.3 Principal Component Analysis

Good presentation of the problem.

a) The singular values of $\mathbf{X}$ and $\mathbf{X}_{noise}$ are shown in **Figure 35 (left)**. It is clear from the singular values of $\mathbf{X}$ that the rank of the input data is 3, since there are only 3 non-zero singular values. Similar to $\mathbf{X}$, $\mathbf{X}_{noise}$ also contains three singular values which are significantly higher than the other singular values. The seven singular values which are originally not present in $\mathbf{X}$ is due to the addition of noise to the input signal. The amplitude of the singular values introduced due to noise can be observed to be about half the magnitude of the original singular values. The addition of noise has also increased the original three singular values of $\mathbf{X}$, although this increase is not as pronounced as that of the remaining 7 singular values. Furthermore, the addition of noise has increased the rank of $\mathbf{X}_{noise}$ to 10, making it a fully ranked matrix.

Next, the squared error between the singular values of $\mathbf{X}_{noise}$ and $\mathbf{X}$ are calculated and shown in **Figure 35 (right)**. It can be seen again that the addition of noise resulted in very large squared error values for singular values $4 - 10$, due to the introduction of non-zero singular values at those instances. The addition of noise however only slightly increased the existing three singular values, such that the magnitude of the squared error for these values are significantly lower. It would be hard to identify the rank of the matrix $\mathbf{X}_{noise}$ if the noise power increase significantly, such that the magnitudes of the

originally zero valued singular values $(4-10)$ are increased to the same values as the original non-zero singular values. This is possible since the addition of noise disproportionately increases the originally zero valued singular values a lot more than the non-zero valued ones.

Exhaustive explanation of results, with the right graphs to support the claims well done!

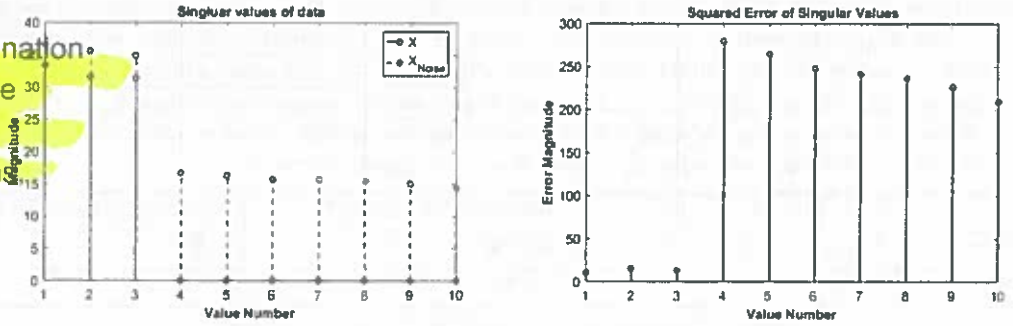


Figure 35: Singular values of $\mathbf{X}_{noise}$ and $\mathbf{X}$, and the Squared Error between the Singular Values

b) Since the rank of $\mathbf{X}$ has previously been ascertained to be 3, there are three most significant principal components in $\mathbf{X}_{noise}$. A low rank approximation is then made, and this is denoted as $\widetilde{\mathbf{X}}_{noise}$. The MSE between the 10 realisations of $\mathbf{X}_{noise}$ and $\mathbf{X}$, as well as the MSE between the 10 realisations of $\widetilde{\mathbf{X}}_{noise}$ and $\mathbf{X}$ are calculated and shown in **Figure 36**. The number of significant principal components, r, is then varied to investigate its effect on the MSE calculated. When $r = 3$, the MSE between the low-rank approximation of $\mathbf{X}_{noise}$ and the original signal is a lot smaller than the MSE between $\mathbf{X}_{noise}$ and the original signal. This is expected as the low-rank approximation essentially discards singular values which are not related to the desired signal, and hence can be thought of as a de-noising process. This not only results in a smaller MSE value, but there will also be significant savings on computational complexity due to the reduction in dimensions of the data matrix. When r is increased to 5, the MSE of the estimated $\mathbf{X}_{noise}$ increases. This is due to the fact that now some components of noise are retained and hence, there is an increase in error. However, some parts of the noise information are still removed, and hence the low rank approximation continues to exhibit lower MSE as compared to $\mathbf{X}_{noise}$. On the other hand, when r is reduced to 1, the MSE of the estimated $\mathbf{X}_{noise}$ at some realisation numbers increases to twice that of $\mathbf{X}_{noise}$. This is because when the low rank approximation is overdone such that r is smaller than the rank of the original data signal, some desired signal components are now being discarded along with the noise, and this leads to a very erroneous approximation of the data.

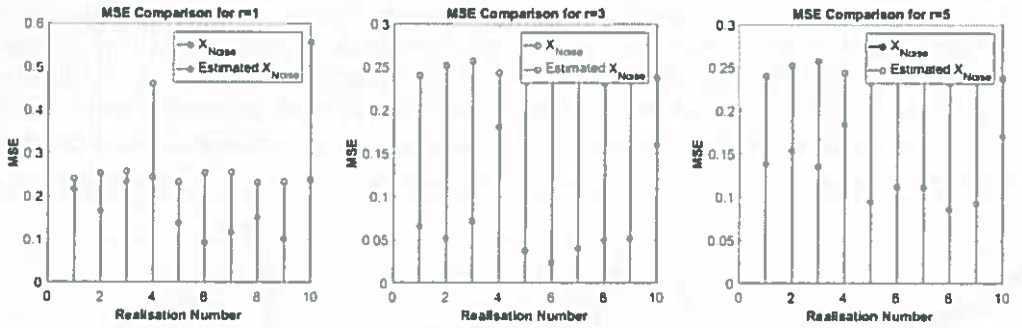


Figure 36: MSE Comparison between $\mathbf{X}_{noise}$ and $\widetilde{\mathbf{X}}_{noise}$ for varying r

c) The ordinary least squares (OLS) and principal component regression (PCR) solutions for the parameter matrix $\mathbf{B}$ which relates $\mathbf{X}_{noise}$ and $\mathbf{Y}$ is calculated using $\mathbf{X}_{noise}$ for the former and $\widetilde{\mathbf{X}}_{noise}$ for the latter. This is done by the following equations:

$$\widehat{\mathbf{B}}_{OLS} = (\mathbf{X}_{noise}^T\mathbf{X}_{noise})^{-1}\mathbf{X}_{noise}^T\mathbf{Y} \tag{23}$$

$$\widehat{\mathbf{B}}_{PCR} = \mathbf{V}_{1:r}(\boldsymbol{\Sigma}_{1:r})^{-1}\mathbf{U}_{1:r}^T\mathbf{Y} \tag{24}$$

where $\mathbf{X}_{noise} = \mathbf{U}\boldsymbol{\Sigma}\boldsymbol{V}^T$. $\widehat{\mathbf{Y}}_{OLS}$ and $\widehat{\mathbf{Y}}_{PCR}$ is then calculated using

$$\widehat{\mathbf{Y}}_{OLS} = \mathbf{X}_{noise}\widehat{\mathbf{B}}_{OLS} \tag{25}$$

$$\widehat{\mathbf{Y}}_{PCR} = \widetilde{\mathbf{X}}_{noise}\widehat{\mathbf{B}}_{PCR} \tag{26}$$

The MSE between $\widehat{\mathbf{Y}}_{OLS}$ and $\widehat{\mathbf{Y}}_{PCR}$ with both the training output data and testing output data are calculated, and these values are shown in **Table 3**, along with the time taken to compute $\widehat{\mathbf{B}}_{OLS}$ and $\widehat{\mathbf{B}}_{PCR}$. From the results, it can be seen that the OLS solution outperforms the PCR solution when training data is used. This is to be expected as the training data $\mathbf{X}_{noise}$ is used to obtain the OLS solution. The $\mathbf{Y}$ output data is constructed from $\mathbf{X}_{noise}$ with the addition of some noise, and since the OLS solution utilises all the available information about $\mathbf{X}_{noise}$, it is able to have a smaller MSE as compared to the PCR

solution, where some noise information has been discarded due to the principal component analysis (PCA) process. However, when the testing data set, $\mathbf{Y}_{test}$ is used, the PCR solution outperforms the OLS solution. This is because the PCR solution is a more generalised solution as it is based on $\mathbf{X}$ instead of $\mathbf{X}_{noise}$, making it more robust to slight fluctuations in the input values used to calculate $\mathbf{Y}$. Thus, the differences between $\widetilde{\mathbf{X}}_{noise}$ and $\mathbf{X}_{test}$ is less pronounced as compared to that between $\mathbf{X}_{noise}$ and $\mathbf{X}_{test}$, thus resulting in a smaller MSE. Looking at the time taken to formulate the estimated $\mathbf{B}$ matrices for both solutions, it is very obvious that the PCR solution is the more computationally efficient one. This is due to the reduction in the number of computations needed due to the large reduction in dimensions when $\widetilde{\mathbf{X}}_{noise}$ is used. Since the performance of both solutions in terms of MSE for both training and testing data are rather similar, the better solution will be the one which is more efficient, and in this case the PCR solution is the clear winner.

	Mean Squared Error for $\mathbf{Y}$	Mean Squared Error for $\mathbf{Y}_{test}$	Time taken to formulate $\widehat{\mathbf{B}}$
OLS Solution	3.5517	22.5950	0.0012
PCR Solution	3.5771	22.5528	0.000289

Table 3: MSE for Testing and Training Data, Time taken to formulate $\widehat{\mathbf{B}}$

d) The estimated regression coefficients $\widehat{\mathbf{B}}$ are now tested over an ensemble of test data. Using the same coefficients as the previous section, the MSE for the PCR and OLS schemes based on the realisations of $\mathbf{Y}$ and $\widehat{\mathbf{Y}}$ using the function `regval` is calculated for 1000 realisations. The averaged MSE is then calculated and shown in **Table 4**. It can be seen that once again, the PCR solution produced the lower MSE, and this again goes to show that $\widehat{\mathbf{B}}_{PCR}$ is a much better estimate than $\widehat{\mathbf{B}}_{OLS}$, and is more robust to the presence of noise since $\widehat{\mathbf{B}}_{PCR}$ is obtained using values which are closer to the true values of $\mathbf{X}$. Hence, it can be concluded that the PCR solution is the better one in terms of both accuracy and speed.

	Mean Squared Error
OLS Solution	2.3583
PCR Solution	2.3369

Table 4: MSE for 1000 new realisations of test data

As before, good usage of tables to present information clearly and effectively.

2.4 Respiratory Sinus Arrhythmia from RR-Intervals

a) The standard periodogram is first applied on the RRI data in order to obtain the power spectral density of the data in each of the three trials. The periodograms are shown in **Figure 37**. For the standard periodograms, hamming windows with lengths equal to the number of data samples are used. The hamming window is used to reduce the effect of spectral leakage in the periodograms, which is significant especially since the signal is very noisy. This allows the identification of a greater number of harmonics. From the periodograms obtained, distinct peaks can be observed in all three trials, which corresponds to the breathing during the trial. Only the fundamental frequency can be observed in Trial 1, while the second harmonic of the breathing can also be discerned in Trial 2. In trial 3, up to the 8th harmonic can be observed.

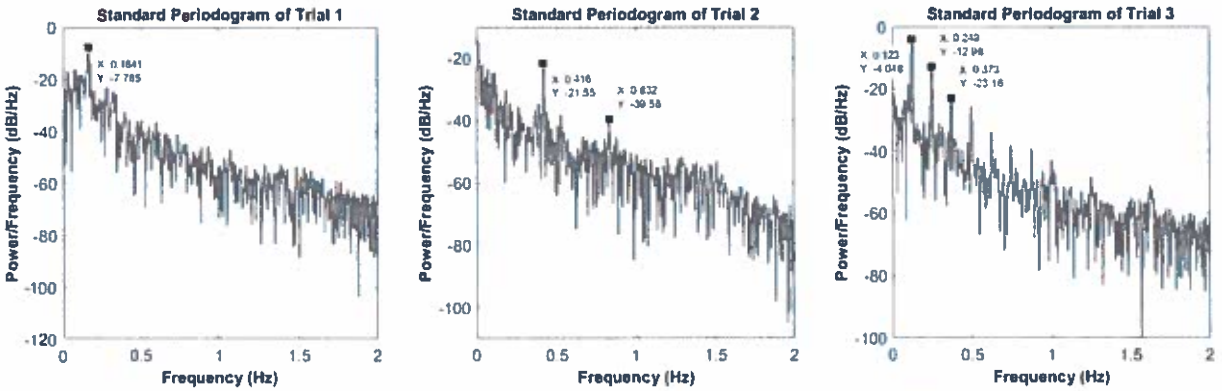


Figure 37: Power Spectra of RRI Data for 3 Trials using a Hamming Windowed Standard Periodogram

Next, the averaged Bartlett's periodogram is used on the same set of RRI data to calculate the power spectral density. The periodograms are plotted for three different hamming window sizes ($50, 25$ and $10s$) and are plotted in **Figure 38**. It can be again observed that the Bartlett's periodogram is very much smoother than the windowed periodogram, as most of the noise has been removed (reduction in variance). The peaks which corresponds to the breathing rate and its harmonics are however wider and lower due to the reduction in frequency resolution as the number of samples used for the calculation of each periodogram is decreased. This trade-off between variance reduction and frequency resolution can be better examined when the Bartlett's periodograms with different window lengths are analysed. When the window length is

Sensible justification of method used.

$50s$, the peaks of interest are sharp, although there still remains a significant amount of fluctuations in the spectrum. When the window length is reduced to $10s$, although most of the fluctuations has been removed, resulting in a very smooth power spectrum, most of the harmonics of the breathing rate has also been removed (especially in Trial 1 and 2). Thus, in order to obtain a good balance between a sufficiently smooth curve with nice distinct peaks, a windowing length of 25s is chosen.

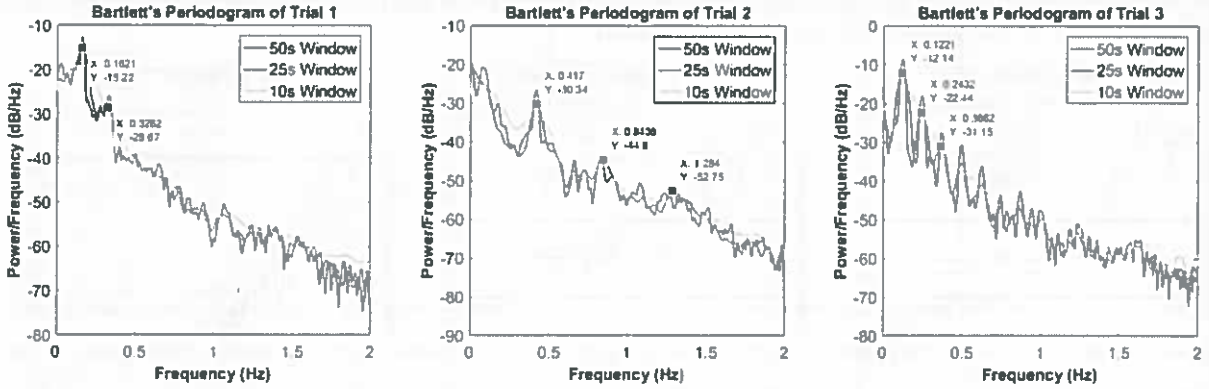


Figure 38: Bartlett's Periodogram of RRI Data for 3 different Trials with varying Window Length

b) The PSD estimates of the RRI data from the three trials are different due to the way they were conducted. In the first trial, the breathing is unconstrained, and for a normal healthy adult, the breathing rate will be about 12-20 breaths per minute. In the second trial, the subject was asked to carry out fast breathing of 25 breaths per minute. In the final trial, the subject is asked to carry out slow breathing at 7.5 breaths per minute. The differences in the breathing rates across the three trials result in the differences in frequency values where the peaks and their harmonic appear.

From the periodograms obtained from the first trial, there is a peak at a frequency of 0.1621. Using the formula

$$Breath\ per\ Minute = 60 * f_0 \quad (27)$$

Explanation of how to properly detect the peaks

the breathing rate of the subject is at approximately 9.7 breaths per minute. Although this is slightly lower than the number of breaths11 per minute of an average adult, the difference is not significant. In the second trial, using Eq. (27) again, the fundamental peak is observed at $0.417Hz$, which translates to 25 breaths per minute. This coincides exactly with the required breathing rate of the trial. Finally, in trial 3, the peak corresponding to the fundamental frequency occurs at $0.1221Hz$. Putting this into Eq. (27) yields a breathing rate of 7.32 breaths per minute, and this again matches the expected breathing rate of 7.5 breaths/minute.

In the power spectral estimate plots using the standard periodogram (**Figure 37**), the peaks corresponding to the fundamental frequency of the breathing rate can be identified in all three trials. Only the second harmonic of $0.832Hz$ can be distinguished in Trial 2, while 8 harmonics can be observed in trial 3. For the Bartlett's periodograms on the other hand (**Figure 38**), the fundamental frequency of all three breath rates can be observed across all window lengths. However, when window lengths of $50s$ and $25s$ are used, the second harmonic of the first trial and the third harmonic of the second trial can also be identified. This was not possible when the standard periodogram is used, and this shows that the reduction of noise does indeed help in accentuating the peaks at the fundamental and more often the harmonic frequencies of the frequencies of interest. When the window length is at $10s$, the only peaks discernible are the fundamental frequency responses due to the huge decrease in frequency resolution as previously discussed..

c) **Figure 39** shows the AR spectrum estimates for the RRI signals for the three different trials. The AR spectra are compared with the Bartlett's periodogram with window length $25s$, since it has previously been established that this window length resulted in the best variance reduction while retaining the frequency resolution of the power spectrum.

In trial 1, to model the fundamental frequency peak, an order of 2 is sufficient. Increasing the model order from 2 to 5 resulted in no significant improvements in the spectrum. However, when the AR model order is increased to 6, the first and third harmonic of the breath rate can be discerned. The third harmonic in the first trial has not been identified by any of the preceding periodogram methods, thus, it can be concluded that the AR(6) model yields the best power spectrum estimate for Trial 1.

For trial 2, the order of the AR model has to be increased drastically to model even the fundamental frequency. At an order of 5 the AR model is still unable to reproduce the peak at the fundamental frequency of $0.417Hz$. When the order number is increased to 18, the first two harmonics of the subject's breathing can be easily identified. The third harmonic as modelled by the AR(18) model however occurs at $1.339Hz$, showing a slight deviation from the expected value of $1.251Hz$. When the order is then increased to 20, although the peaks of the AR spectrum are now more aligned with that of the Bartlett's periodogram, spurious peaks start to surface at frequency between $0.5Hz$ and $0.7Hz$. This signifies that the AR(20) model is overfitting the RRI data.

Lastly, in trial three, the order of the model has to again be increased in order to model the peaks. When the order of the AR model is set to 18, only the first and second (very slightly) harmonic can be identified. However, when the model order is increased to 30, the entire power spectrum of the RRI data is now being modelled very nicely, so much so that up to 13 harmonics can be identified. Although the AR(30) model has thus far yielded the best spectral estimate for trial 3, the computational complexity of a 30 order model is significantly greater than the Bartlett's periodogram, and hence may not be the preferred PSD estimation method in certain applications.

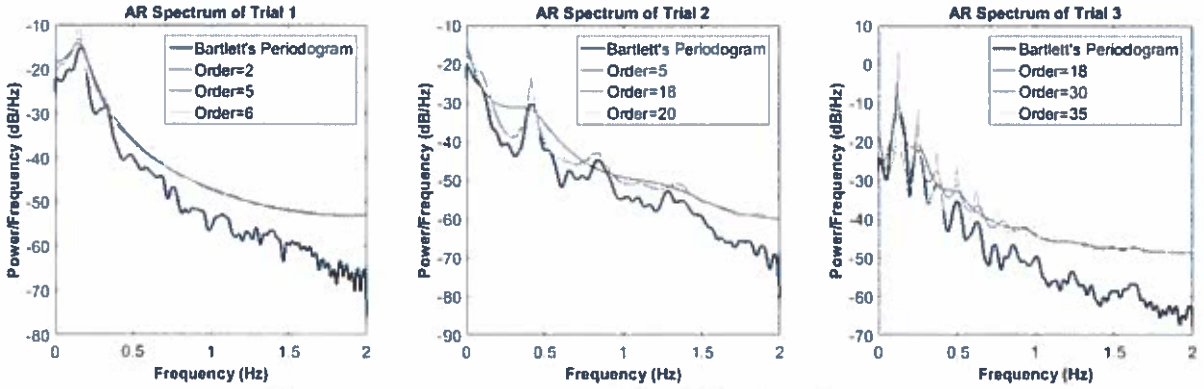


Figure 39: AR spectrum of RRI data for 3 Different Trials, with varying Model Order

In conclusion, the estimated AR spectrums are less noisy than both the standard periodogram and the Bartlett's periodogram. The AR models also produces results which more closely resembles the real power spectrum, but the drawback is that to do this, a model of prohibitively high order needs to be implemented. If the order of the AR model is insufficient, either the peaks of the spectrum will not occur at exactly the right frequency, or the peaks will not appear at all. Compared to the other periodograms, as the AR spectrum is obtained via an all-pole method, the spectrum is incredibly smooth across regions in frequency where there are no significant frequency components since the poles will not be placed near that segment in the complex plane.

3 ADAPTIVE SIGNAL PROCESSING

3.1 THE LEAST MEAN SQUARE (LMS) ALGORITHM

3.1.1 Identification of AR Processes

a) A second-order auto-regressive (AR(2)) process satisfies the following difference equation:

$$x(n) = a_1 x(n-1) + a_1 x(n-2) + \eta(n) \tag{27}$$

where $\eta(n) \sim \mathcal{N}(0, \sigma_n^2)$. The input vector of the LMS filter is given by $\mathbf{x}(n) = [x(n-1)x(n-2)]^T$, and thus the autocorrelation matrix can be written as

$$\begin{aligned}\mathbf{R}_x &= E[\mathbf{x}(n)\mathbf{x}(n)^T] \\ &= \begin{bmatrix} E[x(n-1)x(n-1)] & E[x(n-1)x(n-2)] \\ E[x(n-2)x(n-1)] & E[x(n-2)x(n-2)] \end{bmatrix} \\ &= \begin{bmatrix} r(0) & r(1) \\ r(1) & r(0) \end{bmatrix}\end{aligned} \tag{28}$$

Mathematically rigorous throughout.

where $(k) = E[x(n)x(n-k)]$, and since the autocorrelation is symmetric, $r(k) = r(-k)$. In order to calculate the values of $\mathbf{R}_x$, Eq. (27) is first multiplied with $x(n-k)$, where k is an integer, yielding

$$x(n)x(n-\mathrm{k}) = a_1 x(n-1)x(n-\mathrm{k}) + a_2 x(n-2)x(n-\mathrm{k}) + \eta(n)x(n-\mathrm{k}) \tag{29}$$

Now, the expectation can be taken on both sides

$$\begin{aligned}E[x(n)x(n-k)] = r(k) &= a_1 E[x(n-1)x(n-k)] + a_2 E[x(n-2)x(n-k)] + E[\eta(n)x(n-k)] \\ &= a_1 r(k-1) + a_2 r(k-2) + E[\eta(n)x(n-k)]\end{aligned} \tag{30}$$

Since the noise is uncorrelated for previous values of the process,

$$E[\eta(n)x(n-k)] = \begin{cases} \sigma_n^2 \; for \; k = 0 \\ 0 \; for \; k \neq 0 \end{cases} \tag{31}$$

Now, using $k = 0, 1$ and 2, three equations can be formulated to solve for the two unknowns in Eq. (28). The simultaneous equations are written in matrix notation and is given by

$$\begin{bmatrix} 1 & -a_1 & -a_2 \\ -a_1 & 1-a_2 & 0 \\ -a_2 & -a_1 & 1 \end{bmatrix} \begin{bmatrix} r(0) \\ r(1) \\ r(2) \end{bmatrix} = \begin{bmatrix} \sigma_n^2 \\ 0 \\ 0 \end{bmatrix} \tag{32}$$

Solving the above using $a_1 = 0.1, a_2 = 0.8$ and $\sigma_n^2 = 0.25$ gives $r(0) = 0.9259$, $r(1) = 0.46300$, and $r(2) = 0.7870$. Putting these back into Eq. (28), the autocorrelation matrix for the AR(2) process is

$$\mathbf{R_x} = \begin{bmatrix} 0.9259 & 0.4630 \\ 0.4630 & 0.9259 \end{bmatrix} \tag{33}$$

For the LMS algorithm, convergence in the mean is guaranteed if the following inequality is satisfied

$$0 < \mu < \frac{2}{\lambda_{max}} \tag{34}$$

where μ refers to the step size of the LMS algorithm and λ_{max} is the maximum eigenvalue of the autocorrelation matrix of the input, $\mathbf{R_x}$. Applying eigen-decomposition on $\mathbf{R_x}$, the eigenvalues are 1.3889 and 0.4630. This means that for the LMS estimator of the AR(2) process to converge, μ must lie in the range

$$0 < \mu < 1.44 \tag{35}$$

b) An LMS adaptive filter is implemented using $N = 1000$ samples of the $x(n)$ as defined in Eq. (27), and the squared prediction error, $e^2(n)$ is plotted in the logarithmic scale using step sizes $\mu = 0.05$ and $\mu = 0.01$. This is shown in **Figure 40**. From this plot, the convergence rate and the steady-state error values cannot be gleaned. A better plot would be to plot the average squared error of K different realisations of $x(n)$. The mean squared error is shown in **Figure 41** for values of $K = 100$ and $K = 1000$, along with the optimal MSE, which is given as

$$MSE_{optimal} = \sigma_n^2 \tag{36}$$

It can be seen that when the number of realisations the learning curves are averaged across increases, the fluctuations in the curves decreases, and a smoother learning curve can be obtained. Also, comparing the curves for $\mu = 0.05$ and $\mu = 0.01$ shows that the convergence rate increases as the step size increases (200 samples compared to 100 samples), which is to be expected as a larger step size allows the algorithm to have a steeper descent down the error curve. However, a larger step size is also seen to result in a larger steady state MSE (greater deviation from the optimal MSE shown). This is because when μ is large, slight fluctuations of $x(n)$ in the steady state will affect the weights of the adaptive filter by a greater amount as the filter is more reactive, resulting in a larger MSE.

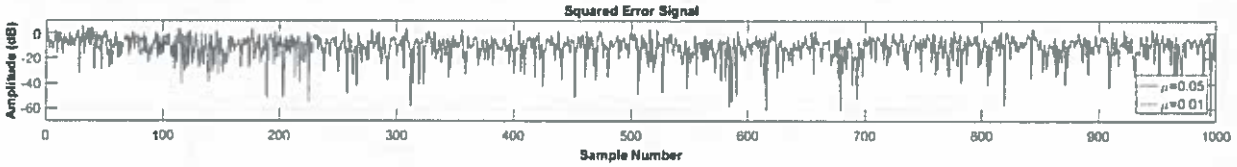


Figure 40: Learning Curve of a single Realisation of $x(n)$ for varying step size

Correct graphs and theory, and highlight of optimal MSE.

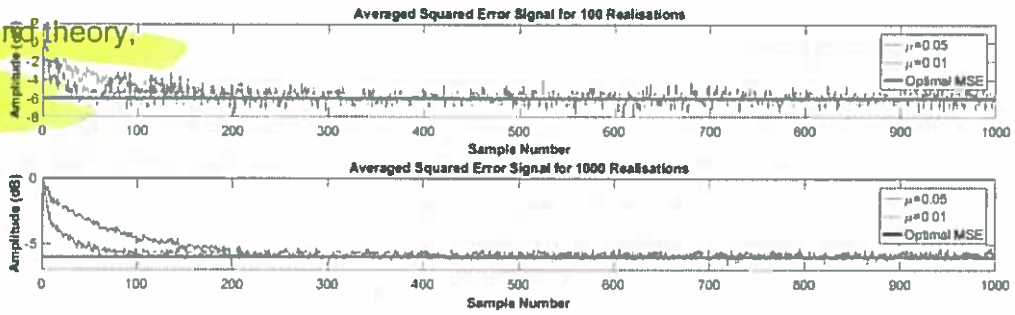


Figure 41: Learning Curve of 100 and 1000 realisations of $x(n)$ for varying step sizes

c) The misadjustment, $\mathcal{M}$, is given by

$$\mathcal{M} = \frac{EMSE}{\sigma_\eta^2} \tag{37}$$

where $EMSE$ refers to the excess mean square error, which is the difference between the mean-square error introduced by the adaptive filter and the minimum attainable mean square error of a Wiener filter. Using 10000 independent trials of the experiment conducted above (so that the averaged learning curves are smoother), the misadjustment errors of the LMS filter for $\mu = 0.05$ and $\mu = 0.01$ are calculate by time-averaging over the steady state of the ensemble averaged learning curved. The results obtained are shown in **Table 4**. Using the autocorrelation matrix $\mathbf{R_x}$, the theoretical misadjustment

error is also calculated for both values of μ. It can be seen that the empirical misadjustment errors are slightly higher than that of the theoretical ones, although the order of magnitudes are similar. This may be due to the number of samples used, and the two values of $\mathcal{M}$ should converge as N increases. Also, the misadjustment error is about five times larger when $\mu = 0.05$ as compared to when $\mu = 0.01$. This means that a larger step size results in a larger MSE in the steady state, which has also been observed in the previous section.

μ	$\mathcal{M}_{\text{empirical}}$	$\mathcal{M}_{\text{theoretical}}$
0.05	0.0517	0.0463
0.01	0.0093	0.0070

Table 5: Theoretical and Empirical Misadjustment Error for varying μ

d) The weights of the adaptive filter, a_1 *and* a_2, are obtained by averaging the weights over 100 realisations of the signal $x(n)$. This is again done so that the resulting learning curves (in terms of weight values) are smoother and thus exhibits a stronger trend which aids in the analysis of the adaptive filters. The evolution of the weight coefficients as the number of iterations of the LMS algorithm increases is shown in **Figure 42** below. The theoretical values of the weight coefficients are also plotted for comparisons to be made. From the figure, it can be seen again that the convergence rate of the filter is higher when μ is larger, as the weights when $\mu = 0.05$ converges faster to the theoretical values (200 iterations for $\mu = 0.05$ and about 800 iterations for $\mu = 0.01$). Furthermore, there is larger deviation between the steady state weight coefficients for $\mu = 0.05$ as compared to $\mu = 0.01$. This again agrees with the previous statement that a larger step size results in a faster convergence at the expense of larger steady state errors. The steady state values of the adaptive filter coefficients are also calculated, and these values are shown in **Table 6**. It can again be seen that when $\mu = 0.01$, the steady state error is smaller since the weights are closer to their true theoretical values.

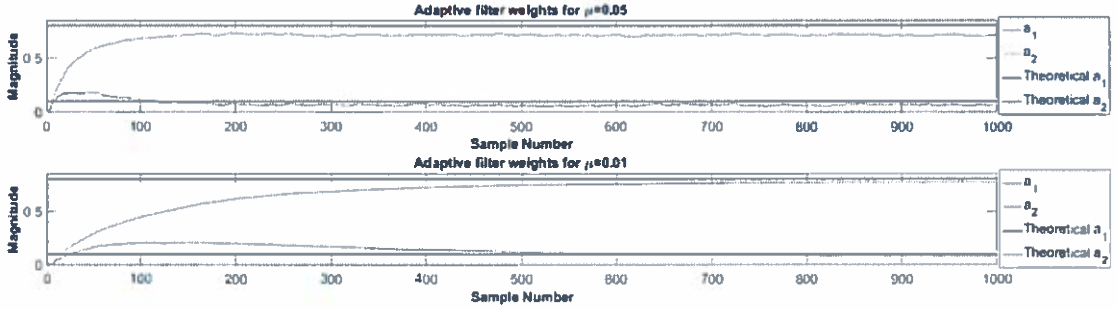


Figure 42: Evolution of Weight Coefficients for 100 Realisations of $x(n)$ with varying μ

	Theoretical	Adaptive, $\mu = 0.05$	Adaptive, $\mu = 0.01$
a_1	0.1	0.0702	0.0890
a_2	0.8	0.7171	0.7737

Table 6: True and Adaptive weight coefficients

3.1.2 The Leaky LMS Algorithm

e) When the input signal $\mathbf{x}(n)$ has an autocorrelation matrix which is only positive semi definite, the LMS adaptive filter has one or more weights which will not converge, leading to instability. The leaky LMS algorithm introduces a leakage coefficient, $0 < \gamma < 1$, to force the weights to converge. The weight coefficients update equation for is given by

$$\mathbf{w}(n+1) = \mathbf{w}(n)(1 - \mu\gamma) + \mu e(n)\mathbf{x}(n) \tag{38}$$

To show that the Leaky LMS algorithm minimises the cost function

$$J(n) = \frac{1}{2}(e^2(n) + \gamma \parallel \mathbf{w}(n) \parallel_2^2) \tag{39}$$

The derivative of the cost function is taken with respect to $\mathbf{w}(n)$, giving (some notations (n) are dropped for easy representation)

$$\frac{\partial J}{\partial \mathbf{w}} = e(n)\frac{\partial e}{\partial \mathbf{w}} + \gamma \mathbf{w}(n) \tag{40}$$

Since $e(n) = \mathbf{x}(n) - \mathbf{w}^T(n)\mathbf{x}(n)$

$$\frac{\partial e}{\partial \mathbf{w}} = \frac{\partial}{\partial \mathbf{w}}[\mathbf{x}(n) - \mathbf{w}^T(n)\mathbf{x}(n)] = -\mathbf{x}(n) \tag{41}$$

Substituting Eq. (41) into Eq. (40) yields

$$\frac{\partial J}{\partial \mathbf{w}} = \gamma \mathbf{w}(n) - \mathbf{x}(n)e(n) \tag{42}$$

Correct proof and equations are overall well aligned.

The update equation which is based on the gradient descent method to reach an optimal value of $\mathbf{w}(n)$ based on the cost function $J(n)$ is given by

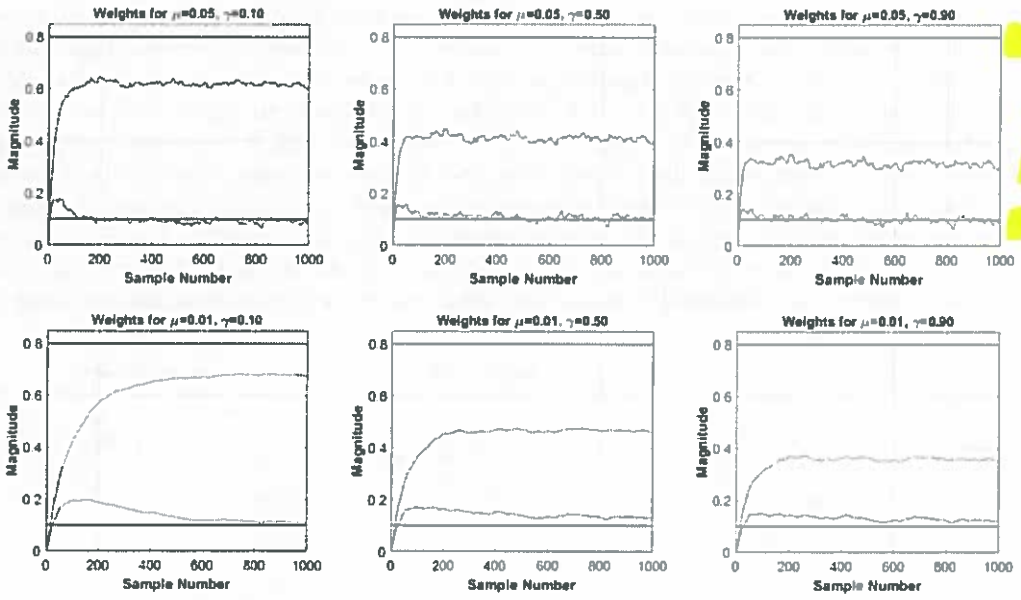
$$\mathbf{w}(n+1) = \mathbf{w}(n) - \mu \frac{\partial J}{\partial \mathbf{w}} \tag{43}$$

Substituting the derivative of the cost function obtained in Eq. (42) yields

$$\begin{aligned} \mathbf{w}(n+1) &= \mathbf{w}(n) - \mu[\gamma \mathbf{w}(n) - \mathbf{x}(n)e(n)] \\ &= \mathbf{w}(n)(1-\mu\gamma) + \mu e(n)\mathbf{x}(n) \end{aligned} \tag{44}$$

which is exactly equal to the update equation of the leaky LMS algorithm (Eq. (38)), showing that the leaky LMS algorithm is equivalent to using a gradient descent method which minimises the cost function $J(n)$. It can be seen from the update equation that gamma acts as a forgetting factor by reducing the impact the current weight coefficients has on the values of the next coefficients.

f) The leaky LMS algorithm is then implemented in Matlab and the adaptive filter coefficients for the signal $x(n)$ are obtained for various values of γ and μ. The coefficient values are again averaged over the values obtained for 100 realisations of $x(n)$. The learning curves of the weight coefficients for the various values of γ and μ tested are shown in **Figure 43**. Firstly, it can be seen that when γ is kept constant, increasing μ shows the expected behaviour of increasing the convergence rate as well as increasing the steady state error by introducing more fluctuations into the steady state portion of the learning curve. Unlike the LMS adaptive filter however, increasing μ also has the added effect of increasing the "leakiness" of the algorithm, such that subsequent weight vectors are less dependent on the current ones. This results in a pronounced difference in the steady state weight values. Next, when μ is kept constant and γ is varied, the phenomenon described above is more observable. As γ increases, the effect of the previous weight vectors on the current one decreases, and this manifests as a decrease in the coefficient values of a_2 and an increase in the coefficient values of a_1. This means that due to the leakiness of the algorithm, there is now a very large difference between the true coefficient values (blue and red bold lines in each plot) and the steady state coefficient values of each adaptive filter.



clear graphs, showing results for different parameters.

Figure 43: Learning Curves of Weight Vectors for the Leaky LMS Algorithm for 100 realisations of $x(n)$, with varying μ and γ

The discrepancy between the steady state coefficients of the leaky LMS algorithm and the true coefficient values can be explained by looking at the expected weight values of the algorithm. The convergence property of the expected weight values of the LMS algorithm is given by

$$\lim_{k\to\infty} E[\mathbf{w}_k] = (\mathbf{R}_x + \gamma \mathbf{I})^{-1}\mathbf{p} \tag{45}$$

where $\mathbf{p}$ is the cross correlation vector. The optimal weight vector given by the Weiner Hopf solution, which in this case is equivalent to the true weight values of the AR process, is given by

$$\mathbf{w}_{opt} = \mathbf{R}^{-1}\mathbf{p} \tag{46}$$

By comparing the long term convergence values of the weight vectors of the leaky LMS algorithm and the true values of the coefficients given by the Weiner-Hopf solution, it is obvious that the two values will not be equal for any non-zero γ, and the differences in both solutions will be increased as γ or μ increases, which was observed previously. From Eq. (45)

it can be seen that the leaky LMS algorithm is able to increase the rank of the autocorrelation matrix if it is not already full rank, allowing the inverse of the matrix to be obtained. This process also makes it harder for the adaptive filter to model the underlying AR process since the SNR at the input is low, resulting in erroneous modelled coefficient values.

3.2 Adaptive Step Sizes

a) The trade-off between convergence rate and steady state error has been covered in great detail in the previous section. A large μ will result in faster convergence but a higher steady state error, while a smaller μ yields a smaller steady state but the algorithm takes a longer time to converge. It is hence obvious that a better LMS algorithm will be one where μ is adaptive, such that it is large at the beginning to allow the weights to converge quickly, but once convergence is attained, it should be small to minimise the steady state error. The Gradient Adaptive Step Size (GASS) algorithm implements an adaptive μ and does just that. The update equation for μ following the GASS framework is given by

$$\mu(n+1) = \mu(n) + \rho e(n)\mathbf{x}^T(n)\boldsymbol{\psi}(n) \tag{47}$$

where $\boldsymbol{\psi}(n)$ is a function which decides how the step sizes are updated and ρ is the step size of the μ update equation. The value of μ is now a function of the error signal, $e(n)$, and hence when the error is large, $\mu(n)$ will be large, enabling faster convergence, and when the error is small, $\mu(n)$ will be small, reducing the steady state error. Three different types of GASS algorithms will be used in this section, and they use different $\boldsymbol{\psi}(n)$ functions as shown below

$$\text{Benveniste}: \boldsymbol{\psi}(n) = [\mathbf{I} - \mu(n-1)\mathbf{x}(n-1)\mathbf{x}^T(n-1)]\cdot\boldsymbol{\psi}(n-1) + e(n-1)\mathbf{x}(n-1) \tag{48}$$

$$\text{Ang \& Farhang}: \boldsymbol{\psi}(n) = \alpha\boldsymbol{\psi}(n-1) + e(n-1)\mathbf{x}(n-1) \tag{49}$$

$$\text{Matthews \& Xie}: \boldsymbol{\psi}(n) = e(n-1)\mathbf{x}(n-1) \tag{50}$$

The three GASS algorithms are implemented and their performances are compared when a real-valued MA(1) system, $x(n)$, is used

$$x(n) = 0.9\eta(n-1) + \eta(n) \qquad \eta \sim \mathcal{N}(0,0.5) \tag{51}$$

Once again, 100 realisations of $x(n)$ are independently generated, and the averaged squared error and weight error curves across all the realisations are obtained for each of the above GASS algorithm and plotted in **Figure 44** below. The average weight values of the second coefficient (corresponding to 0.9) are also plotted in order to exhibit convergence. For the averaged squared error and weight error plots of the algorithms, it may appear that convergence has not been reached and the algorithms should be accessed over a larger sample size, however, a look at the evolution of the filter weights shows that convergence has indeed been achieved. Due to the LMS filter having complete knowledge of the input signal, in this case the noise $\eta(n)$, the steady state errors can be made exceedingly small, and is ultimately only limited by the numerical precision of Matlab, resulting in a pseudo-steady state observed for the Benveniste algorithm. Since the weights of the algorithms converges to the true value in the steady state (except for the standard LMS filter with $\mu = 0.01$ since convergence is not achieved after 1000 iterations), there is no steady state error and thus in the rest of this discussion, only the convergence rate will be discussed.

Excellent organization of graphs to present results.

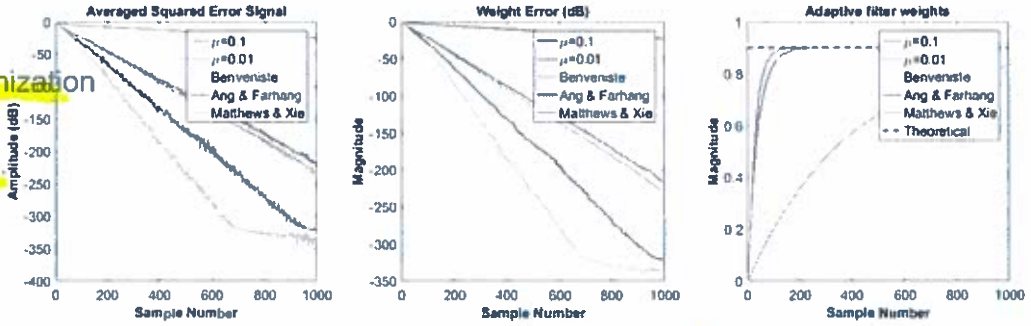


Figure 44: Averaged Squared Error, Weight Error and Adaptive Filter weights plots for Different LMS Algorithms

Looking at **Figure 44**, it can firstly be seen that the original LMS algorithm with a constant μ has a slower convergence rate than all of the GASS algorithms. Once again, as mentioned previously, the LMS algorithm with a larger step size (μ =0.1) shows a faster convergence rate than the algorithm with the smaller step size (μ = 0.01). Next, it can be seen that the Benveniste algorithm has the faster convergence rate, followed by Ang & Farhang's method. Matthew & Xie's algorithm has the slowest convergence rate out of the 3 GASS algorithms. This can be explained by revisiting the formulations of the three algorithms shown in Eq. (48)-(50). For the Benveniste algorithm, the update equation for μ resembles an adaptive low pass filter on the error gradient. The coefficients of the this filter are determined by both the previous value of μ, as well as the magnitude of the previous input signal. The Ang & Farhang algorithm on the other hand uses a fixed low pass filter with parameter α, and finally the Matthews & Xie algorithm removes the filtering altogether,

and the update equation is merely a function of the error gradient. Looking at the three algorithms, it can be seen that the Benveniste method affords the largest degree of freedom and utilises the most information in the adaptation of the $\mu(n)$ values, and thus it exhibits the best convergence properties out of the three GASS algorithms.

However, although the Benveniste algorithm exhibits the best convergence properties, this increase in convergence rate comes at the cost of higher computational complexity. The time taken to carry out the adaptive filtering process is averaged across 100 realisations of the input signal $x(n)$, and the results are shown in **Table 7** below. It can be seen that the Benveniste method takes the longest amount of time to run, requiring three times as much time as the standard LMS algorithm. The Ang & Farhang method requires the next longest amount of time, followed by Matthews & Xie and finally the standard LMS methods. This shows that since the higher convergence speeds in the GASS algorithms are afforded by increasing the degrees of freedom in the adaptation of μ, this increase in speed comes at the price of higher computational complexity.

	LMS, $\mu = 0.1$	LMS, $\mu = 0.01$	Benveniste	Ang & Farhang	Matthews & Xie
Time Taken	0.02613	0.02604	0.06819	0.04278	0.04037

Table 7: Time Taken to run the LMS algorithms over 10000 samples of $x(n)$

Lastly, the effects of varying ρ on the GASS algorithms are investigated. The MSE curves for the three algorithms are plotted separately when ρ is varied, and this is shown in **Figure 45**. Across the three algorithms, the same trend is observed when ρ is increased: the higher the value of ρ, the faster the convergence speed. This corroborates with the idea of ρ being the step size of the update equation for μ. As ρ increases, $\mu(n)$ is more reactive to the error gradient and hence is able to adapt faster, resulting in a higher convergence rate. Again, there appears to be no steady state errors for the GASS algorithms, and the plateauing of the curves seen in the plots of the Benveniste and Ang & Farhang methods are due solely to numerical limitations.

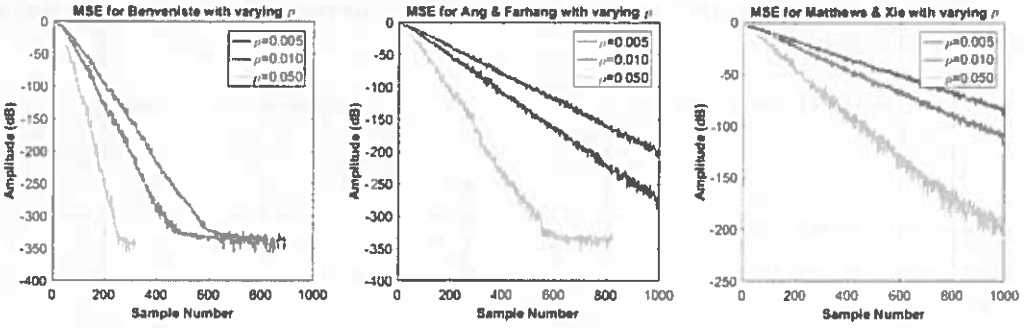


Figure 45: MSE Curved for GASS Algorithm with Varying ρ

b) The update equation of the normalised LMS (NLMS) algorithm is given by

$$\mathbf{w}(n+1) = \frac{\beta}{\epsilon - \|\mathbf{x}(n)\|^2} e(n)\mathbf{x}(n) \tag{52}$$

The update equation based on the a posteriori error, $e_p(n) = d(n) - \mathbf{x}^T(n)\mathbf{w}(n)$ is given by

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \mu e_p(n)\mathbf{x}(n) \tag{53}$$

Multiplying both sides of Eq. (53) by $\mathbf{x}^T(n)$ yields

$$\mathbf{x}^T(n)\mathbf{w}(n+1) = \mathbf{x}^T(n)\mathbf{w}(n) + \mathbf{x}^T(n)\mu e_p(n)\mathbf{x}(n) \tag{54}$$

Shifting and substituting terms gives an expression for $e_p(n)$

$$\underbrace{d(n) - \mathbf{x}^T(n)\mathbf{w}(n)}_{e(n)} - \mathbf{x}^T(n)\mu e_p(n)\mathbf{x}(n) = \underbrace{d(n) - \mathbf{x}^T(n)\mathbf{w}(n)}_{e_p(n)}$$

$$e(n) - \mathbf{x}^T(n)\mu e_p(n)\mathbf{x}(n) = e_p(n)$$

$$e(n) = e_p(n)[1 - \mu\mathbf{x}^T(n)\mathbf{x}(n)]$$

$$e_p(n) = \frac{e(n)}{1 - \mu\mathbf{x}^T(n)\mathbf{x}(n)} \tag{55}$$

Eq. (55) is then substituted back into the a posteriori update equation given in Eq. (53), and this yields

$$\begin{aligned}\mathbf{w}(n+1) &= \mathbf{w}(n) + \frac{\mu e(n)\mathbf{x}(n)}{1 - \mu\mathbf{x}^T(n)\mathbf{x}(n)} \\ &= \mathbf{w}(n) + \frac{\mu}{1 - \mu\|\mathbf{x}(n)\|^2} e(n)\mathbf{x}(n) \\ &= \mathbf{w}(n) + \frac{1}{\frac{1}{\mu} - \|\mathbf{x}(n)\|^2} e(n)\mathbf{x}(n)\end{aligned} \tag{56}$$

Eq. (56) resembles the update equation of the NLMS algorithm given in Eq. (52), with $\epsilon = \frac{1}{\mu}$ and $\beta = 1$. It is thus verified that the update equation based on the a posteriori error is indeed equivalent to the NLMS algorithm. Furthermore, since $\epsilon = \frac{1}{\mu}$, the algorithm will not be unstable even when $\| \mathbf{x}(n) \|^2$ is very small.

c) The Generalised Normalised Gradient Descent (GNGD) algorithm uses an adaptive $\epsilon(n)$ based on a gradient method. The update equation for $\epsilon(n)$ is given as

$$\epsilon(n+1) = \epsilon(n) - \rho\mu \frac{e(n)e(n-1)\mathbf{x}^T(n)\mathbf{x}(n-1)}{\left(\epsilon(n-1)+\mathbf{x}^T(n+1)\mathbf{x}(n-1)\right)^2} \tag{57}$$

This is implemented in Matlab and used to identify the system described by Eq. (51). The weight errors of the coefficient weights are plotted for varying values of ρ and μ, and this is compared with the Benveniste method. The results obtained are shown in **Figure 46**. Once again it can be noted that both algorithms provide weight estimates which asymptotically converges to the real weights, and hence a discussion on the steady state error of both algorithms is unnecessary. When $\rho = 0.005$ is fixed and μ is varied, the performance of the GNGD algorithm with respect to the Benveniste algorithm varies. When μ is small (**Figure 46 (top right)**), the Benveniste algorithm outperforms the GGND algorithm in terms of convergence speed. This is due to the fact that the update equation for $\epsilon(n)$ for the GNGD algorithm uses $\mu\rho$ as a gain coefficient of the error gradient term. Thus, when μ is small and ρ is small (in this case $\mu = 0.05$, $\rho = 0.005$), the change due in $\epsilon(n)$ is almost negligible, and hence the GNGD algorithm does not modify its learning rate at all, thus resulting in a lower convergence rate. On the other hand, since the Benveniste algorithm modifies its step size directly, even if μ starts off small, it will be adapted to reflect the magnitude and direction of the error gradient. This leads to the Benveniste algorithm having a better convergence rate when μ is small. When μ increases, however, the GNGD algorithm outperforms the Benveniste algorithm, as seen in the plot in **Figure 46 (top left)**. The GNGD is able to have a higher convergence rate than its counterpart due to the fact that its learning rate is updated non-linearly, as compared to the Benveniste algorithm where the learning rate is updated linearly. This property of the GNGD also means that it is able to be used effectively when the input signal is non-linear.

interesting comparison between GGND and Benveniste

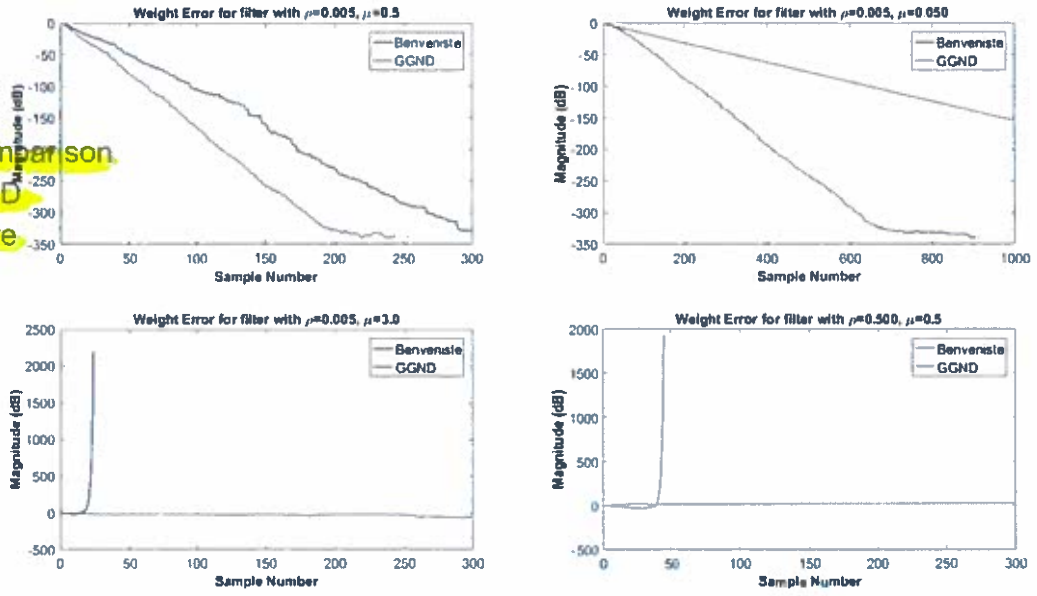


Figure 46: Weight Error of GGND and Benveniste Algorithm for varying μ and ρ

The non-linear updates of the GNGD algorithm also makes it more robust and stable. This is observed in the bottom two plots of **Figure 46**. When μ and ρ are both increased, the Benveniste algorithm is no longer stable while the GGND is still able to predict the weights of the system. In non-stationary signals, the auto-correlation matrix of the input signal varies with time and thus, the maximum eigenvalue of the autocorrelation matrix may exhibit huge fluctuations. It has been established that the maximum value of μ is a function of the maximum eigenvalue of the autocorrelation matrix. Thus, if the eigenvalues change rapidly, the current value of μ used may be too large such that the adaptive filter is no longer stable. Hence, since the GNGD algorithm is able to remain stable over a larger range of parameter variations, the algorithm is more robust and effective than the Benveniste algorithm in processing non-stationary signals.

Next, the computational complexity of the GNGD algorithm and the Benveniste algorithm is analysed. **Table 8** and **Table 9** shows the number of multiplications and additions needed for each iteration of the Benveniste algorithm and

Perfect use of table to break down complexity of algorithm in a clear and correct way.

GNGD algorithm respectively. Here, M refers to the number of coefficients of the model. (Note: For a model with M coefficients, $\mathbf{x}^T(n)\mathbf{x}(n)$ requires M multiplications and $M-1$ additions. $\mathbf{x}(n)\mathbf{x}^T(n)$ requires M^2 multiplications.). It can be seen that the GNGD algorithm is less computationally expensive as compared to the Benveniste algorithm, since it has a complexity of $O(M)$ while the Benveniste algorithm has a complexity of $O(M^2)$. In the experiment conducted where $M = 2$, the GNGD algorithm took $0.0124s$ to carry out 2000 iterations while the Benveniste algorithm required $0.0139s$ to carry out the same number of iterations.

Steps for Benveniste Algorithm	Multiplications	Additions
$e(n) = d(n) - \mathbf{x}^T(n)\mathbf{w}(n)$	M	M
$\mathbf{w}(n+1) = \mathbf{w}(n) + \mu(n)e(n)\mathbf{x}(n)$	$M+1$	M
$\psi(n) = [\mathbf{I} - \mu(n-1)\mathbf{x}(n-1)\mathbf{x}^T(n-1)] \cdot \psi(n-1) + e(n-1)\mathbf{x}(n-1)$	$3M^2 + M$	$2M^2$
$\mu(n+1) = \mu(n) + \rho e(n)\mathbf{x}^T(n)\psi(n)$	$M+2$	M
Total	$3M^2 + 4M + 3$	$2M^2 + 3M$
Order of Complexity	$O(M^2)$	$O(M^2)$

Table 8: Computational Complexity of Benveniste Algorithm

Steps for GNGD Algorithm	Multiplications	Additions
$e(n) = d(n) - \mathbf{x}^T(n)\mathbf{w}(n)$	M	M
$\mathbf{w}(n+1) = \mathbf{w}(n) + \frac{\mathbf{B}}{\epsilon + \mathbf{x}^T(n)\mathbf{x}(n)}$	$2M+2$	$2M$
$\epsilon(n+1) = \epsilon(n) - \rho\mu \frac{e(n)e(n-1)\mathbf{x}^T(n)\mathbf{x}(n-1)}{(\epsilon(n-1) + \mathbf{x}^T(n+1)\mathbf{x}(n-1))^2}$	$2M+4$	$2M$
Total	$5M+6$	$5M$
Order of Complexity	$O(M)$	$O(M)$

Table 9: Computation Complexity of GNGD Algorithm

Based on the results obtained above, it can thus be concluded that the GNGD algorithm is the superior LMS algorithm as it exhibits better convergence speed, greater stability and at the same time is less computationally complex than the Benveniste algorithm.

3.3 Adaptive Noise Cancellation

a) A noise corrupted signal is given by

$$s(n) = x(n) + \eta(n) \tag{58}$$

where $x(n)$ is a unit amplitude sinusoid of angular frequency $\omega_0 = 0.01\pi$ and $\eta(n)$ is coloured noise generate using the moving average filter

$$\eta(n) = v(n) + 0.5v(n-2) \tag{59}$$

An adaptive line enhancer (ALE) can be used to denoise the noisy signal, $s(n)$. The input of the ALE is given as $u(n) = [s(n-\Delta), \dots, s(n-\Delta-m+1)]^T$, where Δ refers to the delay between the signal and the predictor input. Δ is chosen such that the noise in the signal $s(n)$ and $u(n)$ are uncorrelated, thus only the noise and not the desired signal $x(n)$ is suppressed by the linear predictor. For the noise given by Eq. (59), the minimum value for the delay Δ for the ALE can be calculated. To begin, the mean square error equation of the LMS filter can be expanded:

good explanation, with correct and tidy mathematical equations to follow.

$$\begin{aligned} E\left[(s(n) - \hat{x}(n))^2\right] &= E\left[(x(n) + \eta(x) - \hat{x}(n))^2\right] \\ &= E\left[(x(n) - \hat{x}(n) + \eta(n))^2\right] \\ &= E\left[(\eta^2(n)) + (x(n) - \hat{x}(n))^2 + 2\eta(n)(x(n) - \hat{x}(n))\right] \\ &= E[\eta^2(n)] + E\left[(x(n) - \hat{x}(n))^2\right] + 2E[\eta(n)(x(n) - \hat{x}(n))] \end{aligned} \tag{60}$$

Since the optimal solution of the LMS algorithm shows that $MSE = \sigma^2 = E[\eta^2(n)]$, the last two terms of Eq. (60) should be minimised. Expanding the second term yields

$$E\left[(x(n) - \hat{x}(n))^2\right] = E[x^2(n)] + E[\hat{x}^2(n)] - 2E[x(n)\hat{x}(n)] \tag{61}$$

Since changing Δ does not affect the first two terms, the last term is expanded using $\hat{x}(n) = \mathbf{w}^T(n)\mathbf{u}(n)$

$$\begin{aligned} E[x(n)\hat{x}(n)] &= E[x(n)\mathbf{w}^T(n)\mathbf{u}(n)] \\ &= E[x(n)(\textstyle\sum_{i=0}^{m} w_i s(n-\Delta-i))] \end{aligned}$$

$$= E[x(n)(\sum_{i=0}^{m} w_i(x(n-\Delta-i)+\eta(n-\Delta-i)))]$$
$$= E[x(n)(\sum_{i=0}^{m} w_i(x(n-\Delta-i)))] + E[x(n)(\sum_{i=0}^{m} w_i(\eta(n-\Delta-i)))] \quad (62)$$

Since the noise and the desired signal, $x(n)$ are uncorrelated, the last term of Eq. (62) is zero. Now, in order to minimise Eq. (61), the following equation has to be minimised:

$$E\left[\left(x(n)-\hat{x}(n)\right)^2\right] = E[x^2(n)] + E[\hat{x}^2(n)] - 2E[\sum_{i=0}^{m} w_i x(n)x(n-\Delta-i)] \quad (63)$$

Hence, it can be seen that for Eq. (63) to be zero, Δ needs to be zero. However, for the ALE to work, Δ cannot be zero. Furthermore, in order to minimise the above equation requires prior knowledge about the autocorrelation function of the input signal, $x(n)$,which may be impossible to obtain. Hence, there is nothing much which can be done with regards to the minimisation of this term, and attention can be moved to minimising the last term in Eq. (60). Expansion yields

$$E[\eta(n)(x(n)-\hat{x}(n))] = E[\eta(n)x(n)] - E[\eta(n)\hat{x}(n)] \quad (64)$$

Since the noise and signal are again uncorrelated, the first term is zero. Hence, the remaining term can be expanded as

$$E[\eta(n)\hat{x}(n)] = E[\eta(n)\mathbf{w}^T(n)\mathbf{u}(n)]$$
$$= E[\,(\eta(n)(\sum_{i=0}^{m} w_i s(n-\Delta-i))]$$
$$= E[(\eta(n)(\sum_{i=0}^{m} w_i(x(n-\Delta-i)+\eta(n-\Delta-i))]$$
$$= E[\eta(n)(\sum_{i=0}^{m} w_i(x(n-\Delta-i)) + \eta(n)\sum_{i=0}^{m} w_i\eta(n-\Delta-i)] \quad (65)$$

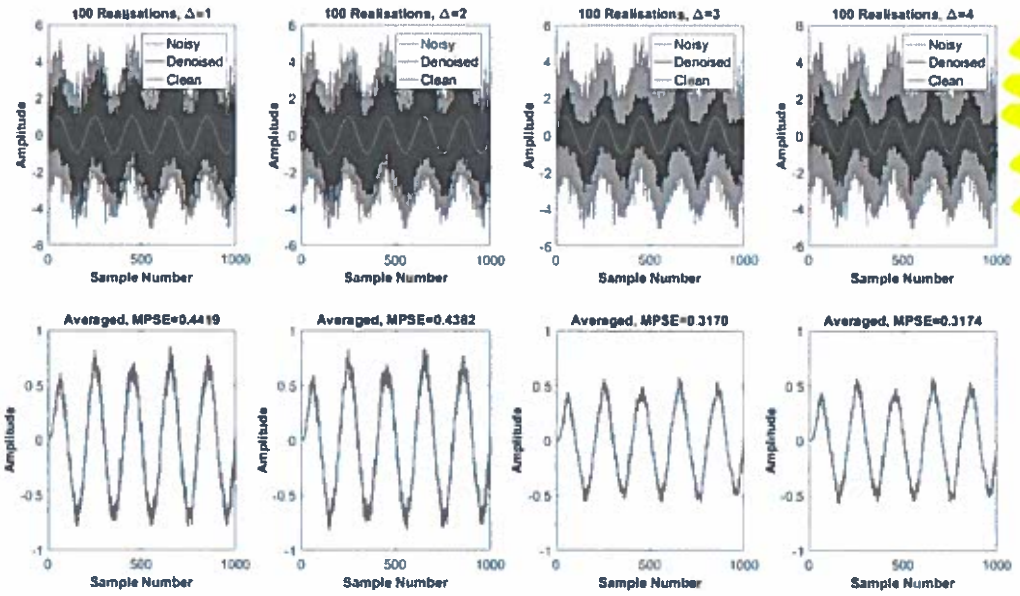
The first term is again zero since the noise term and signals are uncorrelated. This leaves

$$E[\eta(n)\sum_{i=0}^{m} w_i\eta(n-\Delta-i)] = E[(v(n)+0.5v(n-2))(\sum_{i=0}^{m} w_i(v(n-\Delta-i)+0.5v(n-2-\Delta-i))] \quad (66)$$

Since $v(n)$ is WGN, $E[v(i)v(j)] = 0$ when $i \neq j$. With this in mind, the follow equality has to be satisfied

$$E[v(n)v(n-\Delta)] + E[v(n-2)v(n-\Delta)] + E[v(n)v(n-2-\Delta)] + E[v(n-2)v(n-2-\Delta)] = 0 \quad (67)$$

It is easy to see that the above equality will be satisfied for any $\Delta \geq 3$. Thus, the minimum value for the delay Δ to be used in this setting is $\Delta = 3$.



Clear and intuitive graphs to show the performances of the algorithms.

Figure 47: Top: Plots of Noisy and Denoised signal for 100 Realisations of $s(n)$. Bottom: Plots of the ensemble average of the Denoised Signal

In order to justify the choice of Δ, the adaptive line enhancer is implemented in Matlab. 100 realisations of the noisy signal $s(n)$ are independently generated, and these are plotted and shown in red in the top row of **Figure 47**. The denoised signal, $\hat{x}(n)$, from the ALE implemented for varying Δ values are plotted in blue. It is at once obvious that when $\Delta < 3$, the amplitude and variance of the denoised signals is very large as compared to those of the denoised signals when $\Delta \geq 3$. Furthermore, looking at the plots on the bottom row of **Figure 47** which illustrates the ensemble averages of the denoised signal, it can be observed that the graphs for $\Delta \geq 3$ exhibits sine waves which are less noisy as compared to the plots when $\Delta < 3$. In order to quantify this improvement in noise removal, the mean square prediction error of the ALE is calculated when Δ is varied. This is again shown in **Figure 47**, and it can be seen that there is a huge drop in MPSE (a 28% reduction) when Δ is increased from 2 to 3. When Δ is further increased, there appears to be only a very slight improvement in the MPSE.

b) 1000 samples of $s(n)$ are generated and $x(n)$ is estimated using the ALE when different values of filter lengths, M, are used. The adaptation gain of the filter is also varied, and the MPSE values obtained are plotted and shown in **Figure 48 (left)**. Firstly, it can be observed that when μ increases, the MPSE increases. This is expected as the steady state error of the estimator increases when μ increases. A point of interest here is that when μ is increased, the increase in MPSE of the estimator is larger when M is greater. This may be due to the fact that the MPSE formulation used does not take into account the convergence time, and hence the MPSE increases disproportionately when the order of the filter is large, since the convergence speed will be slower.

Next, the effect of increasing M is discussed. As M increases the MPSE first decreases and subsequently increases for all the values of μ tested. This means that for each value of μ, there is an optimal value of M. The optimal filter length for all value of μ always occur at $M = 5$. This may be due to the fact that the sinusoid used is able to be accurately modelled by a 4th order AR process. A reduction in filter length and hence filter order will lead to underfitting of the model, as the model will now have insufficient degrees of freedoms to properly recreate the sinusoid. When M is further increased, the MPSE increases due to the manifestation of inaccuracies due to overfitting. Spurious peaks and artefacts may be introduced into the estimated sine wave, thus increasing the MPSE. Since there is an optimal value of filter length, the use of a 4th order filter may be a pragmatic choice. However, as M increases, the computational complexity and hence costs increases as well. In situations where processing speed is of paramount importance, the optimal value of M may not be the one which provides the lowest MPSE. In this case, other decision criteria such as the MDL be used to determine the optimal filter order.

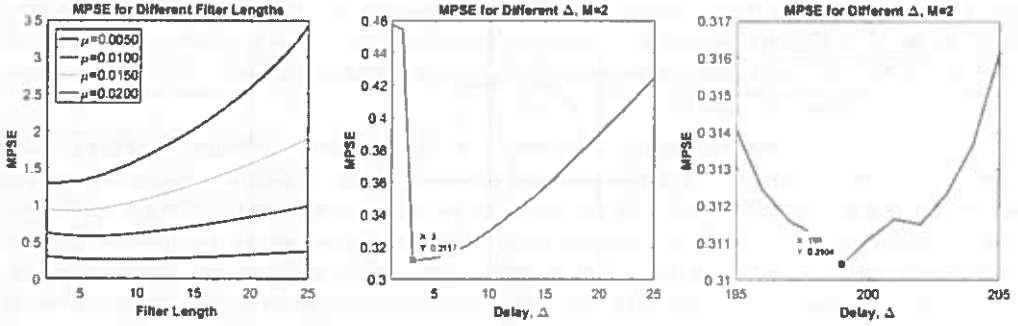


Figure 48: Left: MPSE for different filter lengths and μ. Middle, Right: MPSE for different Δ

Next, the effect of varying Δ on the MPSE is investigated. Δ was first varied from 1 to 25, and the MPSE values obtained are shown in **Figure 48 (middle)**. As expected, when Δ is increased from 1 to 3, there is a huge drop in MPSE since the noise in the signal $s(n)$ and the predictor input are now uncorrelated, as explained previously. However, when Δ continues to increase from 3 to 25, the MPSE increases. This can be explain by revisiting Eq. (63) which is shown below:

$$E\left[\left(x(n) - \hat{x}(n)\right)^2\right] = E[x^2(n)] + E[\hat{x}^2(n)] - 2E[\textstyle\sum_{i=0}^{m} w_i x(n)x(n-\Delta-i)] \tag{68}$$

The term $E\left[\left(x(n) - \hat{x}(n)\right)^2\right]$ has to be minimised, and to do this, $x(n)$ and $\sum_{i=0}^{m} w_i x(n-\Delta-i)$ must be fully correlated. As the delay increases, since $x(n)$ is a sinusoid, the autocorrelation will decrease. This decrease in autocorrelation leads to an increase in $E\left[\left(x(n) - \hat{x}(n)\right)^2\right]$ due to the negative sign of the third term, and this in turn increases the MSE, resulting in a higher MPSE. Now, since it is known that the input is a sinusoid, the periodicity of the sinusoid can be exploited. Since the autocorrelation of a sinusoid will be unity if the delay is exactly equal to an integer multiple of its period, Eq. (68) can be minimised when Δ is equal to 199. When $\Delta = 199$, both second and third terms in Eq. (60) are zero, and hence the MSE will be exactly equal to σ^2. The MPSE of the ALE is obtained when Δ is varied from 195 to 205 **(Figure 48 (right))**. As expected, there is a sharp dip when $\Delta = 199$, and the MPSE at this value is 0.3104, which is smaller than the MPSE at $\Delta = 3$ (0.3117).

c) The adaptive noise cancellation (ANC) configuration of the LMS filter is implemented in Matlab. The denoised waveforms for both the ALE and ANC algorithms are plotted and shown in **Figure 49**, along with the ensemble average of the denoised signals. It is clear from the figure that the ANC algorithm has better denoising performance than the ALE algorithm once the algorithm has converged. This can be seen by the smoother ensemble average of the estimated $x(n)$ of the ANC algorithm as compared to that of the ALE algorithm. However, the ANC algorithm takes a much longer time than the ALE algorithm to converge to a steady state solution. This is evident from the large transient present in the ANC's denoised signals plot. The MSPE values for the entire signal, as well as the steady state MPSE values are calculated for both algorithms and reproduced in **Table 10**. From the MPSE values, it can be concluded that the ANC configuration performs better than the ALE configuration. Furthermore, from the large difference between its full MPSE and steady state MPSE (about 3 times) illustrates that the ANC has very slow convergence rate, as compared to the ALE configuration since the full MPSE and steady state MPSE of the ALE configuration are very similar.

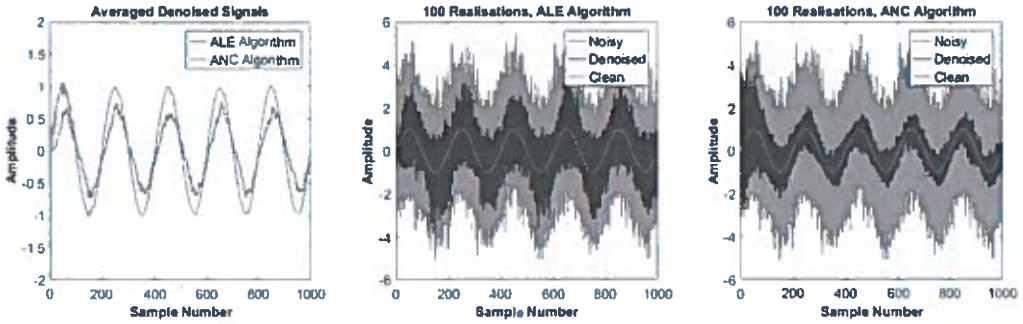


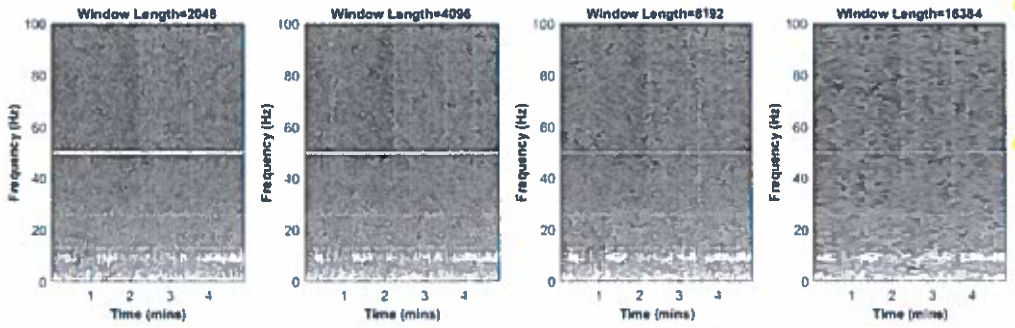
Figure 49: Comparison of ALE and ANC Configurations for Denoising of a Sinusoid

	ANC	ALE
$MPSE_{full}$	0.0779	0.3081
$MPSE_{steady\ state}$	0.0265	0.3050

Table 10: MPSE for ANC and ALE configurations

d) The ANC configuration is used to suppress the strong 50Hz component introduced in the mains in the POz data by generating a synthetic reference input composed of a 50Hz sinusoid corrupted by WGN ($\sigma^2 = 1$). The step size, μ, and filter length, M, of the ANC filter are varied and the spectra of the "denoised" signals are plotted using the `spectrogram` function in Matlab. A 50% overlap between segments is used, while a hamming window is used for each segment since it has good side lobe suppression properties (low spectral leakages).

To begin, the spectrogram of the original POz signal is plotted and shown in **Figure 50** when the window length of the spectrogram function is varied. As expected, when the window length is small (2048), the yellow line corresponding to the mains interference is very diffuse and thick due to a decrease in frequency resolution (wide main lobes). When the window length is large (16384), the line corresponding to $50Hz$ is very thin and strong, but the lower frequency appears blurred due to the increased variance. The line at a frequency of $26Hz$ cannot be discerned. A good balance between low variance and high resolution is achieved when the window length is 4096, and this is used for the rest of the spectrograms in this section.



Care has been taken not to corrupt the signal.

Figure 50: Spectrogram of Original POz Signal with varying Window Length

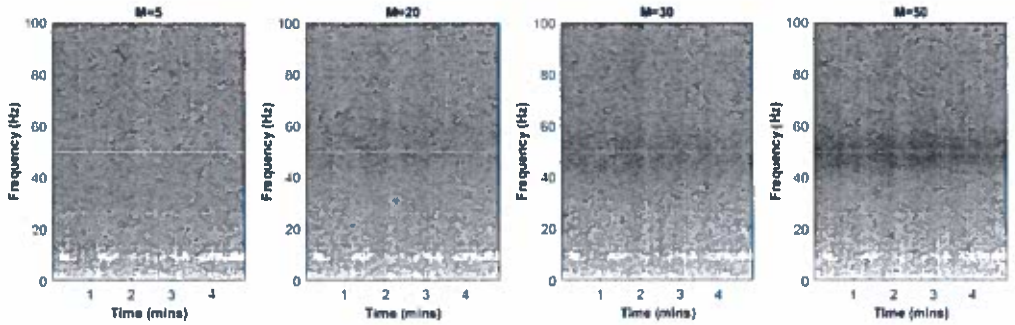


Figure 51: Spectrogram of Denoised Signal with varying M

Next, the effect of varying M, the filter length, is investigated. The spectrograms obtained are shown in **Figure 51**. It can be seen that when M is too small (M=5), the ANC filter is unable to model the $50Hz$ sine wave, and hence the $50Hz$ mains component is not removed. When $M = 20$, the $50Hz$ mains component is adequately removed, while the

spectrum due to the other frequency components are preserved. When M is further increased ($M = 30$), some of the frequency lines, especially the one at $26Hz$ can no longer be seen. Increasing M further to 50 further blurs the lower end of the frequency spectrum, while introducing distortions into the power spectrum by suppressing signal components in the range of $40Hz - 60Hz$. The above discussion is corroborated by looking at the Bartlett's Periodogram of the denoised signal when different values of M are used. Based on the both the spectrogram and periodogram, the best value of M which results in sufficient suppression of the $50Hz$ interference without affecting any of the other frequency components is chosen to be $M = 20$.

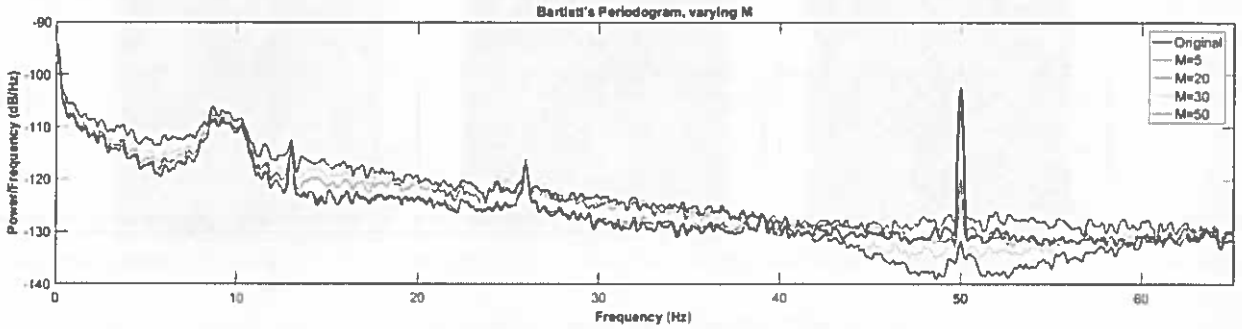


Figure 52: Bartlett's Periodogram of Denoised Signal with varying M

In order to reduce the effect of the ANC filter on frequency components apart from the $50Hz$ mains interference, μ can be varied. Since the length of the POz signal is very large (354000) and the signal is stationary, the convergence speed is not a significant issue. Thus, a small μ can be used to reduce the steady state misadjustment error, so that the power spectrum of the signal is closer to the unfiltered version at all other frequencies except for $50Hz$. The spectrograms obtained when μ is varied is shown in **Figure 53**. As expected, when $\mu = 0.001$ is small, the spectrogram obtained has the least distortion at low frequencies, although the $50Hz$ line has been removed. This is evident again from the present $26Hz$ spectral line, which disappears as μ is increased. When μ is increased significantly to 0.02, the blurring of the spectral lines of the lower frequency components increases, and this is undesirable. The Bartlett's periodogram is again plotted for all four denoised signals (**Figure 53**), and the estimated PSD agrees with the above discussion. When $\mu = 0.001$, it can be seen that the periodogram of the denoised signal closely resembles that of the original signal except at $50Hz$, where the peak has been largely diminished.

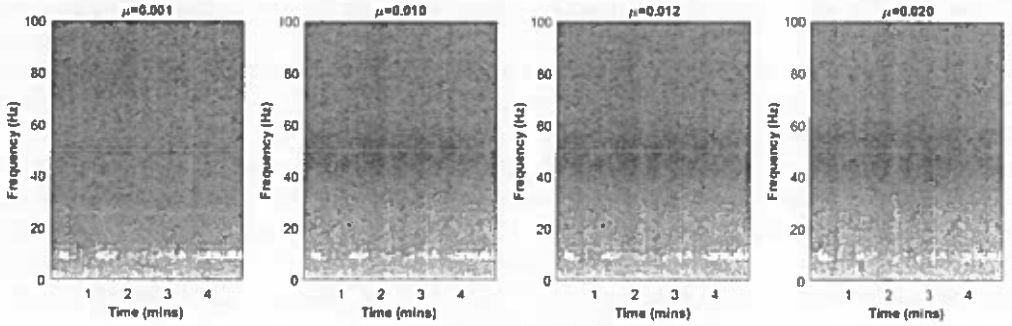


Figure 53: Spectrogram of Denoised Signal with varying μ

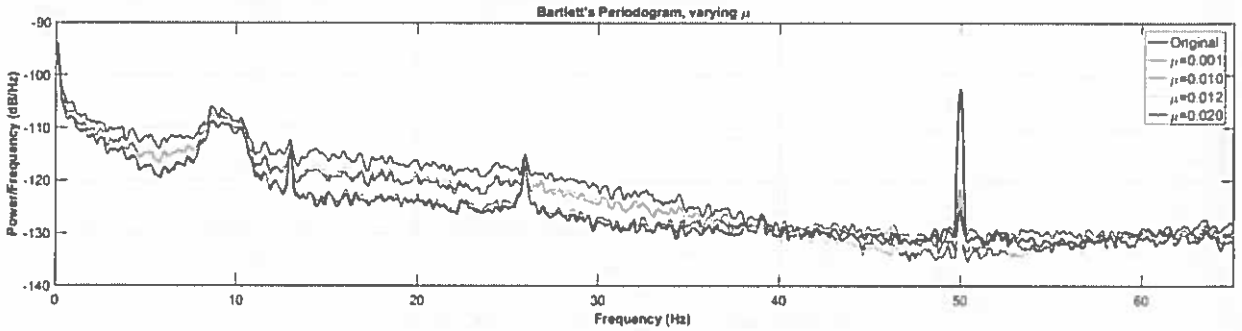


Figure 54: Bartlett's Periodogram of Denoised Signal for varying μ

Lastly, the noise used to corrupt the synthetic sine wave of $50Hz$ is varied and it is discovered that when the noise power is reduced ($\sigma^2 = 1$ in the previous discussions), a much lower value of M can be used to produce similar results.

Figure 55 shows the spectrogram of the denoised signal when $\mu = 0.001$ and $M = 10$. As can be seen, when $\sigma^2 = 0.1$, the performance of the ANC is almost perfect, with the $50Hz$ component removed while the rest of the frequency components are unchanged. However, when σ^2 is increased, the mains interference is no longer removed since the ANC can no longer model the noisy sine wave. In this example, it has been shown that an ANC filter of length $M = 10$ is able to produce similar results as an ANC filter of length $M = 20$ just by varying σ^2. Since the computation cost increases with M, this method of denoising may be more computationally efficient.

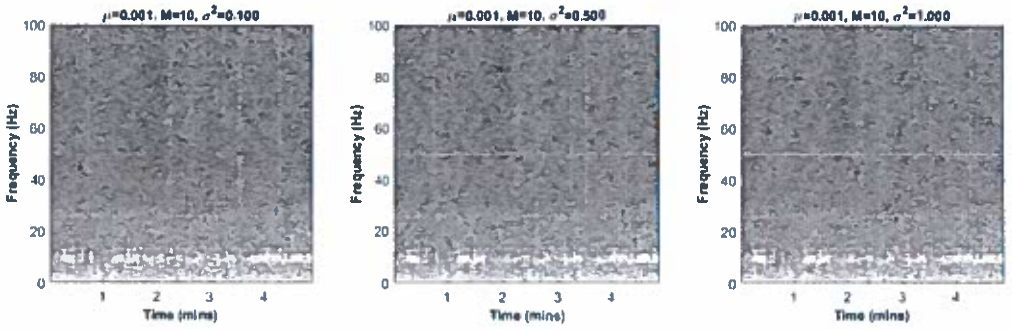


Figure 55: Spectrogram of Denoised Signal with varying Noise Power, σ^2

4 WIDELY LINEAR FILTERING & ADAPTIVE SPECTRAL ESTIMATION

Appropriate comparison of CLMS and ACLMS algorithms.

4.1 COMPLEX LMS AND WIDELY LINEAR MODELLING

a) A first order widely-linear-moving-average process (WLMA(1)) is given by

$$y(n) = x(n) + b_1 x(n-1) + b_2 x^*(n-1) \qquad x \sim \mathcal{N}(0,1) \tag{69}$$

where $b_1 = 1.5 + 1j$ and $b_2 = 2.5 - 0.5j$. The WLMA(1) process is generated and the complex LMS (CLMS) and augmented complex LMS (ACLMS) methods are used to identify the WLMA model. The WLMA(1) process is not a circular process. The learning curves of the CLMS and ACLMS algorithms are obtained and plotted in **Figure 56**. It can be seen that the CLMS algorithm is unable to model the WLMA(1) process, resulting in large steady state squared error ($8.8543dB$). The ACLMS algorithm on the other hand is able to model the WLMA(1) process very perfectly, since its steady state squared error is only $-307.26dB$. The large error values obtained when the CLMS algorithm is used is due to the fact that the CLMS algorithm does not have sufficient degrees of freedom to model the $x^*(n-1)$ term in the WLMA system. The steady state values of the weight coefficients of the CLMS filter is $h_{1,CLMS} = 1.0053 + 0.0043j$ and $h_{2,CLMS} = 1.5059 + 0.9940i$. These two coefficient values are very close to the true coefficient values of $x(n)$ and $x(n-1)$ in the WLMA(1) system. This shows that although the CLMS is able to accurately model the $x(n)$ terms in the system, the large error occurs because the conjugate terms are not being modelled at all. The ACLMS is thus able to very accurately model the process since the additional weights $\mathbf{g}(n)$ allows the ACLM filter sufficient degrees of freedom to model both the $x(n)$ and $x^*(n)$ terms. The steady state values of the weight vector $\mathbf{g}(n)$ is $g_{1,ACLMS} = 0$ and $g_{2,ACLMS} = 2.5 - 0.5j$ and these corresponds to the coefficients of the $x^*(n)$ terms in the WLMA(1). This increase in the degree of freedom allows the ACLMS filter to model non-circular data, such as the WLMA(1) system.

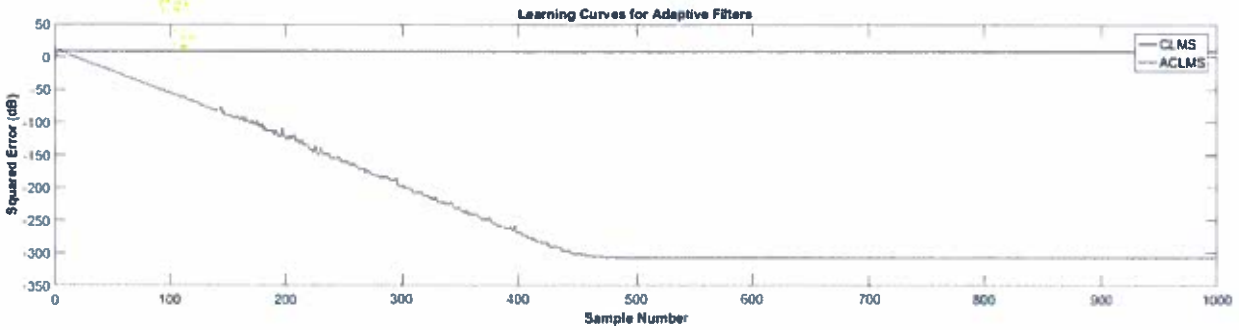


Figure 56: Learning Curves of CLMS and ACLMS Filters

b) **Figure 57** shows the scatter plots (also known as circularity plots) of the signal for the three wind regimes. The red dotted lines in the figure represents the mean of the data, while the black dotted line represents the origin. Although at first glance, it may be concluded that the low wind data is the least circular since its scatter plot resembles an ellipse the

most, while the scatter plot of the high wind data is the most circular as it most close resembles a circle. This is however, incorrect. The circularity of a complex-valued random variable is dependent on its probability distribution. If the probability distribution is only affected by its magnitude and not the direction (in the complex plane), then the complex-valued random variable is circular. This means that instead of checking if the distribution of a complex random variable is rotational invariant about its mean, the correct way to determine if the random variable is circular is to check if it is rotational invariant about the origin in the complex plane. A better way to check for the circularity of a complex variable is to utilise the concept of covariance and pseudo covariance. The covariance of a complex random variable is given by $c = E[\mathbf{z}\mathbf{z}^*]$ and the pseudo covariance is given by $p = E[\mathbf{z}\mathbf{z}]$, where $\mathbf{z}$ is a row vector of N realisations of the complex random variable Z. The circularity coefficient is thus defined as

$$\eta = \frac{|p|}{c} \tag{70}$$

The higher the circularity coefficient, the less circular the random variable is. Using this formula, the circularity coefficients of the three different wind data are calculated, and they are shown in **Figure 58**. This shows that in actual fact, the high wind data is the least circular, while the low wind data is the most circular. This can be explained by looking at how the wind signal is formed. The complex component of the wind signal is formed from the measured wind speeds in the north-south direction, while the real component is formed from the measured wind speeds in the east-west direction. When there is strong wind, the wind speeds are highly directional, and this results in the high wind data being the most non-circular.

Good plots of the three wind regimes, showing a deep understanding of the circularity coefficient and its physical meaning.

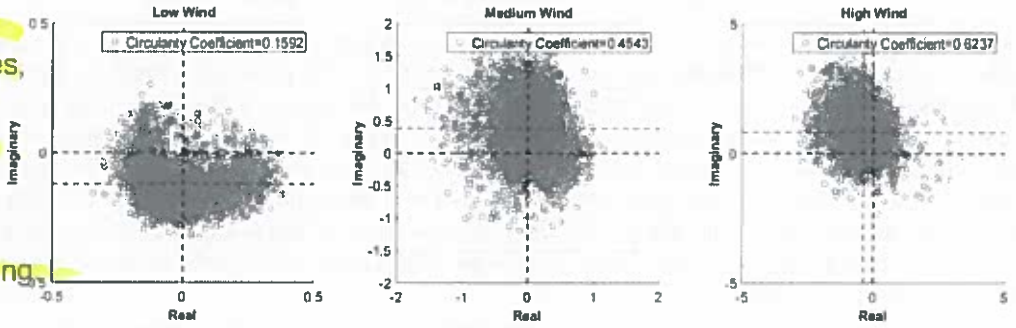


Figure 57: Circularity Plot for Wind Signal from three Wind Regimes

Next, the CLMS and ACLMS algorithms are used to perform a one step ahead prediction of the complex wind data from each of the three wind regimes, and the MPSE obtained when the filter length, M, is varied is shown in **Figure 58**. A constant value of $\mu = 0.001$ is used in all cases. It can be seen from the plots that the performance of the CLMS and ACLMS filters are as expected. As previously mentioned, the CLMS filter is unable to model non-circular data, and this inability is again evident. For the low and medium wind regime, the ACLMS and CLMS filters performed equally well since the wind data has low non-circularity. However when the non-circularity of the wind data increases as is the case in the high wind regime, the CLSM performs worse than the ACLMS filter at small M. For all three wind regimes, the MPSE of the filters decreases initially as model length increases. This is because there are now sufficient coefficients to properly model the wind data, and underfitting of the model is avoided. However, as M continues to increase, the MPSE starts to rise in the high and medium wind regimes. This happens at $M = 6$ for the medium wind regime and $M = 2$ for the high wind regime. This happens due to the overfitting of the model. In a one-step prediction model, overfitting happens because the filter models even the spurious fluctuations in the current data, resulting in a loss in generality and hence a poorer prediction ability.

well presented comparison between the MPSE computed with the CLMS and ACLMS.

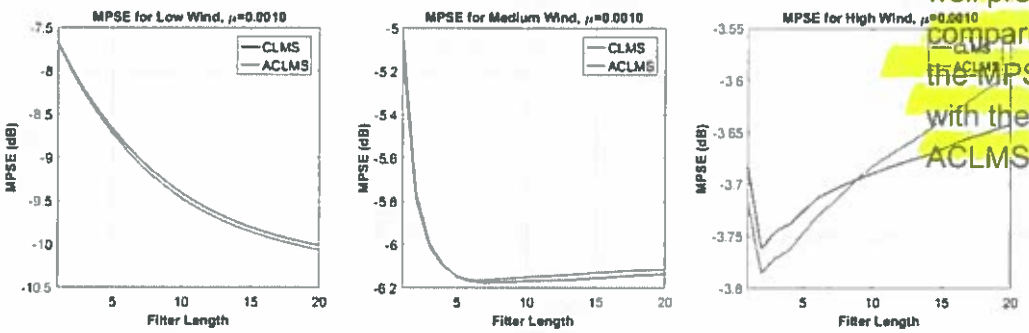


Figure 58: Plot of MPSE for CLMS and ACLMS Filters with varying M

Another observation which can be made is that as M increases and overfitting occurs, the MPSE of the ACLMS filter increases at a faster rate than that of the CLMS filter. This is due to the fact that the ACLMS filter has more degrees of freedom than the CLMS filter with equal filter length (ACLMS has twice the number of filter coefficients) and hence,

the effects of overfitting is more severe for the ACLMS algorithm, and its performance degrades faster as M is increased. Overfitting does not happen in the low wind regime due to the lack of sudden fluctuations. In medium and high wind regimes, there are turbulent flows which result in sudden changes in direction or magnitude of the wind velocity. An overfitted model will attempt to model these sharp and abrupt changes in the current sample, and will hence be unable to accurately predict the value of the next sample. However, since such turbulent air flow is not present when the wind speed is low, the effects of overfitting are not pronounced as the wind speed and direction are both either nearly constant or slow changing. Lastly, when wind speed increases, the MPSE of both filters increases, implying that there are more errors in prediction at higher wind speeds.

4.1.1 Frequency Estimation in Three Phase Power Systems

c) The complex-valued voltages in a three phase system arising from the Clarke Transform can be expressed as

$$v(n) = \mathrm{A}(n)e^{j\left(2\pi\frac{f_o}{f_s}n+\phi\right)} + \mathrm{B}(n)e^{-j\left(2\pi\frac{f_o}{f_s}n+\phi\right)} \tag{71}$$

with

$$\mathrm{A}(n) = \frac{\sqrt{6}}{6}(V_a(n) + V_b(n)e^{j\Delta_b} + V_c(n)e^{j\Delta_c}) \tag{72}$$

$$B(n) = \frac{\sqrt{6}}{6}(V_a(n) + V_b(n)e^{-j(\Delta_b+\frac{2\pi}{3})} + V_c(n)e^{-j(\Delta_c+\frac{2\pi}{3})}) \tag{73}$$

A system is balanced when $V_a(n) = V_b(n) = V_c(n)$, and $\Delta_b = \Delta_c = 0$. A balanced complex voltages is generated and its circularity plot is shown in **Figure 59 (left)**. Imbalanced complex values are also generated when V_a, V_b and V_c are varied (**Figure 59 (middle)**), as well as when Δ_b and Δ_c are changed (**Figure 59 (right)**). When the voltage is balanced, the complex signal is circular, as it is rotational invariant about the origin. However, when either the amplitudes are unequal or if the phase differences, Δ, are not equal to zero, the resulting voltage is no longer circular. This is manifested in the circularity plot as either a stretching or shrinking of the circular plot in a certain direction, changing the circularity plot of the imbalanced signal into an ellipse. Thus, it is possible to tell whether there is a fault in a system just by using a circularity plot of the voltages. If there is a fault (i.e. the voltages are imbalanced or a phase shift is introduced), the circularity plot of the voltage will no longer be circular.

Clear representation of faulty systems, with different parameters.

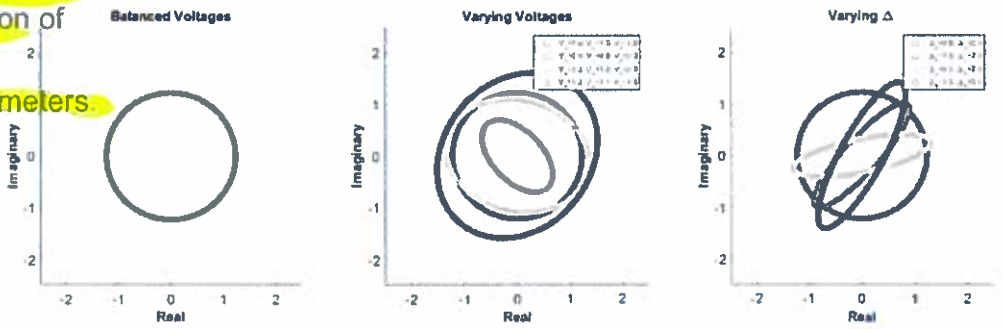


Figure 59: Circularity Plots of Balanced and Unbalanced Complex Voltages

d) A strictly linear autoregressive model of order 1 can be described as

$$v(n+1) = h^*(n)v(n) \tag{74}$$

And the $\alpha - \beta$ voltage of a balanced system is represented as

$$v(n) = \sqrt{\frac{3}{2}}Ve^{j\left(2\pi\frac{f_o}{f_s}n+\phi\right)} \tag{75}$$

Substituting Eq. (75) into Eq. (74) yields

$$\sqrt{\frac{3}{2}}Ve^{j\left(2\pi\frac{f_o}{f_s}(n+1)+\phi\right)} = h^*(n)\sqrt{\frac{3}{2}}Ve^{j\left(2\pi\frac{f_o}{f_s}(n+1)+\phi\right)} \tag{76}$$

Rearranging Eq. (76), and using the identity $z = x + iy = |z|e^{j(\arctan\left(\frac{\Im(z)}{\Re(z)}\right)}$,

$$\frac{e^{j\left(2\pi\frac{f_o}{f_s}n\right)}e^{j\left(2\pi\frac{f_o}{f_s}\right)}e^{i\phi}}{e^{j\left(2\pi\frac{f_o}{f_s}n\right)}e^{i\phi}} = h^*(n)$$

$$e^{j\left(2\pi\frac{f_o}{f_s}\right)} = h^*(n)$$

$$= |h(n)|e^{-j\left(\arctan\left(\frac{\Im\{h(n)\}}{\Re\{h(n)\}}\right)\right)} \tag{77}$$

The phase can then be equated, yielding an expression for $f_o(n)$

$$2\pi\frac{f_o}{f_s} = -\arctan\left(\frac{\Im\{h(n)\}}{\Re\{h(n)\}}\right)$$

$$f_o(n) = -\frac{f_s}{2\pi}\arctan\left(\frac{\Im\{h(n)\}}{\Re\{h(n)\}}\right) \tag{78}$$

And since frequency is positive (negative frequencies merely defines the direction of angular rotation)

$$f_o(n) = \frac{f_s}{2\pi}\arctan\left(\frac{\Im\{h(n)\}}{\Re\{h(n)\}}\right) \quad (shown) \tag{79}$$

For unbalanced voltages on the other hand, the widely linear AR(1) model has to be used, and this is given by

$$v(n+1) = h^*(n)v(n) + g^*(n)v^*(n) \tag{80}$$

Substituting the imbalanced voltage equation given by Eq. (71) yields

$$\begin{aligned}&\mathrm{A}(n+1)e^{j\left(2\pi\frac{f_o}{f_s}(n+1)+\phi\right)} + \mathrm{B}(n+1)e^{-j\left(2\pi\frac{f_o}{f_s}(n+1)+\phi\right)}\\ &= \mathrm{h}^*(n)A(n)e^{j\left(2\pi\frac{f_o}{f_s}n+\phi\right)} + h^*(n)\mathrm{B}(n)e^{-j\left(2\pi\frac{f_o}{f_s}n+\phi\right)} + y^*(n)\mathrm{A}^*(n)e^{-j\left(2\pi\frac{f_o}{f_s}n+\phi\right)} + g^*(n)\mathrm{B}^*(n)e^{j\left(2\pi\frac{f_o}{f_s}n+\phi\right)}\end{aligned} \tag{81}$$

Next, equating and eliminating exponential terms on both sides gives

$$A(n+1)e^{j\left(2\pi\frac{f_o}{f_s}\right)} = h^*(n)\mathrm{A}(n) + g^*(n)\mathrm{B}^*(n) \tag{82}$$

$$\mathrm{B}(n+1)e^{-j\left(2\pi\frac{f_o}{f_s}\right)} = h^*(n)\mathrm{B}(n) + g^*(n)\mathrm{A}^*(n) \tag{83}$$

Assuming that $v_i(n) \approx v_i(n+1)$ for $i = (1,2,3)$ such that $\mathrm{B}(n+1) \approx \mathrm{B}(n)$ and $\mathrm{A}(n+1) \approx \mathrm{A}(n)$,

$$A(n)e^{j\left(2\pi\frac{f_o}{f_s}\right)} = h^*(n)\mathrm{A}(n) + g^*(n)\mathrm{B}^*(n) \tag{84}$$

$$\mathrm{B}(n)e^{-j\left(2\pi\frac{f_o}{f_s}\right)} = h^*(n)\mathrm{B}(n) + g^*(n)\mathrm{A}^*(n) \tag{85}$$

Now, the equation is rearranged

$$e^{j\left(2\pi\frac{f_o}{f_s}\right)} = \frac{h^*(n)\mathrm{A}(n)+g^*(n)\mathrm{B}^*(n)}{\mathrm{A}(n)} = h^*(n) + g^*\frac{\mathrm{B}^*(n)}{\mathrm{A}(n)} \tag{86}$$

$$e^{-j\left(2\pi\frac{f_o}{f_s}\right)} = \frac{h^*(n)\mathrm{B}(n)+g^*(n)\mathrm{A}^*(n)}{\mathrm{B}(n)} = h^*(n) + g^*\frac{\mathrm{A}^*(n)}{\mathrm{B}(n)} \tag{87}$$

Taking the conjugate of Eq. (87) and substituting it into Eq. (86) yields

$$h(n) + g\frac{\mathrm{A}(n)}{\mathrm{B}^*(n)} = h^*(n) + g^*\frac{\mathrm{B}^*(n)}{\mathrm{A}(n)} \tag{88}$$

Let $\frac{\mathrm{B}^*(n)}{\mathrm{A}(n)} = x$ and do a change of variables and rearrange to obtain a quadratic equation

$$h(n) + g(n)\frac{1}{x} = h^*(n) + g^*(n)x$$

$$g^*(n)x^2 + [h^*(n) - h(n)]x - g(n) = 0 \tag{89}$$

Now, solve for x

$$\begin{aligned}x &= \frac{[h(n)-h^*(n)]\pm\sqrt{(h^*(n)-h(n))^2+4|g(n)|^2}}{2g^*(n)}\\ &= \frac{2\Im[h(n)]j\pm\sqrt{-4\Im[h(n)]^2+4|g(n)|^2}}{2g^*(n)}\\ &= \frac{2\Im[h(n)]j\pm 2j\sqrt{\Im^2[h(n)]-|g(n)|^2}}{2g^*(n)}\\ &= \frac{j\left[\Im[h(n)]\pm\sqrt{\Im^2[h(n)]-|g(n)|^2}\right]}{g^*(n)}\end{aligned} \tag{90}$$

Substituting x back into Eq. (86) gives

$$\begin{aligned}e^{j\left(2\pi\frac{f_o}{f_s}\right)} &= h^*(n) + g^*(n)\left[\frac{j[\Im[h(n)]\pm\sqrt{\Im^2[h(n)]-|g(n)|^2}]}{g^*(n)}\right]\\ &= h^*(n) + \Im[h(n)]j \pm j\sqrt{\Im^2[h(n)] - |g(n)|^2}\\ &= \Re[h(n)] \pm j\sqrt{\Im^2[h(n)] - |g(n)|^2}\\ &= |c|e^{j\left[\arctan\left(\frac{\pm\sqrt{\Im^2[h(n)]-|g(n)|^2}}{\Re[h(n)]}\right)\right]}\end{aligned} \tag{91}$$

where c is an arbitrary constant. Equating exponentials again yields

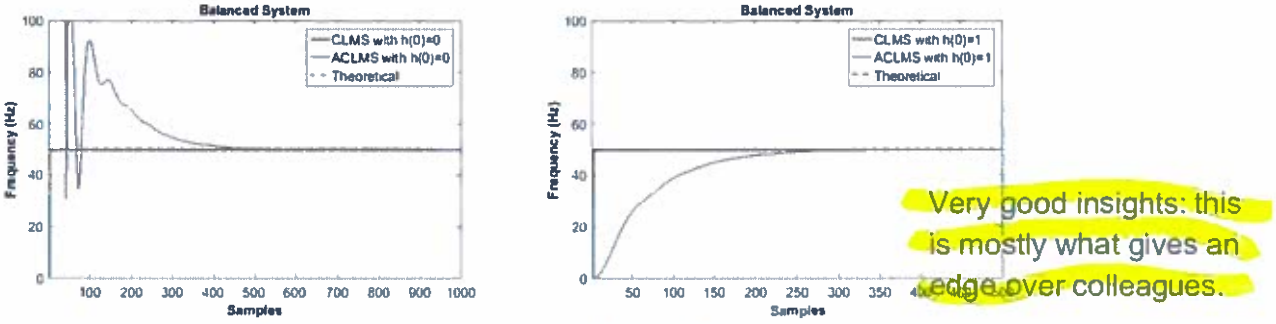
$$2\pi\frac{f_o}{f_s} = \arctan\left[\frac{\pm\sqrt{\Im^2[h(n)]-|g(n)|^2}}{\Re[h(n)]}\right]$$

$$f_o = \pm\frac{f_s}{2\pi}\arctan\left[\frac{\sqrt{\Im^2[h(n)]-|g(n)|^2}}{\Re[h(n)]}\right] \quad (92)$$

Again, ignoring the negative frequency, f_o is given as

$$f_o = \frac{f_s}{2\pi}\arctan\left[\frac{\sqrt{\Im^2[h(n)]-|g(n)|^2}}{\Re[h(n)]}\right] \ (proved) \quad (93)$$

e) The ACLMS and CLMS algorithms are used to estimate the frequency of $\alpha-\beta$ voltages of three systems: a balanced systems, an unbalanced system due to voltage differences, and an unbalanced system due to phase differences. The value of f_0 was set to $50Hz$ to mimic the frequency of the UK power system. The balanced voltage is circular, while the imbalanced voltages are not. The circularity coefficients of the 2 unbalanced voltages are: $\eta_{2nd\ signal} = 0.4547$ and $\eta_{3rd\ signal} = 0.6719$. The order of the system is set to 1, and $\mu = 0.01$ for both algorithms. The balanced voltage is first tested on the two filters when $\mathbf{h}(0) = 0$ and the estimated frequency is shown in **Figure 60 (left)**. It can be observed that the CLMS algorithm converges at a much faster rate (3 samples) than the ACLMS algorithm (500 samples), and both algorithms are able to correctly predict the $50Hz$ frequency in the steady state. The predicted frequency plot for the ACLMS algorithm shows a large overshoot initially (reaching infinite value) in the third iteration, before gradually decreasing. This does not happen in the implementation of the CLMS algorithm.



Very good insights: this is mostly what gives an edge over colleagues.

Figure 60: Comparison of CLMS and ACLMS Algorithms for Balanced Voltages

To explain why this happens, the method used to estimate frequency is important. For the ACLMS algorithm, the method of calculating the estimated frequency (Eq. (93)) includes a square root function. During the first few iterations of the algorithm, when the weight vectors are all initialised to zero, $\Im^2[h(n)] - |g(n)|^2$ will sum to negative, and this creates an imaginary number. In order to take the *arctan* of an imaginary number, the Matlab function `atan` must be used. For the CLMS algorithm on the other hand, the frequency estimation does not have a square root function, and hence, imaginary numbers can never be introduced and the `atan2` function in Matlab can be used instead. The first problem with using the `atan` function instead of the `atan2` function is that if $\Re[h(n)] = 0$, `atan` will return `NaN` while `atan2` will return the correct value of $\frac{\pi}{2}$. This results in the loss of the first two frequency values when `atan` is used. The second problem arises when $\Im^2[h(n)] - |g(n)|^2$ gives a negative number and $\frac{\sqrt{\Im^2[h(n)]-|g(n)|^2}}{\Re[h(n)]} \approx 1j$. The inverse tangent of the imaginary number 1j gives a value of infinity, and this is why the initial portion of the response for the ACLMS filter tends to infinity. The subsequent overshoots are then a function of the sampling frequency, i.e. if the sampling frequency is large, the overshoot is large. This is due to the f_s factor in Eqn. (93). The solution to circumvent this problem is simple. $h(0)$ can be initialised to a non-zero value such that the term in the square root will not be negative, and by doing so `atan2` can be used. For the signals used, this is achieved by initialising $h(0) = 1$. In order to obtain a fair comparison between performance of the ACLMS and CLMS, the initialisation is done for both algorithms, and the resulting frequency estimation curve is shown in **Figure 60 (right)**. As can be seen, the overshoot in the ACLMS curve is removed, and convergence is achieved after about 350 samples, which is better than the convergence rate when $h(0) = 0$. For the CLMS algorithm, convergence is again achieved after 3 iterations, and thus, the change in initialisation value of $h(n)$ did not affect its performance.

The performance of the CLMS and ACLMS algorithms are then compared when non-circular data is used. In this experiment, $h(0) = 1$ for reasons explained previously. The frequency estimation curves of the two algorithms when two unbalanced voltages with different circularity coefficients are shown in **Figure 61**. Once again, the CLMS exhibits a faster convergence speed as compared to the ACLMS algorithm. The convergence speed of the CLMS is also unaffected by the changes in circularity coefficients of the voltage signals. The ACLMS algorithm on the other hand shows faster convergence when the circularity coefficient of the voltage signal is lower (1000 samples for $\eta = 0.455$ and 1500 samples for $\eta = 0.672$). When unbalanced voltages are used the ACLMS algorithm is still able to converge to the correct frequency

value of $50Hz$ without any bias in the steady state. The CLMS algorithm however, is unable to do so, converging to a steady state value of $44Hz$ when $\eta = 0.455$ and $37Hz$ when $\eta = 0672$. This is expected since the CLMS algorithm is only able to model circular data sets due to the lack of sufficient degrees of freedom (since it only has a set of weight vectors) to handle both the signal and its complex conjugate, which is needed if the input signal is non-circular. This is corroborated by the fact that when the circularity coefficient decreases (the signal is more circular), the bias at steady state decreases, as well as the error ripple in the steady state estimation values. As η approaches zero, the response of the CLMS algorithm will approach the plot seen previously in **Figure 60**.

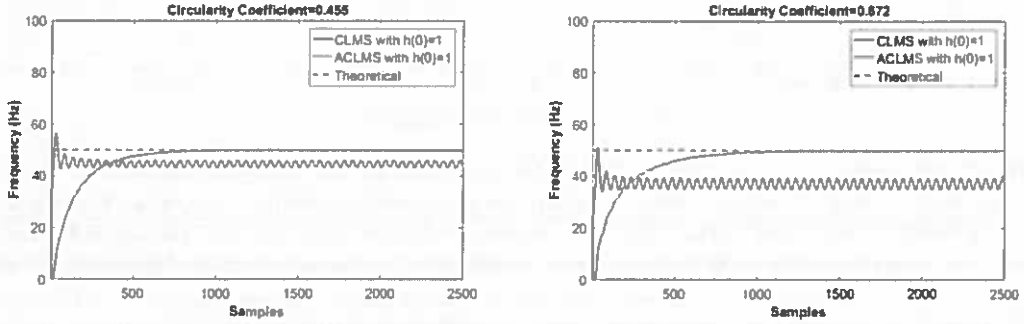


Figure 61: Comparison of CLMS and ACLMS for Unbalanced Complex Voltages

4.2 Adaptive AR Model Based Time-Frequency Estimation

a) A complex frequency modulated (FM) signal is generated and corrupted with circular complex-valued white noise with zero mean and variance $\sigma^2 = 0.05$. The frequency variation of the FM signal is plotted and shown in **Figure 62 (left)**. It can be seen that the frequency variation can be segmented into three blocks of 500 samples each. In the first block, the frequency is constant ($100Hz$), in the second block the frequency increases linearly and in the third block, the frequency increases quadratically. The input signal is thus not stationary. The signal is next modelled as an AR(1) process and the resulting PSD estimate is shown in **Figure 62 (right)**. Since the `aryule` function performs block based analysis on the FM signal, the resulting PSD estimate is unable to capture all the frequency variations of the FM signal. Instead, the PSD estimate only has a single peak at a frequency of 190Hz. This frequency value corresponds to the average frequency of the FM signal ($186Hz$) and it goes to show that the `aryule` function treated the FM signal as stationary and carried out block-based estimate of the AR coefficient.

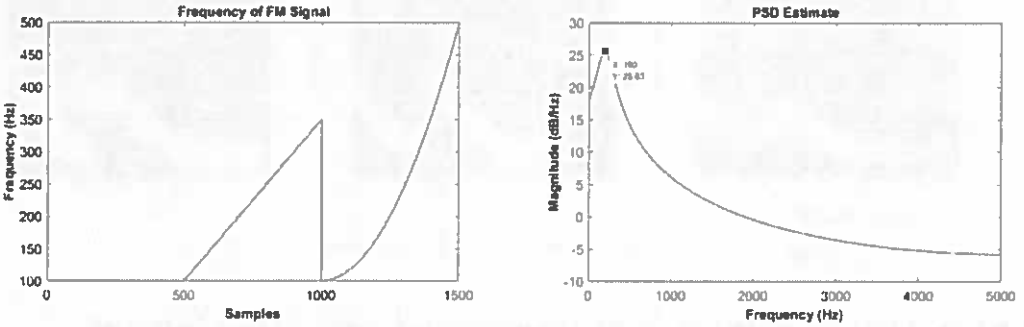
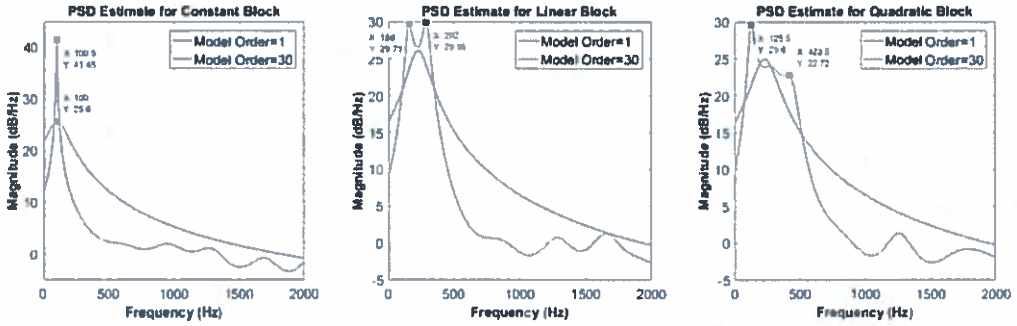


Figure 62: Left- Frequency Variation of FM signal. Right- PSD estimate of FM signal using an AR(1) model

Since the FM signal can be split into three blocks as mentioned previously, AR modelling can be done on the three different segments of the signal separately. AR models of order 1 and 30 are then used to model the FM signal in each block, and the PSD estimates obtained from the AR coefficients are shown in **Figure 63**. In the first block where the frequency is constant, both AR models are able to accurately predict the frequency of the signal, which is at $100Hz$. In the second block, since the signal is non-stationary, the PSD estimate is unable to reflect the changes in frequencies. However, the peak of the PSD estimate of the AR(1) model again corresponds to the average frequency in the block ($225Hz$), while the two peaks observed when an AR(30) model is used gives a very loose indication of the range of frequencies of the FM signal within the block. For the last block, again since the signal is non-stationary, the AR models are unable to predict PSD which captures the frequency variation in the signal. Again, the mean of the frequency ($234Hz$) can be obtained with the AR(1) model while the peaks of the AR(30) model is somewhat representative of the range of frequencies present. In general, block based analysis will be unable to capture the variations in frequencies of a non-stationary input signal. Other methods which utilise time-frequency analysis techniques have to be used in study non-stationary signals.

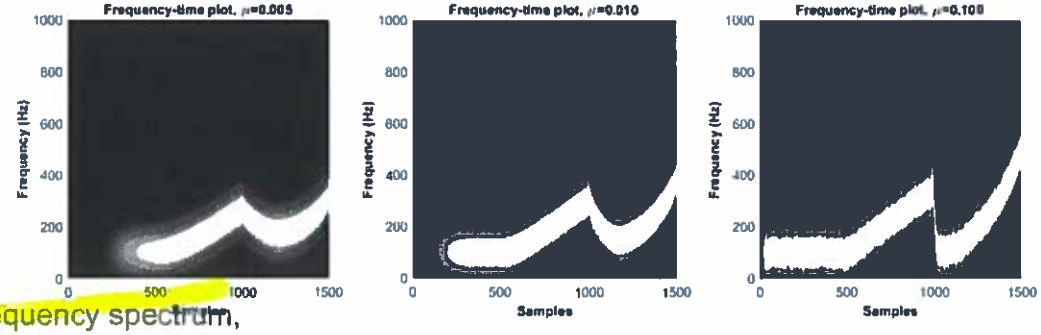


Appropriate graphs, even though not requested. They enrich the overall report.

Figure 63: PSD Estimates of the Different Segments of FM Signal

b) The CLMS algorithm is used to estimate the AR coefficient of the FM signal, and at each time instant the frequency spectrum is obtained using the AR coefficient. The frequency spectrum is then plotted together with time to produce a time-frequency spectrum. The time-frequency spectra obtained when μ is varied is shown in **Figure 64**. Unlike the AR spectra in a), the frequency variations can now be visualised using the time-frequency spectrum. For all values of μ, the general shape of the frequency variation resembles the theoretical frequency variation of the FM signal used. However, in the time-frequency spectrum, bands are observed instead of lines. This is due to the large main lobe width of the PSD estimate obtained when an AR(1) model is used, as seen in the left plot in **Figure 63**. If an AR(30) model is used instead, the AR spectrum will be sharper and the bands observed will also be thinner.

The closest resemblance to the theoretical frequency variation is achieved when μ is large (0.1). A large μ results in a faster learning rate and hence enables the CLMS algorithm to better adapt to abrupt changes in the signal. Hence, the sharp drop in frequency from the second block to the third block of the FM signal was only captured when $\mu = 0.1$. The effect of μ on the convergence rate can also be seen by observing the start of each time-frequency spectrum. The amount of samples needed before the frequency spectra can be observed increases as μ decreases. The bands in the time-frequency spectra also illustrates the relationship between μ and the steady state error. When μ is large, the bands have a very sketchy effect, and this is due to the large fluctuations of the CLMS at steady state. When μ is decreased, the bands becomes thinner and less diffuse about the edges as the steady state error decreases..



Clear time frequency spectrum, appropriate choice of parameters.

Figure 64: Time-frequency Spectrum using the CLMS algorithm with varying μ

4.3 A Real Time Spectrum Analyser Using Least Mean Square

a) The least squares problem is solved by minimising the following cost function

$$\min_{\mathbf{w}} \| \mathbf{y} - \hat{\mathbf{y}} \|^2 = \min_{\mathbf{w}} \sum_{n=0}^{w-1} |y(n) - \hat{y}(n)|^2 \tag{94}$$

This can be rewritten in vector form as

$$\begin{aligned} \| \mathbf{y} - \hat{\mathbf{y}} \|^2 &= (\mathbf{y} - \hat{\mathbf{y}})^H (\mathbf{y} - \hat{\mathbf{y}}) \\ &= (\mathbf{y}^H - \hat{\mathbf{y}}^H)(\mathbf{y} - \hat{\mathbf{y}}) \\ &= \mathbf{y}^H\mathbf{y} - \mathbf{y}^H\hat{\mathbf{y}} - \hat{\mathbf{y}}^H\mathbf{y} + \hat{\mathbf{y}}^H\hat{\mathbf{y}} \end{aligned} \tag{95}$$

Using $\hat{\mathbf{y}} = \mathbf{F}\mathbf{w}$, the derivative of Eq. (95) is then taken with respect to $\mathbf{w}$ since it has to be minimised with respect to $\mathbf{w}$

$$\begin{aligned} \frac{\partial \|\mathbf{y} - \hat{\mathbf{y}}\|^2}{\partial \mathbf{w}} &= \frac{\partial}{\partial \mathbf{w}} (\mathbf{y}^H\mathbf{y} - \mathbf{y}^H\mathbf{F}\mathbf{w} - \mathbf{w}^H\mathbf{F}^H\mathbf{y} + \mathbf{w}^H\mathbf{F}^H\mathbf{F}\mathbf{w}) \\ &= 0 - \mathbf{y}^H\mathbf{F} - \mathbf{y}^H\mathbf{F} + \mathbf{w}^H(\mathbf{F}^H\mathbf{F} + \mathbf{F}\mathbf{F}^H) \\ &= 2\mathbf{w}^H(\mathbf{F}\mathbf{F}^H) - 2\mathbf{y}^H\mathbf{F} \end{aligned} \tag{96}$$

To minimise the cost function, the derivative is set to zero

$$0 = 2w^H(FF^H) - 2y^HF$$
$$w^H(FF^H) = y^HF \tag{97}$$

Finally, the transpose is taken on each side

$$\mathbf{F}^H\mathbf{F}\mathbf{w} = \mathbf{F}^H\mathbf{y}$$
$$\mathbf{w} = (\mathbf{F}^H\mathbf{F})^{-1}\mathbf{F}^H\mathbf{y} \quad (shown) \tag{98}$$

The relationship between Eqn. (98) and the DFT formula can be seen by first looking at the inverse DFT of a signal $\hat{x}(n)$

$$\hat{x}(n) = \frac{1}{N}\sum_{k=0}^{w-1} X(k)e^{j\frac{2\pi nk}{N}} \tag{99}$$

This can be expressed in matrix notation,

$$\begin{bmatrix} \hat{x}(0) \\ \hat{x}(1) \\ \vdots \\ \hat{x}(N-1) \end{bmatrix} = \begin{bmatrix} g_{0,0} & g_{0,1} & \cdots & g_{0,N-1} \\ g_{1,0} & g_{1,1} & \cdots & g_{1,N-1} \\ \vdots & \vdots & & \vdots \\ g_{N-1,0} & g_{N-1,1} & \cdots & g_{N-1,N-1} \end{bmatrix} \begin{bmatrix} X(0) \\ X(1) \\ \vdots \\ X(N-1) \end{bmatrix} \tag{100}$$

where $g_{nk} = \frac{1}{\sqrt{n}}e^{j\frac{2\pi nk}{N}}$. In compact notation, Eq. (100) can be written as $\hat{\mathbf{x}} = \mathbf{G}\mathbf{X}$, and this resembles the formula $\hat{\mathbf{y}} = \mathbf{F}\mathbf{w}$, with $\hat{\mathbf{x}} = \hat{\mathbf{y}}$, $\mathbf{G} = F$ *and* $\mathbf{w} = \mathbf{X}$. Hence, the DFT formula can be written as $\mathbf{X} = (\mathbf{G}^H\mathbf{G})^{-1}\mathbf{G}^H\mathbf{x}$, and it can be seen that the DFT is a least square solution. This means that the Fourier coefficients, $\mathbf{X}$, obtained from DFT allows the estimation of $\hat{\mathbf{x}}$ which minimises the squared error between $\hat{\mathbf{x}}$ and $\mathbf{x}$. Furthermore, if the matrix $\mathbf{F}$ as defined in the assignment is used, Eq. (98) is returning the coefficients of each frequency bin, which is exactly the same as doing DFT (in fact, $\mathbf{G} = \frac{1}{\sqrt{N}}\mathbf{F}$).

b) Using the matrix representation of the DFT process, $\mathbf{X} = (\mathbf{G}^H\mathbf{G})^{-1}\mathbf{G}^H\mathbf{x}$, it can be seen that the DFT operation obtains a set of coefficients $\mathbf{X}$ by projecting the original signal $\mathbf{x}$ onto the vector space spanned by the columns of $\mathbf{G}$. Since the columns of $\mathbf{G}$ form an orthonormal basis of N complex exponentials, by projecting $\mathbf{x}$ onto the column space of $\mathbf{G}$, there is a change of basis from time to frequency. The vector $\mathbf{X}$ now contains information regarding how much of each complex exponential there is in $\mathbf{x}$. Thus, by executing $\hat{\mathbf{x}} = \mathbf{G}\mathbf{X}$, an approximation of $\mathbf{x}$ which is based on the orthonormal basis can be obtained. This corresponds to the inverse DFT process.

c) The DFT-CLMS algorithm is implemented in Matlab, and applied to the FM signal from Section **4.2 a)** with a step size of $\mu = 1$. The magnitude of the weight vector obtained is plotted to obtain a time-frequency spectrum and this is shown in **Figure 65**. The shape of the spectrum matches the ideal time-frequency response of the FM signal, however, instead of just the outline of the shape obtained as seen in the spectrum from the AR-spectrum analyser (**Figure 65**), there appears to be streaks across the spectrum, suggesting that the frequency components corresponding to these lines are everlasting in the FM signal, which is not the case. This problem arises due to the adaptive nature of the weights. Since the weights are not recalculated at every time instance but are instead updated at each iteration using the error signal. In the case of the AR spectrum analyser, this did not appear to be a problem since there was only 1 coefficient which needed to be updated. However, in the DFT-CLMS algorithm, there are a total of N coefficients which needs to be updated. As a result, it takes a significantly longer amount of time for the effects of the error to be propagated down to a specific weight because there are so many of them. This means that the value of the weights corresponding to their respective frequency bins are not changing fast enough once they have first been updated, leading to the streaks observed. Furthermore, although the time-frequency spectrum of the DFT-CLMS algorithm does not resemble the theoretical response as much as the spectrum from the AR spectrum analyser does, it can be seen that the DFT-CLMS offers way better frequency resolution than its counterpart. This can be seen from the presence of lines instead of the thick bands in the time-frequency spectrum.

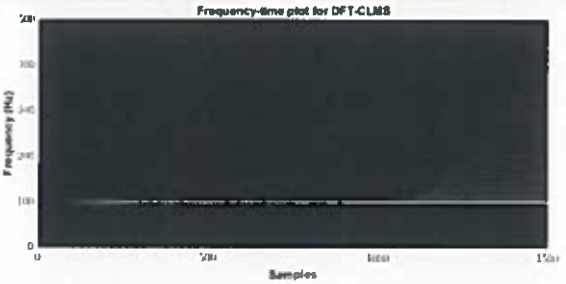


Figure 65: DFT-CLMS Time-frequency Spectrum of FM Signal

Excellent explanation of DFT CLMS and its limitations, points often missed by colleagues.

To improve the performance of the DFT-CLMS algorithm in producing an accurate time-frequency spectrum, the slow updating rate of the filter weights need to be addressed. One way of achieving this is to introduce a forgetting factor such that the previous weight vector will play less of a part in the determination of the current weight vector. This can

easily be done by implementing the leaky LMS algorithm. Recall that in the leaky LMS algorithm (Eq. 38), in calculating the next weight values, the current weight values are multiplied by a factor of $1 - \mu\gamma$. γ here can be seen as the memory of the filter. If γ is large, the filter has very short memory and previous instances of the weight vector will only affect the current weight values very slightly (when $\gamma = 0$, the filter has no memory and each weight vector is calculated at every time instance). When γ is small however, the current weight can be seen as a weighted sum of all the previous weights and thus, it will be take a long time before the error can propagate down the system. The leaky DFT-CLMS algorithm is implemented and the time-frequency spectrum is plotted for different values of γ in **Figure 66**. It can be seen that when γ is small (0.005), the streaks are present but as γ is increased, the streaks begin to disappear but now the lines in the spectrum becomes thicker and thicker due to the increase in error due to the increased fluctuations and uncertainty since essentially a smaller number of input samples are now used to calculate the weights. The leaky CLMS algorithm is thus able to provide a more accurate time-frequency spectrum than the AR method if γ is chosen properly, and is this case $\gamma = 0.01$ produced the best spectral plot that resembles the original frequency variations of the FM signal.

Bonus points acquired!!!!

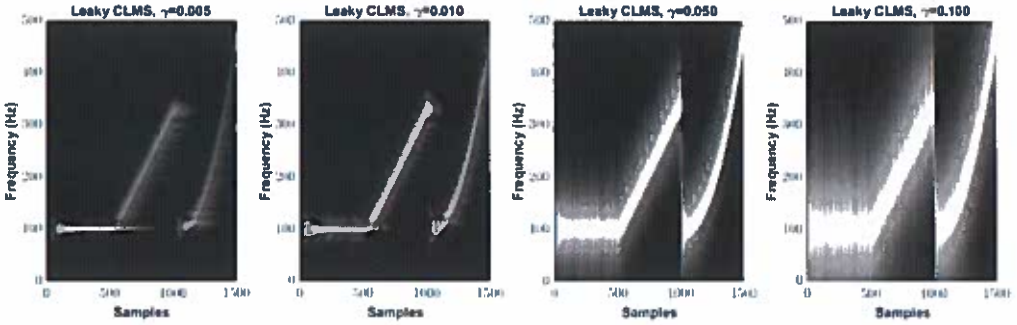
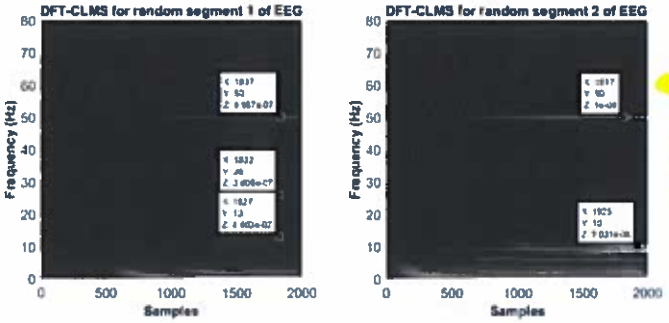


Figure 66: Leaky DFT-CLMS Time-frequency spectrum for varying γ

d) The DFT-CLMS algorithm is used to analyse the POz signal used previously in **Section 1.4**. The two segments of length 2000 of the POz signal are used to obtain the time-frequency spectrum, which are shown in **Figure 67**. The first segment is taken near the start of the experiment, while the second segment is taken near the end of the experiment. In the first segment of the EEG used, the first and second harmonic of the SSEVP can be easily identified in the time-frequency spectrum at $13Hz$ and $26Hz$ respectively due the presence of strong red streaks. The mains interference of $50Hz$ can also be strongly identified. In the second segment of the EEG used, the first and second harmonic of the SSEVP can no longer be easily identified. However, in its place, the alpha rhythm which causes the bright red region in the $8 - 10Hz$ range can be discerned. Since this EEG segment was taken towards the end of the experiment, the fatigue of the subject, coupled with the decrease in focus on the flashing stimulus resulted in a spectrum where the alpha-rhythm is present while the SSEVP components are not.



Good final representation of spectrum on EEG data.

Figure 67: Time-frequency Spectrum for EEG Data